

The natural-proofs barrier against data structure lower bounds

Michal Koucký Bruno Loff Tulasimohan Molli Mike Saks

Abstract

Consider a data structure problem with possible data coming from a set $\mathcal{D}$, queries coming from a set $\mathcal{Q}$, and in the dynamic case updates coming from a set $\mathcal{U}$. Then, the current state of the art in data structure lower bounds is $t = \tilde{\Omega}(\log |\mathcal{Q}|)$ for static data structure problems, and $\max(t_q, t_u) = \tilde{\Omega}((\log n)^2)$ where $n = \max(|\mathcal{Q}|, |\mathcal{U}|, \log |\mathcal{D}|)$ for dynamic.

We port Razborov and Rudich’s natural-proofs framework to the setting of static and dynamic data structures in the cell probe model, in a way that strongly suggests this state of the art is unlikely to be improved anytime soon. A similar direction was recently taken also by Korten, Pitassi and Impagliazzo (FOCS 2025) who look at static data structure lower bounds in a different regime of parameters. Our contribution is:

- We define notions analogous to pseudorandom functions (PRF). We call these primitives *local PRFs*, in the context of static data structures, and *local and locally updatable (LLU) PRFs*, in the context of dynamic data structures.
- We then formulate cryptographic conjectures, namely, that secure local PRFs and secure LLU PRFs exist, precisely at the frontier where we are no longer able to prove static, respectively dynamic, data structure lower bounds. If these conjectures are true, it follows that the current state of the art in data structure lower bounds cannot be improved by a natural proof.
- We show that (almost) every single known data structure lower bound proof is a natural proof, by surveying all lower bounds in the literature (known to us). (The only exception is proofs based on lifting theorems.)
- It follows that, if our cryptographic conjecture is true, then all known lower bound proof techniques (minus the one exception) are unable to improve upon the state of the art. (We also attempt to address the exception.)
- Further, we provide concrete candidate constructions for our two pseudo-random primitives. We conjecture that our constructions are secure for parameters just above the state-of-the-art lower bounds.
- We also show that, whether or not they are secure, our candidate PRFs at least satisfy the natural properties appearing in all (but one) known proofs.
- So if one is interested in improving upon the state of the art in static or dynamic data structure lower bounds, one must either find a non-natural method of proving such lower bounds (no such method currently exists), or one may as well begin by trying to break our PRF candidates.

Contents

1	Introduction	1
1.1	Key challenges for data structure lower bounds	1
1.2	Barriers to data structure lower bounds	3
1.3	Our contribution	5
2	Preliminaries	11
2.1	Notation	11
2.2	Decision trees	12
2.3	Data structures in the cell-probe model	12
2.4	Communication Complexity	15
3	What is as a natural proof?	16
4	Proposal: A natural proofs barrier against static data structure lower bounds	19
4.1	Local PRFs, and a natural proofs barrier against static data-structure lower-bounds	20
4.2	Two candidates for local PRFs	22
4.3	PRGs vs. PRFs, and why it matters	25
5	Thesis: All known static data structure lower bounds are natural	27
5.1	The richness method	27
5.2	Round Elimination	31
5.3	Cell sampling arguments	33
5.4	Proofs via CNF refutation techniques	34
5.5	The single exception we know: Lifting theorems	36
6	Proposal: A natural proofs barrier against dynamic data structure lower bounds	38
6.1	LLU PRGs, and a natural proofs barrier against dynamic data-structure lower-bounds	39
6.2	A candidate RLLU generator for Conjecture 6.5	41
6.3	Pseudorandom properties of our PRF	44
7	Thesis: All known dynamic data structure lower bounds are natural	47
7.1	The chronogram method	47
7.2	Patrascu and Demaine’s information transfer technique	50
7.3	Non-adaptive data-structures: Brody and Larsen’s Lower-bounds	52
7.4	Semi-adaptive data-structures: Ko and Weinstein’s Lower-bounds	55
7.5	Chronogram + Cell Sampling	55
7.6	Chronogram + Weak One-way Simulation	59
A	k-wise independent LDPC code	65
	Bibliography	66

ERRATA
(CURRENT VERSION: DECEMBER 27, 2025)

1. An hour before submission we discovered what seems like a genuinely new new data-structure lower-bound technique, appearing in the paper “New Sampling Lower Bounds via the Separator” by Emanuele Viola. We don’t expect any surprises (we predict the proof will naturalize), but we didn’t check yet.
2. After inspecting this paper, Emanuele Viola pointed out that one of the barriers mentioned in the introduction [DGW19, RR20] which shows that new static data-structure lower-bounds imply the existence of certain rigid matrices, is no longer a barrier, since matrices with this rigidity have been proven in the meantime (he cites Alman and Chen, Efficient Constructions of Rigid Matrices using an NP oracle, and Bhangale, Harsha, Paradise and Tal. Rigid matrices from rectangular PCPs).

We did not know of these two papers. It follows that Viola’s barrier [Vio19] is the only other barrier “left standing”.

1 Introduction

A major goal of complexity theory is to establish lower bounds on the complexity of explicit functions in various computational models. Despite decades of effort, and many beautiful results, there is a huge gap between the lower bound results that have been proved, and what is widely believed to be true. Given this gap, researchers have sought to understand why existing proof techniques seem incapable of proving stronger lower bound results. This has led to the formulation of so-called “barrier” results of the following form: For a given computational model, proving a strong lower bound for an explicit function using a specified class of proof techniques is either impossible, or would have an unexpected consequence. The “*relativization*” barrier for Turing machine computation, due to Baker, Gill and Solovay [BGS75], says that it is impossible to separate P from NP using a proof technique that relativizes (applies to computation relative to an arbitrary oracle). The “*natural proofs*” barrier for circuit complexity, due to Razborov and Rudich [RR94], introduces the notion of natural proof for a circuit lower bound (which essentially encompasses all known circuit lower bounds) and shows that the existence of a natural proof that separates P from NP would imply that there are no pseudo-random generators which are secure against 2^{n^ϵ} -time distinguishers, thus contradicting a widely believed cryptographic assumption.

1.1 Key challenges for data structure lower bounds

Data structure lower bounds encompass a major subarea within complexity theory, and the cell probe model is the standard non-uniform model for studying data structure lower bounds. There are many static and dynamic data structure problems such that the best known implementations have much higher complexity than the best known lower bounds. In some cases we even have conditional lower bounds, which makes us believe that the known implementations are optimal or nearly-optimal. Our goal is to understand what are the obstacles to proving tight unconditional lower bounds.

In the static setting input data (the “*database*”) is to be pre-processed into a data structure to be stored in memory, in such a way that we should be able to quickly answer certain queries about the data. The data structure should have some size s in memory, and any given query should be answered by probing at most t memory locations: t is called the *query time*. The memory used s is to be substantially smaller than the total number of possible queries, since, when s is larger than the number of queries, a trivial implementation can simply pre-compute and store the answer to every query, so the query time becomes 1. The question here is what is the trade-off between the query time t and the space s used to represent the data in memory. Often one assumes some natural space bound, usually close to the information-theoretic minimum, and asks about the query time in that setting.

A prototypical example of a static data structure problem is the orthogonal range counting problem. Here the database represents a set of n points from some discrete

space of some fixed dimension $d \geq 1$, e.g. $[n]^d$. Each query specifies a d -dimensional rectangle (axis aligned) in that space and expects as the answer the number of points from the set that lie in the specified rectangle. Here the number of databases is n^{dn} and the number of queries is n^{2d} . The typical space we are interested in is $s = n \cdot \text{polylog}(n)$. The best known lower bound on query time for this problem for fixed $d \geq 2$ is $\Omega(\log n / \log \log n)$ [Pät07] while the best upper bound is $O((\log n / \log \log n)^{d-1})$ [Wil85, JMS04, CW13]. Is there a reason for this large gap between the known lower bounds and upper bounds?

In the dynamic data structure setting the stored database can be modified (*updated*) using an allowed set of updates. Here we are interested not only in the query time t_q but also in the update time t_u — the time needed to perform an update of the memory representation of the data. There is usually an implicit bound on the space used by the data structure: If m updates are enough to potentially take the data structure from any database to any other database then the space used by the data structure is reasonably at most $O(m(t_u + t_q))$. So in the dynamic setting we usually study the trade-off between the query time and update time, assuming a reasonable space bound.

A prototypical example of such a data structure problem is the graph reachability problem. Here, each database is a directed graph on n vertices, we can update the graph by adding an edge or removing an edge, and we can query whether a vertex u is reachable from a vertex v . All known implementations require either t_u or t_q be polynomial in n , i.e., n^δ for some fixed $\delta > 0$, while the best lower bound establishes that either t_u or t_q must be $\tilde{\Omega}(\log^{3/2} n)$ [LY23]. Hence there is an exponential gap between the lower bound and upper bounds. So is there a reason for this wide gap?

We will argue that, in both the static and dynamic cases, there is a *natural proofs barrier* against proving stronger lower bounds.

For the purpose of lower bound proofs we abstract data structure problems as follows: a data structure problem is specified by a 4-tuple $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U})$ (called a *frame*) and a $\mathcal{Q} \times \mathcal{D}$ matrix f (called a *table* over $\mathcal{F}$). Here $\mathcal{D}$ is a set of *databases*, $\mathcal{Q}$ is a set of *queries*, $\mathcal{A}$ is a set of *answers*, $\mathcal{U}$ is a set of *updates* each of which is a function that transforms one database into another. The table entries $f(q, x) \in \mathcal{A}$ specify the desired output of query q on database x . In an *implementation* of $(\mathcal{F}, f)$ with space s , word size w and time t , databases are represented by arrays of s memory cells, each taking a value in $\{0, 1\}^w$ and each query and update is implemented by a decision tree of depth t that accesses cells and branches on the value in the cell, where the output value at a leaf of the decision tree associated to a query is the answer to the query, and the output value at a leaf of the decision tree associated to an update specifies updated values for the cells that were read along the path to the leaf. (See Section 2 for more details.) In the static setting the set $\mathcal{U}$ is empty and is omitted.

In the static setting, the question of interest is: For a given s and w , what is the minimum t so that a particular data structure can be implemented with query decision trees of depth at most t ?

If the space s is sufficiently large, namely at least $|\mathcal{Q}| \log |\mathcal{A}| / w$, there is a trivial (and essentially optimal) implementation with time $t = \lceil \log |\mathcal{A}| / w \rceil$ by storing the answer to

each query (encoded in binary) in a separate block of $\log |\mathcal{A}|/w$ cells. On the other hand, if the space is too small, assuming that no two databases give the same answers on all queries, the problem can't be solved at all for information theoretic reasons. So in the static setting the interesting range of space is $\log |\mathcal{D}|/w \leq s \leq |\mathcal{Q}| \log |\mathcal{A}|/w$.

There is a large body of research proving lower bounds on time for static data structure problems. If the space is limited to be equal to the information theoretic bound, or only very slightly larger (e.g. a $1 + o(1)$ factor greater than the information-theoretic minimum), then some strong lower bounds on query time have been proved (see, e.g. [GM07, Pă09, CKL18]). However, in the more important case that the space is not so constrained, the best lower bounds known do not exceed $\log |\mathcal{Q}|$. Indeed $\log |\mathcal{Q}|$ is an inherent upper limit on any such lower bound that is proved using the connection to communication complexity that was observed in the seminal paper of Bro Miltersen et al. [MNSW98] (See Section 5 for details.)

On the other hand, there are many data structure problems known where the best available implementations require significantly more time per query than $\log |\mathcal{Q}|$. Thus the following problem summarizes where research on lower bounds for static data structures is currently stuck:

Open Problem 1.1. *For some explicit static data structure problem, prove that the worst case time of a query exceeds $\log |\mathcal{Q}|$ for any amount of space that not too close to (e.g. it is at least twice) the information theoretic minimum.*

Two well-known candidate problems for achieving the goal stated in this problem are the orthogonal range counting problem mentioned above and the vector-matrix-vector multiplication problem (VMV) defined in Section 4.

In the dynamic setting the best lower bounds we have for explicit problems do not exceed $\log |\mathcal{Q}| \log |\mathcal{U}|$, but there are many data structure problems for which the current best upper bounds are $|\mathcal{Q}|^\beta + |\mathcal{U}|^\beta$ for some $\beta > 0$. This is the case for the dynamic graph reachability problem.

Open Problem 1.2. *For some explicit dynamic data structure problem, prove that the worst case time of an update or query exceeds n^ϵ where $n = \max(|\mathcal{Q}|, |\mathcal{U}|, \log |\mathcal{D}|)$.*

1.2 Barriers to data structure lower bounds

The situation with data structure lower bounds thus seems similar to that for circuit lower bounds: the best lower bounds we can prove fall far short of what we expect. Over the past decade several papers have appeared that attempt to identify barriers to significantly improving the existing lower bounds.

Viola [Vio19] showed that any static data structure lower bound improving the state of the art would imply a strongly super-linear lower bound on the wire complexity of an explicit function. This barrier applies to dynamic data structure problems, also, if they are of *decomposable* type. One caveat is that this barrier only works in the setting where the space s is close to maximum, namely, $s = \Omega((\log |\mathcal{Q}|)^{1-\epsilon})$.

Around the same time, Dvir, Golovnev and Weinstein [DGW19] showed that any $\omega(\log^2 |\mathcal{Q}|)$ static data structure lower bound would imply new semi-explicit constructions of rigid matrices (matrices that can not be expressed as the sum of a low rank matrix and a sparse matrix). Rigid matrices were introduced by Valiant as an approach to proving explicit lower bounds in algebraic circuit complexity, and constructing explicit (or even semi-explicit) rigid matrices is a well known unsolved problem. A follow-up paper by Ramamoorthy and Rashtchian [RR20] showed that a lower bound of $\omega(\log |\mathcal{Q}|)$ on the vector-matrix-vector (VMV) problem, which is widely believed to have complexity $\tilde{\Omega}((\log |\mathcal{Q}|)^2)$, would also imply new constructions of rigid matrices. These barrier results hold even if the lower bound is only proven against non-adaptive data structures with close to minimal space.

In the above papers, the “barrier” identified is that making progress on a data structure problem would yield explicit or semi-explicit constructions of combinatorial objects which have defied all previous efforts to construct them. This is very different from the barrier identified by Razborov and Rudich.

Earlier this year, Korten, Pitassi, and Impagliazzo [KPI25] identified a barrier for static data structure lower bounds that much more closely relates to the barrier identified by Razborov and Rudich. They adapted the natural proofs framework to the setting of static data structure lower bounds and proved that a natural proof of sufficiently strong data structure lower bounds would prove the non-existence of certain pseudo-random generators which are believed to exist.

For example they show that, under standard assumptions about the existence of NC_0 pseudo-random generators, for every constant $\epsilon > 0$ there exists a constant $t(\epsilon)$ such that no natural proof can prove a lower bound of $t(\epsilon)$, for data-structure problems where $|\mathcal{Q}| \geq |\mathcal{D}|^\epsilon$, against implementations that are allowed to use space $|\mathcal{Q}|^\epsilon$. This is a strong result, but does not directly pertain to [Open Problem 1.1](#) where the goal is to prove stronger lower bounds on time when the space used is not too large, when compared with the information theoretic bound. Unfortunately, the parameter regime to which their barrier applies excludes most natural problems, such as the ORC and VMV problems. For example, for the ORC problem, a natural proof by Patrascu [Pät07] shows a super-constant $t = \Omega(\frac{\log n}{\log \log n})$ lower-bound. So this lower-bound proof goes beyond the barrier raised in [KPI25]. What is happening here? Well, from Patrascu’s proof we can get a distinguisher which can break all NC_0 PRGs. This does not contradict the standard cryptographic conjectures on NC_0 PRGs, because the complexity of the distinguisher (2^{n^2}) is higher than the conjectured security of NC_0 PRGs ($2^{n^{o(1)}}$). Patrascu’s proof shows that, in order to get a natural proofs barrier that holds in all parameter regimes, it cannot be established via NC_0 PRGs as done in [KPI25]. Instead, we establish the barrier by constructing PRFs directly, and our PRFs will, in particular, fool the distinguisher arising from Patrascu’s proof. We explain the difference between these two approaches in [Section 4.3](#).

Furthermore, Korten, Pitassi, and Impagliazzo have also shown that some of the lower-bound techniques used in the literature are natural proofs. But they only showed this for a small subset of all known proofs. And, in order to fully establish a natural proofs

barrier, one should aim to show that *every* existing static data structure lower-bound proof can be naturalized. Otherwise, one might still argue that one of the techniques which was not treated could conceivably be used to prove a new lower bound beyond the state of the art. To rule this out, then, one should strive to cover every single technique that was ever used in the literature. Finally, the approach of [KPI25] does not address dynamic data structure lower bounds and [Open Problem 1.2](#).

Here, we will attempt to extend their work to address all of these shortcomings.

1.3 Our contribution

Our goal is to show that the natural proofs paradigm gives us a barrier for pushing the frontier of data structure lower bounds, and in particular, in solving Problems [1.1](#) and [1.2](#).

Lower-bound proofs can be thought of as identifying a certain “combinatorial” property of the data structure problem that makes the problem hard. For the lower-bound to work, this property should be such that any function for which it holds needs large time/space to solve the problem. (This is referred to as *usefulness* of the property.) In most proofs, whether a data structure problem possesses the property can be decided by an efficient algorithm that inspects the truth-table of the data structure problem. (This is referred to as *constructivity* of the property.) Furthermore, the kind of property that shows up in most proofs is such that most data structure problems will have it, meaning, a randomly selected truth-table will satisfy the property with high probability. (This is referred to as *largeness* of the property.)

A lower bound proof which uses a useful, constructive, large property is called a *natural proof* and such a property is called a *natural property*. (In many cases the property used in the proof may not be natural, but can be slightly modified to be made natural, and we still call this a natural proof.) The issue with such natural properties is that they contradict the existence of *pseudo-random function generators*. Let us explain.

In our framework, when considering a particular frame $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U})$, each table over $\mathcal{F}$ (that is, each $\mathcal{Q} \times \mathcal{D}$ matrix with entries in $\mathcal{A}$) specifies the desired behavior of queries and thus represents a different data structure problem. We refer to a probability distribution over tables as *function generators*.

Now, suppose that we have managed to show that a given data structure problem (a table f) cannot be implemented in space s , word size w and time t , by showing that f satisfies some natural property Φ . Now consider function generators supported on *easy tables*, namely, every function f in the support of the generator *can* be implemented in space s , word size w and time t . We refer to such distributions as *local function generators*. Because of the usefulness condition, the probability that the output of such a generator satisfies the natural property is 0. (We emphasize that the notion of usefulness depends on the particular choice of s , w and t under consideration.) However, because of the largeness requirement, a non-trivial fraction of all tables satisfy Φ . Hence, the natural property serves as a *distinguisher* between the output of any local function generator, and the uniform distribution over all tables. This distinguisher is efficient, by constructivity.

By contrapositive, if for some s, w and t , there *does* exist a local function generator that *fools* all efficient distinguishers, i.e., can not be distinguished from the uniform distribution by an efficient algorithm — we will call such a generator a *local PRF* — then we must conclude that no natural property can exist, that is, there is no property that simultaneously satisfies usefulness, largeness and constructivity. And hence there exists no natural proof establishing a lower-bound with parameters s, w, t .

This is a straightforward adaptation of the natural proofs paradigm, which was proposed by Razborov and Rudich for circuit lower bounds, to data structures, and this was used in [KPI25]. In [KPI25] the authors do not explicitly consider local function generators. Instead they consider only a very restricted family of local function generators which they call locally computable pseudo-random generators. These have the advantage that the existence of such generators that fool natural properties follows from standard assumptions in the theory of pseudo-randomness. However, they have the disadvantage that they only are able to rule out natural proofs for data structure lower bounds where the space is large. In particular, their barrier results do not apply to [Open Problem 1.1](#). They also do not address dynamic data structures.

Our framework incorporates the update structure into the underlying frame so it applies both to the static and dynamic case. To rule out natural proofs for a specific s, w, t , it suffices to identify local pseudo-random function generators that fool all natural properties (with respect to the given s, w, t). Considering the full class of such generators (rather than the restricted version considered in [KPI25]) provides more flexibility to construct such a local PRG, but this added flexibility has the drawback that we don't know how to apply standard assumptions about PRGs to show that such generators exist.

What we do is to propose two concrete local function generators, one for the static case and one for dynamic. We conjecture that the generator for the static case fools natural properties that seek to answer [Open Problem 1.1](#) and that the generator for the dynamic case fools natural properties that seek to answer [Open Problem 1.2](#).

Because these generators seem to differ in significant ways from generators that have been considered previously, we are not able to relate their security to standard assumptions about pseudo-random generators. Instead we provide evidence of their security by proving that they fool the natural properties that arise in nearly all of the existing lower bound proofs in the literature.

One key insight that allows us to address the dynamic case is to restrict attention to so-called *history independent* implementations of data structure problems. A history independent implementation has the property that the memory representation of data is independent of the sequence of updates that lead us to the current database. Hence each database has a unique representation in memory. This notion was defined originally in the context of cryptographic applications [NT01] but found interesting applications in design of usual data structures, e.g. [BCF⁺22, BFGK24].

History independence puts additional constraints on the space of possible implementations so proving lower bounds on history independent implementations is potentially easier than proving lower bounds for general implementations. We will argue that even if

we restrict to history independent implementations, proving lower bounds for the problems in question require overcoming natural proofs barrier.

For the remainder of this introduction we give an overview of our generators.

Static pseudo-random function generator. For the static case we will consider two natural pseudo-random function generators that combined together will give our candidate pseudo-random function generator. For simplicity let us assume $w = 1$, so we are dealing with bit-probe model. Let us set $\mathcal{D} = \{0,1\}^n$, $|\mathcal{Q}| = 2^{o(n)}$, $\mathcal{A} = \{0,1\}$, $t = \omega(\log n + \log |\mathcal{Q}|)$ and $s = \omega(nt)$. The pseudo-random function generator will take as its seed a table of $|\mathcal{D}| \times s$ bits specifying the memory representation of each database, and a description of $|\mathcal{Q}|$ decision trees, where for each $q \in \mathcal{Q}$, the q -th tree represents the algorithm that answers query q by querying the memory representation of the current database at t positions. The length of the description of each of the decision trees is bounded by $O(\log s \cdot 2^t)$. Assuming that for some $c > 1$ and $\delta < 1$ we have $t \leq (\log |\mathcal{D}|)^\delta$, $s \leq |\mathcal{Q}|^\delta$ and $s \leq (\log |\mathcal{D}|)^c$, the seed length is at most $O(|\mathcal{D}| \cdot |\mathcal{Q}|^\delta + |\mathcal{D}|^{\delta+o(1)} \cdot |\mathcal{Q}|)$ while the output size is $|\mathcal{Q}| \cdot |\mathcal{D}|$. Hence the seed of the pseudo-random generator is shorter than its output.

One could hope that this pseudo-random function generator is going to be hard to distinguish from random. One can verify that the output restricted to a particular database, i.e. the query truth-table for that database, is uniformly random. Unfortunately, any row in the table f output by this generator will be $2^{-O(t)}$ -biased, which is much more than the bias of a randomly-chosen function, hence this local function generator can be distinguished from random.

To remove the bias we will combine this generator with another one. Our second pseudo-random function generator is based on low-density parity-check matrix of a binary code. We consider a LDPC matrix with $|\mathcal{Q}|$ rows and s columns, where each row has at most t ones. We fix such a matrix P and the seed to the generator will be the $|\mathcal{D}| \times s$ memory representation of all databases. The output of the generator for a database x and query q is the inner product of the memory representation of x with the q -th row of P . Since the row contains at most t ones, the output can be determined by a decision tree of depth t .

One can show that the output of the second pseudo-random function generator contains no large monochromatic rectangle with high probability. So it exhibits certain pseudo-random properties. However it can still be distinguished by a rank test from a truly random.

We take as our candidate pseudo-random function generator the bit-wise XOR of the two pseudo-random generators from above applied on the same seed. (Technically this pseudo-random function generator will have a query complexity $2t$ but one can adjust the parameters accordingly.) Details of the construction are given in [Section 4.2](#).

In [Section 5](#) we will show that almost every proof technique used to prove static data structure lower bounds is natural. That means that almost every known technique produces an efficient distinguisher which separates easy functions (below the proved lower bound) from random functions. We will show that our local PRF candidate fools every

one of these distinguishers. We only found two exceptions. The first is lifting theorems — here it seems that the natural proofs barrier simply does not apply, as has been observed already in the work on the KRW conjecture [KRW95], which precisely seeks to circumvent this barrier. The second non-naturalizing proof is a proof in [KPI25] which uses a property of the table which they call being γ -biased. This property is *not* efficiently computable, nor does there seem to be any obvious way of making it efficiently computable. Nonetheless, we will show that our PRF also fools this hardness-implying property, which shows that the proof is “essentially natural” (see Definition 3.3).

We have tried to break our candidate pseudo-random function, so far with no success. Perhaps they can be broken, or perhaps they are secure. One thing can surely be said: any proof of a static data structure lower bound better than $\omega(\log \log |\mathcal{D}| + \log |\mathcal{Q}|)$, with $s = \Omega((\log |\mathcal{D}|)^{2+\epsilon})$, will either not be natural, or will have to somehow break every single local generator, including the generators we propose. So one could either believe that our generators are secure, and try to conceive of how a data structure lower bound could avoid being natural, or we could believe that data structure lower bounds can be proven by natural proofs, in which case all local PRFs can be broken, and one might as well begin by attempting to break our candidate. In either case, the main point is that the natural proofs barrier offers a suggestion of what needs to be done next.

Dynamic pseudo-random function generator. In Section 6.1, we will define a notion of a generator, called a *local and locally updatable (LLU) generator*, and propose the conjecture that such a generator exists which is sufficiently secure to rule out natural lower bound proofs. As far as we know, this is the first fully general barrier result which applies to *dynamic* data structure lower bounds.

It should be said, however, that our level of confidence on the existence of a LLU PRG is not as high as in the static case. In fact, the notion of *locally updatable* will have two versions, a natural notion of locally updatable derived directly from the existence of a dynamic data structure implementation, and another, less stringent notion of locally updatable generator (Relaxed Local-and-Locally-Updatable, RLLU) derived from the existence of a *relaxed* implementation. The difference between implementations and relaxed implementations is as follows: in an implementation to a dynamic data structure problem, for each possible update $u \in \mathcal{U}$ there must exist a $\leq t_u$ -depth read-write decision tree T_u which transforms an encoding $E(x)$ to an updated encoding $E(u(x))$. The fact that T_u is low-depth implies a bound on the Hamming distance: $\Delta_H(E(x), E(u(x))) \leq t_u$. In a relaxed implementation, there does not need to exist such a decision tree T_u ; instead, the *only requirement* is that $\Delta_H(E(x), E(u(x))) \leq t_u$ for all $x \in \mathcal{D}, u \in \mathcal{U}$.

In Section 6.2, we construct a candidate RLLU generator which we believe is secure. At first, it might appear that we have relaxed the notion of a generator too much: indeed the requirement that the Hamming distance between $E(x)$ and $E(u(x))$ is at most t_u appears to be a very weak requirement, it gives no algorithmic guarantees. Nonetheless, in Section 7, we will show that *every single dynamic lower-bound proof in the literature is a natural proof*, which proves lower bounds not just against implementations, but also against relaxed implementations! So whatever difference there might exist between implementa-

tions and relaxed implementations, the known lower-bound techniques fail to make use of it.

The different possibilities are then as follows (with parameters matching the state-of-the-art in dynamic data structure lower bounds):

- Secure LLU generators exist, in which case no natural proof of dynamic data structure lower bounds is possible beyond the current state of the art.
- Or perhaps only secure RLLU generators exist (for example, our candidate), in which case a natural proof of dynamic data structure lower bounds might still be possible beyond the state of the art, but such a proof would have to (somehow, unlike all proofs in the literature) use something about the update decision-trees T_u , other than the fact that $T_u(E(x))$ is close to $E(x)$ in Hamming distance. This is a real possibility, and deserves further thought.
- Or perhaps RLLU generators don't exist, and only in this case could we use the known lower bound techniques to go beyond the state of the art. In this case, lower-bound provers may as well begin by breaking our candidate RLLU generator.

Here we give a high-level overview of the candidate generator for the case when databases come from $\{0,1\}^n$, each of the n update functions flips one of the bits of a database and the answer set A equals $\{0,1\}$. The generator will take as its seed an (implicit) description of memory representation of each database x , and also descriptions of decision trees answering the data structure queries from the memory representation of databases. The output of the pseudo-random generator is the truth-table of the associated data structure problem.

Each memory representation $E(x)$ consists of $\ell + 2 = \theta(\log n)$ blocks, each block $j = 0, \dots, \ell$ will be of length $(1 + \kappa)^j d$, for some suitable parameter $\kappa > 0$. Instead of specifying the content of $E(x)$ for each x , the seed will specify the action of i -th update on the memory representation, $i \in [n]$. Hence for each update $i \in [n]$ and block $j = 0, \dots, \ell$, the seed will describe $O(d)$ bits that are flipped by the i -th update in the j -th memory block. We will assume that blocks $0, \dots, \ell$ are all-0 for the database 0^n , so the memory representation of each x (except for the block $\ell + 1$) can be obtained by applying the relevant updates on that memory representation, i.e. XOR-ing them bit-wise together. The last, $(\ell + 2)$ -th, block will be of length d and will be specified explicitly by the seed of the pseudo-random generator. Hence, except for this very last block updates to all the other blocks can be performed by depth $O(d)$ read-write decision trees.

For each data structure query the seed specifies a random decision tree that along each path reads d random locations from each block $j = 0, \dots, \ell$ in the order of the blocks, and then reads the entire block $(\ell + 1)$. The decision tree outputs the label of the reached leaf which is selected from a balanced labeling of the leaves of the tree. Thus the query complexity of the tree is $(\ell + 2)d$.

In [Section 6.2](#) we give a more general definition also for non-boolean domains. In [Section 7](#) as we overview all lower-bound techniques in dynamic data-structures (that

we know of), and show that they are all natural, we will also attempt to show that our PRF fools the natural properties that arise from such proofs. We have managed to show this formally for all but one of the proof techniques. We do believe our PRF fools the remaining natural property, also, but we have not been able to prove it, yet.

Conclusion. We will establish a natural-proofs barrier for static and dynamic data-structures. The barrier seems to kick in exactly at the border $(\log |\mathcal{Q}|$ for static, $(\log n)^2$ for dynamic, where $n = \max(|\mathcal{Q}|, |\mathcal{U}|, \log |\mathcal{D}|)$) above which we don't know how to prove static or dynamic lower-bounds.

This seems to suggest that any further progress on data-structure lower-bounds is contingent on overcoming the same barrier which is preventing us from proving unconditional lower bounds in Boolean circuit complexity.

2 Preliminaries

It will not be possible to make much sense of this paper if one has never seen lower-bounds against data structures in the cell probe model. The definitions may seem a little arbitrary and overly formal if one has never seen them, and our overview of the known proofs is very terse, but our purpose here is not to familiarize. In this case, we can recommend two excellent video lectures by Huacheng Yu¹, or by Kasper Green Larsen², and a survey by Miltersen on cell probe complexity [Mil99].

2.1 Notation

$\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U}$ The sets of databases, queries, updates, and answers.

$|\mathcal{Q}|$ The number of queries.

$|\mathcal{D}|$ The number of databases.

$\mathcal{P}$ A data-structure problem, either static (Definition 2.2) or dynamic (Definition 2.8).

s The space used, meaning, the number of words in the encoding of the data.

w The number of bits in each word of the encoding.

t The query time of an implementation to a static data-structure problem (Definition 2.5).

t_q, t_u The query time and update time of an implementation for a dynamic data-structure problem (Definition 2.10).

T A decision tree.

E An map for encoding the data $E : \mathcal{D} \rightarrow (\{0, 1\}^w)^w$.

$\mathcal{F}, \text{Tables}(\mathcal{F}), f$ A frame, the set of tables of a given frame $\mathcal{F}$, and a table $f \in \text{Tables}(\mathcal{F})$ (Definitions 2.3 and 2.6).

$\mathcal{B}$ A complexity bound (Definitions 2.4 and 2.9).

S The size of a distinguisher (a circuit to distinguish random from pseudorandom inputs).

ℓ The number of epochs in a Chronogram argument.

U_i, n_i The sequence of updates performed in the i -th epoch, and its bitlength $n_i = |U_i|$.

k For k -wise independence and similar notions.

¹<https://www.youtube.com/playlist?list=PLKVCRT3MRed6Y0wYqbp0lWZaroWWUWAg>

²https://youtube.com/playlist?list=PLJWyfGi_JNUsyw2ZS1Xwp0GhciNGOKRj-

Comparison of the notation from [KPI25]. For readers familiar with the work of Korten, Pitassi and Impagliazzo [KPI25] we provide here a brief dictionary of our notation so that one could translate between them. For our notation the goal was to avoid the use of N and M , as their choice is somewhat arbitrary.

N vs. $|\mathcal{Q}|$ [KPI25] uses N for the number of queries, we use $|\mathcal{Q}|$.

M vs. $|\mathcal{D}|$ [KPI25] uses M for the number of databases, we use $|\mathcal{D}|$.

S vs. s [KPI25] uses capital S for the space used by an implementation, we use uncapitalized s . We use capital S for the complexity of a distinguisher.

w, t, f We both use the same notation for these parameters.

2.2 Decision trees

Definition 2.1. A decision tree over s -long $\{0, 1\}^w$ -words with output set $\mathcal{A}$ is a rooted, node-labeled and edge-labeled tree T , such that:

- Every non-leaf node v is labeled with an index $i_v \in [s]$, and has 2^w children $v_{0^w}, \dots, v_{1^w}$, so that the edge $v - v_z$ is labeled by $z \in \{0, 1\}^w$. We say that index i_v is probed at node v .
- Every leaf node v is labeled by an element of $\mathcal{A}$.

Given an element $y \in (\{0, 1\}^w)^s$, we write $T(y)$ for the label of the leaf obtained as follows: we start with v being the root, and then repeatedly set v to be the child $v_{y_{i_v}}$ (i.e. we follow the edge labeled by y_{i_v} ; we say that position i_v is probed at node v), until v is a leaf.

The depth or number of probes of T is the maximum number of edges of a root-to-leaf path.

A read-write decision tree is a decision tree T with output set $\mathcal{A} = (\{0, 1\}^w)^{\leq s}$, and such that every leaf node at depth d is labelled by an element of $(\{0, 1\}^w)^d$. Such a decision tree naturally defines a map, which we denote by $\bar{T} : (\{0, 1\}^w)^s \rightarrow (\{0, 1\}^w)^s$: if on input $y \in (\{0, 1\}^w)^s$ the tree T probes indices $(i_1, \dots, i_d) \in [s]^d$ and outputs $(z_1, \dots, z_d) \in (\{0, 1\}^w)^d$, then $y' = \bar{T}(y)$ equals y except on positions i_j where instead $y'_{i_j} = z_j$. (So the read-write decision tree examines some part of its input and outputs a list of modifications that shall be done to the input.) $\lrcorner$

2.3 Data structures in the cell-probe model

The next definition appears already in [MNSW98, Section 1.1]:

Definition 2.2. A static data-structure problem is given by a tuple $((\mathcal{D}, \mathcal{Q}, \mathcal{A}), f)$ where

- $\mathcal{D}$, $\mathcal{Q}$, and $\mathcal{A}$ are finite sets: $\mathcal{D}$ is the set of databases, $\mathcal{Q}$ is the set of queries, and $\mathcal{A}$ is the set of possible answers to queries.
- $f : \mathcal{Q} \times \mathcal{D} \rightarrow \mathcal{A}$ is a map where $f(q, x)$ is the answer to query q about database x .

┘

In the above definition the tuple $(\mathcal{D}, \mathcal{Q}, \mathcal{A})$ specifies the universes that f applies to and it will be useful to have terminology to refer to this tuple.

Definition 2.3. • A static data-structure frame or static frame is a tuple $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A})$ of finite sets.

- $\text{Tables}(\mathcal{F})$ is the set of all functions $f : \mathcal{Q} \times \mathcal{D} \rightarrow \mathcal{A}$. We refer to elements of this set as tables.

┘

We think of elements of $\text{Tables}(\mathcal{F})$ as being analogous to the *truth table* of a Boolean function. This also explains the choice of the name “frame”: the frame indicates the shape of the table and the range of values in the table before being filled-in with the values $f(q, x)$.

The goal in a static data structure problem $((\mathcal{D}, \mathcal{Q}, \mathcal{A}), f)$ is to construct an efficient implementation, consisting of an encoding that represents each database in $\mathcal{D}$ by a vector of some fixed length s with entries in $\{0, 1\}^w$ and represent each query by a decision tree of depth t where s, w, t are “small”. This motivates the following definitions:

Definition 2.4. A static data-structure complexity bound, or simply static bound, is a tuple $\mathcal{B} = (s, w, t)$ of positive integers.

┘

Definition 2.5. Let $\mathcal{B} = (s, w, t)$ be a static bound. A $\mathcal{B}$ -implementation of a static data-structure problem $((\mathcal{D}, \mathcal{Q}, \mathcal{A}), f)$ is given by a tuple $(E, (T_q)_{q \in \mathcal{Q}})$ where

- $E : \mathcal{D} \rightarrow (\{0, 1\}^w)^s$ is an encoding map,
- For each $q \in \mathcal{Q}$, $T_q : (\{0, 1\}^w)^s \rightarrow \mathcal{A}$ is a decision tree over s -long $\{0, 1\}^w$ -words with output set $\mathcal{A}$, having depth at most t , and such that

$$\forall x \in \mathcal{D} \quad T_q(E(x)) = f(q, x).$$

We say that $((\mathcal{D}, \mathcal{Q}, \mathcal{A}), f)$ is $\mathcal{B}$ -implementable if it has such an implementation and define $\text{Tables}_{\mathcal{B}}(\mathcal{F})$ to be the set of $\mathcal{B}$ -implementable tables for frame $\mathcal{F}$.

┘

In the above definition, the encoding map E specifies the memory representation of a given database x using s cells of w bits each.

The above notion of an *implementation* was originally formalized by Yao [Yao78] and is called a *solution* in [MNSW98, Section 1.1].

Note. As we will see below, the notion of natural proof depends on these four notions, of frame, table, bound, and implementation. More generally, for a given computational model, a frame defines a family of problems that share some parameters in common, a table is one problem in this family, a bound gathers together the metrics of how efficient is an algorithm in the model, and an implementation is an algorithm for solving a problem within some bound.

For dynamic data structure problems, the notion of frame must be augmented to account for updates:

Definition 2.6. • A dynamic data-structure frame or dynamic frame, is a tuple $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U})$, where $(\mathcal{D}, \mathcal{Q}, \mathcal{A})$ is a static frame, $\mathcal{U}$ is a set of updates, where update is a function from $\mathcal{D}$ to $\mathcal{D}$. $\mathcal{U}$ is called the update structure of the frame, and can be thought of as an edge-labeled directed graph on vertex set $\mathcal{D}$ where for each database x and update $u \in \mathcal{U}$, there is a directed edge labeled u from x to $u(x)$.

- $\text{Tables}(\mathcal{F})$ is the set of all functions $f : \mathcal{Q} \times \mathcal{D} \rightarrow \mathcal{A}$.

┘

Thus a dynamic frame is obtained by taking a static frame and augmenting it by an update structure. The definition of table for dynamic frames does not depend on the update structure.

The reader should note the asymmetric way in which we treat queries and updates. A frame specifies the names of queries but not the values that queries assign to databases; this assignment is specified by a table for the frame. However, the frame fixes both the names of the updates and the action of updates on databases. This asymmetry is an important feature of our framework.

The most natural and important dynamic frame has the following update structure:

Example 2.7 (Hamming Frame). Let $\mathcal{D} = \Sigma^n$ for some set Σ and integer n . For $i \in \{1, \dots, n\}$ and $\phi : \Sigma \rightarrow \Sigma$ define the Hamming update $u_{i,\phi}$ to be the function that maps $x \in \mathcal{D}$ by changing the value of the i th coordinate x_i to $\phi(x_i)$, i.e., $u_{i,\phi}(x) = x_1 \dots x_{i-1} \phi(x_i) x_{i+1} \dots x_n$. In a Hamming frame for $\mathcal{D}$, the set of updates is a subset of the set $\{u_{i,\phi}\}$.

┘

Analogous to the definition of a static data structure problem we have:

Definition 2.8. A dynamic data-structure problem is given by a tuple $((\mathcal{F}), f)$ where $\mathcal{F}$ is a dynamic frame and $f \in \text{Tables}(\mathcal{F})$.

┘

History-independence. Next we want to define the notion of an implementation of a dynamic data-structure problem. In our definition we restrict attention to implementations that are *history-independent*. A history-independent implementation is one in which each database has a unique encoding, so the encoding in memory depends only on the data being encoded, not on the sequence of updates that took us there. Since history independence imposes a restriction on the implementation this might make it easier to prove lower bounds. However, in this paper we will argue that the natural-proofs paradigm explains the difficulty of proving lower-bounds for dynamic data-structure implementations even when restricting attention to history independent implementations.

Definition 2.9. A dynamic data-structure complexity bound dynamic bound is a tuple $\mathcal{B} = (s, w, t_u, t_q)$ of positive integers

┘

Definition 2.10. Let $\mathcal{B} = (s, w, t_u, t_q)$ be a dynamic bound. A $\mathcal{B}$ -implementation to a dynamic data-structure problem $((\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U}), f)$ is given by a tuple $(E, (T_q)_{q \in \mathcal{Q}}, (T_u)_{u \in \mathcal{U}})$ where

- $E : \mathcal{D} \rightarrow (\{0, 1\}^w)^s$ is an encoding map.

- For each $u \in \mathcal{U}$, $T_u : (\{0,1\}^w)^s \rightarrow (\{0,1\}^w)^s$ is a read-write decision tree over s -long $\{0,1\}^w$ -words, having depth at most t_u , such that

$$\forall x \in \mathcal{D} \quad \bar{T}_u(E(x)) = E(u(x)).$$

- For each $q \in \mathcal{Q}$, $T_q : (\{0,1\}^w)^s \rightarrow \mathcal{A}$ is a decision tree over s -long $\{0,1\}^w$ -words with output set $\mathcal{A}$, having depth at most t_q , and such that

$$\forall x \in \mathcal{D} \quad T_q(E(x)) = f(q, x).$$

We say that $(\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U}, f)$ is $\mathcal{B}$ -implementable if it has such an implementation and define $\text{Tables}_{\mathcal{B}}(\mathcal{F})$ to be the set of $\mathcal{B}$ -implementable tables for frame $\mathcal{F}$. $\lrcorner$

2.4 Communication Complexity

Communication Complexity serves as a powerful technique to prove lower bounds across different areas of computer science, especially in cases involving information bottlenecks. Many lower bounds found in Data Structures are either directly proved through Communication Complexity or can be more easily expressed using its terminology.

This is not an accident. One can formally see static data-structures in the cell-probe model as a restricted communication game. One player, say Alice, knows the query q that needs to be answered, Bob knows the data x , and they must communicate until Alice learns $f(q, x)$. The communication is organized in rounds, so that Alice communicates $\log s$ bits per round, and Bob communicates w bits. The “only” additional restriction that data-structures must obey, when compared to this general communication setting, is that in data-structures Bob must behave non-adaptively, namely, his answer must depend only on what Alice communicated in the last round, ignoring all previous rounds. This single difference makes a huge difference, in that with this additional restriction there are functions $f(q, x)$ that need $\omega(|q|)$ communication to be solved, whereas if Bob is adaptive Alice can just send an encoding of q in $|q| / \log s$ rounds, and Bob directly answers $f(q, x)$. A similar analogy can also be done for dynamic data-structures [Pă10].

Hence, in this section, we recall some basic notions of communication complexity, which we will refer to in the rest of the paper. We will be fairly informal: for more details, two good references in this topic are [KN97, RY20].

Deterministic communication complexity

Deterministic communication complexity is a computational model where two parties, Alice and Bob, communicate in order to solve some joint computational task. Typically, Alice knows some input $x \in \mathcal{X}$ and Bob knows some input $y \in \mathcal{Y}$, and they wish to compute a function $f(x, y)$ of their joint input. In order to compute f , Alice and Bob agree on a strategy, called a *protocol*, where they take turns to speak and exchange messages such

that, at the end of the communication, both the parties have learned $f(x, y)$. The sequence of the messages exchanged in this process is called the *transcript*.

We could formally define a protocol as a binary tree π , where each internal node v is labeled with the player whose turn it is to communicate, alongside a function $m_v : \mathcal{X} \rightarrow \{0, 1\}$ or $m_v : \mathcal{Y} \rightarrow \{0, 1\}$ (depending on which player is speaking) that determines the bit to be transmitted next. The players start at the root, and then successively follow the edge given by $m_v(x)$ and $m_v(y)$, until they reach a leaf. The leaves are labeled by the answers from A , and the output $\pi(x, y)$ is the label of the leaf reached by the players. We say π computes f if $\pi(x, y) = f(x, y)$ for all x, y . The *cost* of a deterministic protocol is the depth of this tree. The Communication complexity of a function f is the smallest cost of a deterministic protocol which computes f .

A *combinatorial rectangle* R is a product set $R = A \times B$ for some $A \in \mathcal{X}$ and $B \in \mathcal{Y}$. The most basic insight about protocols is that, for each node v in a protocol, the set of input pairs (x, y) that go through v is a combinatorial rectangle R_v . It follows that a protocol computes f precisely if the every combinatorial rectangle R_v corresponding to a leaf v is *monochromatic*, namely, $f(x, y)$ is constant for $(x, y) \in R_v$.

Asymmetric Communication

The *asymmetric* setting is when $\mathcal{X}$ is significantly smaller than $\mathcal{Y}$. In this setting, we keep track of the number of bits sent across by Alice and Bob separately.

Definition 2.11. An $[a, b]$ -protocol is a protocol where the total number of bits that Alice sends to Bob is at most a and the total number of bits that Bob sends to Alice is at most b . $\lrcorner$

Definition 2.12. A $[t, a, b]^A$ -protocol is a protocol where each of Alice's messages contains at most a bits and each of Bob's messages contains at most b bits and the players alternate, sending at most t messages in total, with Alice sending the first message. A $[t, a, b]^B$ protocol is defined similarly, but with Bob sending the first message. $\lrcorner$

3 What is as a natural proof?

The notion of a *natural proof* is not a formal notion: there is no rigorous definition, and as we will see this is for good reason. Instead, the formal notion is that of a natural property. We now formalize the notion of natural property in the context of data structures.

The reason for defining the four concepts *frame*, *bound*, *table* and *solution* using the same names as above is that the definition of natural proof is the same for both static and dynamic data structures, or in fact for any computational model, provided we properly instantiate the notions of frame, table, bound and solution as appropriate to the model.

Definition 3.1. Let $\mathcal{F}$ be a frame, $\mathcal{B}$ a bound, $p \in (0, 1)$ and $C \in \mathbb{N}$. An $(\mathcal{F}, \mathcal{B}, p, C)$ -natural property is a predicate $\Phi : \text{Tables}(\mathcal{F}) \rightarrow \{0, 1\}$ such that:

- (Usefulness) If $f \in \text{Tables}(\mathcal{F})$ is a table such that $(\mathcal{F}, f)$ has a $\mathcal{B}$ -solution, then $\Phi(f) = 0$.

- (Largeness) For f chosen uniformly at random from $\text{Tables}(\mathcal{F})$, $\Pr[\Phi(f) = 1] \geq p$.
- (Constructivity) Φ is computable by a circuit of size C .³

┘

Now let us argue why the definition of a natural proof should actually be an informal definition. Note that, if we have an $(\mathcal{F}, \mathcal{B}, p, C)$ -natural property Φ such that $\Phi(f) = 1$, this necessarily implies that the data-structure problem f does not have solutions with bound $\mathcal{B}$. I.e. $\Phi(f) = 1$ gives us a lower-bound on the complexity of f (this follows from *usefulness*). One might then be tempted to define a natural proof as a proof that explicitly constructs a natural property Φ and shows that $\Phi(f) = 1$. However, the reality is that most lower-bounds in the literature explicitly use specific properties of the function f , which are not true of a random function, or properties which appear to be hard to compute efficiently when given the truth-table of f . It would then seem that such lower-bounds are not *natural proofs*, because the lower-bound-implying property does not obey largeness, or constructivity, hence it is not natural.

The very surprising fact is that, whenever one finds such a seemingly-unnatural proof in the literature, then systematically, time and again, after a bit of thinking, one concludes that the proof can be changed only slightly, to extract from it a similar argument which does not actually require any of the specific properties of f that were used in the original argument, and which indeed also applies to a random function. And we will find that this extracted property is very simple to compute when given f as input. One has then obtained a natural property from the original lower-bound proof. We will see many instances of this in this paper.

This process of extraction of a natural property is called *naturalizing* a lower-bound proof. We may then think of it like so: the proof didn't *seem* natural, but deep down it actually was a natural proof. So in practice we call *natural proof* to any *naturalizable* lower-bound proof. Of course, such a concept of natural proof is necessarily imprecise: there is no way to formally specify or even predict what will be necessary to do in order to naturalize a given lower-bound proof. Nonetheless, if a natural proofs barrier exists, no lower-bound proof can exist which can be naturalized, no matter how one might try to do it. So this imprecise concept is somehow the right concept. We must therefore settle for an informal definition. In the context of static or dynamic data structures, the informal definition is the following:

Definition 3.2 (informal). Let $\mathcal{F}$ be a frame, $f \in \text{Tables}(\mathcal{F})$ a table, and $\mathcal{B}$ a bound. If we have a proof that the problem $(\mathcal{F}, f)$ does not have $\mathcal{B}$ -solutions, and from this proof we can somehow extract an $(\mathcal{F}, \mathcal{B}, p, C)$ -natural property Φ , where p is reasonably large (e.g., $p = \Omega(1)$), C is not too large (e.g. $C \leq |f|^{O(1)}$, or maybe $C \leq 2^{(\log |f|)^{O(1)}}$), and the proof can be adapted to show that $\Phi(f) = 1$, then we call this proof a *natural proof*. ┘

³Here we have opted for defining constructivity in a non-uniform way, i.e. with respect to circuit size instead of deterministic time. This simplifies the treatment, and it is a reasonable choice to make, since often PRGs are thought of as being secure against circuits, and not just uniform algorithms. But the notion of a natural property could also be defined, *mutatis mutandis*, for uniform algorithms.

We again remark that both notions of natural property and natural proof generalize to other computational models, if the correct analogues of frame, table, bound and solution are defined.

We wish to make one further generalization of the notion of natural proof, which relaxes the notion of constructivity. For this to make sense, one must understand how the notion of natural proof gives rise to the natural proofs barrier. As explained in the introduction, any natural proof provides us with a distinguisher Φ which accepts a random function w.h.p. but rejects all easy functions. And crucially, by constructivity, this distinguisher is efficient, i.e. Φ can be computed efficiently in the size of f . Now suppose that, within a certain model, we have a pseudorandom function generator, i.e. we can generate a pseudorandom function f which is easy, but the pseudorandom distribution we get on the tables fools any efficient distinguisher. It then follows that no natural proof can prove the hardness of any function. Here we used the fact that PRFs fool efficient distinguishers, together with constructivity of natural proofs Φ , to show that Φ cannot prove a lower-bound. But, indeed, we can skip the middle step: It isn't essential that Φ is constructive, *per se*, what is important is that Φ is a property which can be fooled by a pseudorandom family of easy functions! This leads to the following informal definition:

Definition 3.3 (informal). *Let $\mathcal{F}$ be a frame, $f \in \text{Tables}(\mathcal{F})$ a table, and $\mathcal{B}$ a bound. Suppose that we have a proof that the problem $(\mathcal{F}, f)$ does not have $\mathcal{B}$ -solutions, and from this proof we can somehow extract an $(\mathcal{F}, \mathcal{B}, p, C)$ -large-and-useful property Φ , where p is reasonably large, C is not too large, and the proof can be adapted to show that $\Phi(f) = 1$. Suppose further that there exists a PRF, namely a pseudorandom distribution over $\text{Tables}(\mathcal{F})$, which fools all efficient distinguishers, and also fools the property Φ (even though Φ itself might not be constructive). Then we call this proof an essentially natural proof.* $\square$

Sometimes we are faced with lower-bound proofs that appear to use large, useful, but non-constructive properties Φ . In most cases, we can change Φ into some other property Φ' which is also constructive, but sometimes we do not know how to do that. Nonetheless, we find that the property, although it is not obviously constructive, is still easy to fool by PRFs. Then the property *also* cannot be used to prove lower-bounds against the class where the pseudorandom functions are sampled from! I.e. the proof might not be natural, but it is still *essentially natural*, as per the above informal definition.

In this paper we will strive to show that all known data-structure lower-bound proofs are natural. There are a few proofs where making the property constructive is possible but technically challenging, and two exception, both in the static case, where we do not know how make it constructive at all. One of these exceptions (lower-bounds based on CNF refutations via γ -biased property) is essentially natural (our PRFs fool it), and the other exception (lifting theorems) does not appear to be natural at all. In the dynamic setting, all known proofs are natural.

4 Proposal: A natural proofs barrier against static data structure lower bounds

We will now consider two fundamental problems, which exemplify where we are stuck in static data structure lower-bounds.

The first is the VMV problem. Here, the data set is $\mathcal{D} = \mathbb{F}_2^{n \times n}$, the set of all $n \times n$ Boolean matrices, our query set is $\mathcal{Q} = \mathbb{F}_2^{n+n}$, the set of all pairs of n -dimensional vectors, and our problem is $f : \mathcal{Q} \times \mathcal{D} \rightarrow \mathbb{F}_2$ is given by $f((u, v), x) = u^\top \cdot x \cdot v$. I.e., we wish to store a matrix x and for each pair $q = (u, v)$ of vectors the corresponding query outputs the vector-matrix-vector product. We will then call the VMV *frame* to the static data-structure frame $(\mathbb{F}_2^{n^2}, \mathbb{F}_2^{2n}, \mathbb{F}_2)$. Of course, most tables of this same frame will completely ignore the linear structure of the data and queries, so we should think of the VMV frame as being just $([2^{n^2}], [2^{2n}], \{0, 1\})$.

The best known lower-bound for the VMV problem⁴ is:

$$t = \Omega \left(\frac{\log |\mathcal{Q}|}{\log \left(\frac{sw}{\log |\mathcal{D}|} \right)} \right) = \Omega \left(\frac{n}{\log \left(\frac{sw}{n^2} \right)} \right).$$

The second prototypical problem we consider is the orthogonal range counting, or ORC, problem. Here our data is a collection of n points in a d -dimensional cube in Euclidean space, which we have discretized into a coarse grid, so $\mathcal{D} = [n]^{dn}$. A query specifies a *range*, in the orthogonal case this is just a box within the grid, so $\mathcal{Q} = \{q = (a_1, b_1, \dots, a_d, b_d) \in [n]^{2d} \mid 1 \leq a_i \leq b_i \leq n\}$. Our function $f(q, x)$ returns the number of points in the collection x which are inside the box q . The ORC frame is thus $([n^{dn}], [n^{2d}], \{0, \dots, n\})$.

The best lower-bounds for this problem [Pă07] state that, for any $d \geq 2$, if the space $s = O(n \text{polylog } n)$ and width $w = O(\log n)$, then any (s, w, t) -solution has

$$t = \Omega \left(\frac{\log |\mathcal{Q}|}{\log \frac{sw}{\log |\mathcal{D}|}} \right) = \Omega \left(\frac{\log n}{\log \log n} \right).$$

The best upper-bounds, however, are strictly greater when $d \geq 3$, standing at $t \leq O((\log n / \log \log n)^{d-1})$ [Wil85, JMS04, CW13]. So one could hope to prove, e.g., a $\Omega((\log n / \log \log n)^3)$ lower-bound for $d = 4$. However, in a not-as-tight but more general result as Ramamoorthy-Ratshtchian, which in fact preceded and inspired it, Dvir, Golovnev and Weinstein [DGW19] have shown that proving lower-bounds of the form $t = \omega((\log |\mathcal{Q}|)^2)$, for *any* static data structure problem, would imply better constructions of rigid matrices. This quadratic loss makes the [DGW19] result not meaningful for the VMV problem, but it is still meaningful for ORC and other problems.

⁴This was proven using lifting theorems in [CKLM18], and later using a cell sampling argument in [RR20].

It should be said, the expression $\Omega\left(\frac{\log |\mathcal{Q}|}{\log \frac{stw}{\log |\mathcal{D}|}}\right)$ appearing in both lower-bounds above is not a coincidence: no better lower-bound is known for *any* static data-structure, over *any* frame.

4.1 Local PRFs, and a natural proofs barrier against static data-structure lower-bounds

In this section, we spell out the cryptographic assumption which we require for the natural proofs barrier. Our definitions are slightly more general than those of [KPI25], and this is an important difference, so we will explain it in [Section 4.3](#).

Definition 4.1. Let $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A})$ be a static data-structure frame.

- A function generator over $\mathcal{F}$ is a distribution μ over $\text{Tables}(\mathcal{F})$.
- For $\mathcal{B} = (w, s, t)$ we say that a function generator is $\mathcal{B}$ -local if every f in the support of μ is $\mathcal{B}$ -implementable. (So for a $\mathcal{B}$ -implementable generator every table in the support has columns labeled by vectors in $(\{0, 1\}^w)^s$ and rows labeled by depth t decision trees over s variables taking values in $\{0, 1\}^w$.)
- For any property $\mathcal{P}$ of tables over $\mathcal{F}$ (i.e. a mapping from $\text{Tables}(\mathcal{F})$ to $\{0, 1\}$) we say that the function generator μ ϵ -fools $\mathcal{P}$ if

$$|\Pr_{f \sim \mu}[\mathcal{P}(f) = 1] - \Pr_{f \sim \text{Tables}(\mathcal{F})}[\mathcal{P}(f) = 1]| < \epsilon.$$

- A function generator μ is (S, ϵ) -secure, if it fools any property that is computable by a boolean circuit C of size S that takes as input the table.
- An $(\mathcal{F}, \mathcal{B}, S, \epsilon)$ -local pseudorandom function (local PRF) is a $\mathcal{B}$ -local, (S, ϵ) -secure generator over $\mathcal{F}$.

⌋

The notion of local PRF does not appear explicitly in [KPI25] but they implicitly consider the following special case of local PRF's that are induced by a locally computable pseudorandom generator (PRG).

Example 4.2. • A locally computable pseudorandom generator for bound $\mathcal{B} = (s, t, w)$ and frame $\mathcal{F}$ is a tuple $G = (G_q : q \in \mathcal{Q})$ where G_q is a depth t decision tree over s -long $\{0, 1\}^w$ -words having output set $\mathcal{A}$. Such a generator defines a probability distribution over functions from $\mathcal{Q}$ to $\mathcal{A}$ by selecting z uniformly from $(\{0, 1\}^w)^s$ and evaluating $G(z) = (G_q(z) : q \in \mathcal{Q})$.

- The local PRF for frame $\mathcal{F}$ induced by the local PRG G is obtained by choosing a uniformly random encoding function $E : \mathcal{D} \rightarrow (\{0,1\}^w)^s$ and for each $x \in \mathcal{D}$ setting column x of the table to be $G(x)$.

┘

We then conjecture that sufficiently secure local PRFs exist, right at, or very near to, the $t = \Omega(\frac{\log |\mathcal{Q}|}{\log \frac{sw}{\log |\mathcal{D}|}})$ border where we know how to prove lower-bounds. This could be stated using many different choices of frames and bounds. We exemplify the conjecture for the VMV and ORC frames.

Conjecture 4.3. *For any constant $c > 1$, the following holds for all large enough n .*

- (a) *Let $\mathcal{F} = ([2^{n^2}], [2^{2n}], \{0,1\})$ be the VMV frame, let $\mathcal{B} = (w, s, t)$ be a bound with $w = 1$, $s \geq n^c$, and $t = \omega(n)$, and let $S = 2^{n^{O(1)}}$, $\varepsilon = o(1)$. Then, there exists an $(\mathcal{F}, \mathcal{B}, S, \varepsilon)$ local PRF.⁵*
- (b) *Let $d = \Theta(1)$ be a large enough constant, $\mathcal{F} = ([n^n], [n^{2d}], \{0, \dots, n\})$ be the ORC frame, let $\mathcal{B} = (w, s, t)$ be a bound such that $w = \Theta(\log n)$, $s \geq n^c$, and $t = \omega(\log n)$, and let $S = 2^{n^{O(1)}}$, $\varepsilon = o(1)$. Then, there exists an $(\mathcal{F}, \mathcal{B}, S, \varepsilon)$ local PRF.*

The conjecture above is tailored to almost match the known lower-bounds. To exactly match them, one would instead use the condition $t = \omega\left(\frac{\log |\mathcal{Q}|}{\log(\frac{sw}{\log |\mathcal{D}|})}\right)$, for the VMV frame. We prefer to state the conjecture in such a way that it is agnostic over whether the logarithmic denominators can be removed or not. Indeed, we would rather focus on the larger point of whether one can prove *any* static data-structure lower-bound such as $t = \omega(\log |\mathcal{Q}|)$, i.e., which distinguishes data-structure complexity from communication complexity⁶.

We now observe that the above conjecture implies the existence of a natural proofs barrier. Indeed, if super-linear lower-bounds on the VMV problem could be shown by a natural proof, this would give us an $(\mathcal{F}, \mathcal{B}, \Omega(1), 2^{n^{O(1)}})$ natural property, which would reject every table $f \in \text{Tables}(\mathcal{F})$ in the image of the conjectured local PRF, and yet accept an $\Omega(1)$ fraction of all tables in $\text{Tables}(\mathcal{F})$. The same reasoning applies to the ORC problem.

Theorem 4.4. *If Conjecture 4.3 (a) is true, then there does not exist a natural proof showing that the VMV problem needs $t = \omega(n)$. Likewise, if Conjecture 4.3 (b) is true, then there does not exist a natural proof showing that the ORC problem needs $t = \omega(\log n)$.*

⁵For the VMV frame we can consider the bit-probe model, where each cell stores $w = 1$ bit, instead of the usual $w = \Theta(\log n)$, since we are unconcerned about $\log n$ factors.

⁶One can consider the communication complexity of the associated communication problem, where Alice is given q and Bob is given x , and they wish to output $f(q, x)$. This problem can be solved with little communication if the data structure has an efficient solution, but in this setting Alice can always send q to Bob, who computes the answer and sends it back, so we have an upper bound of $\log |\mathcal{Q}|$. We believe that such an upper bound is impossible for static data-structure problems such as the VMV problem.

4.2 Two candidates for local PRFs

We now present two local generators, which we conjecture are sufficiently secure to be local PRFs. Of course, proving that any generator is secure would require proving that $P \neq NP$ along the way. Instead, one could hope to devise a generator whose security is dependent on the intractability of some known hard problem. This is the case with several previous constructions of pseudorandom functions, whose security is based on the conjectured hardness of factoring, discrete log, and lattice problems (see [BR17] for a survey). However, it appears that local PRFs are not expressive enough to carry out any of these constructions, which are inherently very non-local. The paper [KPI25] proposes a way of constructing a pseudorandom function using a pseudorandom generator. Their approach is not fully general, for example our PRF candidates are not captured by it: we will discuss this further in the [Section 4.3](#).

Here, we propose two different candidates, one of which is non-adaptive, and the other is adaptive. We have tried to break these generators, so far with no success. Perhaps they can be broken, or perhaps they are secure. One thing can surely be said: any proof of a static data-structure lower-bound better than $\omega(\log \log |\mathcal{D}| + \log |\mathcal{Q}|)$, with $s = \Omega((\log |\mathcal{D}|)^{2+\epsilon})$, will either not be natural, or will have to somehow break every single local generator, including the two generators we propose. So one could either believe that our generators are secure, and try to conceive of how a data structure lower-bound could avoid being natural, or we could believe that data structure lower-bounds can be proven by natural proofs, in which case all local PRFs can be broken, and one might as well begin by attempting to break our two candidates. In either case, the main point is that the natural proofs barrier offers a suggestion of what needs to be done next.

Our constructions were eventually found by starting with two different attempts, finding out that they can be broken by particular tests, and finding a way of overcoming these tests.

The frame and bound. Let $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \{0, 1\})$ be a frame with $|\mathcal{D}| = 2^n$ data elements, $|\mathcal{Q}| \leq 2^{o(n)}$ queries, and Boolean answers. Let $t = \omega(\log n + \log |\mathcal{Q}|)$ and $s = \omega(nt) = \omega(n \log |\mathcal{Q}|)$, with $s = o(|\mathcal{Q}|)$.⁷ Notice that this choice of parameters is compatible with both the VMV and ORC frames.

And also notice that $s \approx n \log |\mathcal{Q}|$ is not ideal, we would like to have a PRF with $s \approx n$, i.e. using close to minimum space so to exclude lower bounds for very compact data structures. However, our construction does not achieve this.

1st attempt: random non-adaptive queries. A natural first thing to try, to obtain a non-adaptive local PRF, is as follows. Choose a random family $(S_q)_{q \in \mathcal{Q}}$ of subsets $S_q \subseteq [s]$ of size t .

Now sample a table $f \in \text{Tables}(\mathcal{F})$ as follows. We choose the encoding function $E : [2^n] \rightarrow \{0, 1\}^s$ uniformly at random, so that each encoding $E(x)$ is an independently sampled, uniformly random binary string of length s . We then choose functions $f_q :$

⁷The bound $s = o(|\mathcal{Q}|)$ is needed for the setting to make sense, since with $s = |\mathcal{Q}|$ we can solve any problem with 1 query.

$\{0,1\}^t \rightarrow \{0,1\}$, independently for each $q \in \mathcal{Q}$, uniformly at random. Finally, $f(q, x) = f_q(E(x)|_{S_q})$, i.e., the function f_q applied to the S_q positions of $E(x)$.

How does one break this generator? It is enough to look at a single row, i.e., we take any $q \in \mathcal{Q}$, and examine the bias $\frac{1}{2^n} \sum_x f(q, x)$ of that row. Since $f(q, x) = f_q(E(x))$, then no matter how f_q was fixed, since $E(x)$ is chosen independently at random, $f_q(E(x)|_{S_q})$ is a bit with the same bias as f_q . Unless f_q is perfectly balanced, which only happens with probability $\tilde{O}(2^{-t})$, the bias of f is at least $\frac{2}{2^t}$, and hence the entire row is very likely to have a bias $\Omega(2^{-t})$. But a *uniformly random* row will have bias $\tilde{O}(2^{-\frac{n}{2}})$ with high probability, which is much less. So an algorithm can distinguish the two cases by computing the bias.

One can try to fix this by sampling f to be a uniformly chosen, *perfectly balanced* function, i.e., with vanishing degree-0 Fourier coefficient, but now a similar trick works: we can fix any two rows q_1 and q_2 , and look at the bias of the parity $f_{q_1}(E(x)) \oplus f_{q_2}(E(x))$. It is only $\tilde{O}(2^{-t})$ likely over the choice of f_{q_1} and f_{q_2} that such a bit (with random $E(x)$) is unbiased, and if this is biased then its bias is at least 2^{-2t} . But if f is a random table, each bit in the parity of two rows is completely unbiased, so again the average bias of the parity of the entries of these two rows would be $\tilde{O}(2^{-\frac{n}{2}})$, which is much smaller than 2^{-2t} .

2nd attempt: LDPCs. Our second generator is specifically designed to fool all such discrepancy-like tests that check for biases of rows and submatrices and so forth (which the first generator did not fool).

To do this we will need a $|\mathcal{Q}| \times s$ matrix P such that (1) each row has at most t 1's and (2) any k rows are linearly independent. There is a way of doing this using low-density parity-check codes (LDPCs). Let $N(s, k, t)$ be the largest number of rows in a matrix P that has s columns, where each row has at most t ones, and such that every k rows are linearly independent (mod 2). It can be proven (see Appendix A) using a variant of an argument by Lefman, Pudlák and Savický [LPPS96], that

$$N(s, k, t) \geq \exp((1 - o(1))t) \geq |\mathcal{Q}|, \text{ if } s = \omega(kt).$$

Let us then choose $k = \omega(n)$, and notice that for our choice of $s = \omega(n \log |\mathcal{Q}|)$, we do indeed have $s = \omega(kt)$.

We now sample a table $f \in \text{Tables}(\mathcal{F})$ as follows. The encodings $E(x) \in \{0,1\}^s$ are chosen uniformly at random. Let P be a $|\mathcal{Q}| \times s$ matrix as above, and let P_q be the q th row of P and define $f(q, x) = \langle P_q, E(x) \rangle = \sum_{i \in [s]} P_{q,i} E(x)_i \pmod{2} = \bigoplus_{i \in P_q} E(x)_i$. By construction, P_q has t 1's, hence f has an $(s, 1, t)$ -solution.

Many consequences will follow from this, including the fact that such a matrix cannot have large monochromatic rectangles. More on this later. The issue now is that this PRF can be broken by a linear algebra test. Indeed, think of each row $f(q, \cdot)$ as a vector in $\mathbb{F}_2^{2^n}$. Since each row is a parity of some of the bits in the encodings $E(x) \in \{0,1\}^s$, then these rows, as vectors of dimension 2^n , live in a space of dimension s (they are spanned by the s -many single-bit parities), and so the rank of f (as a matrix from $\mathbb{F}_2^{|\mathcal{Q}| \times 2^n}$) is at most $s + 1$.

But a uniformly random table has full rank $|\mathcal{Q}| \gg s$ with high probability, so this rank test distinguishes the two cases.

Candidate 1 (non-adaptive). Our first candidate, then, is simply the XOR of the above two attempts. I.e., we choose two encodings $E_1(x)$ and $E_2(x)$ independently and uniformly at random, and $f(q, x) = f_1(q, x) \oplus f_2(q, x)$ where $f_1(q, x) = f_q(E_1(x)|_{S_q})$ and $f_2(q, x) = \bigoplus_{i \in P_q} E_2(x)_i$, where $(S_q)_{q \in \mathcal{Q}}$ is a random family of size- t subsets of $[s]$, $(f_q)_{q \in \mathcal{Q}}$ are randomly chosen functions from $\{0, 1\}^t$ to $\{0, 1\}$, and P_q is the q -th row in the above LDPC matrix. (The number of queries is now $2t$ instead of t , but let us ignore such constant factors.)

Candidate 2 (adaptive). Our second candidate is obtained slightly differently. We choose $E_1(x)$, $E_2(x)$, and P_q as before. But instead of random S_q and f_q , which we can think of as random non-adaptive decision trees, we sample random adaptive decision trees T_q of depth t , as follows.

- For each internal node v of T_q , by order of ancestry starting from the root, we sample a coordinate $i \in [s]$ uniformly at random among those coordinates in $[s]$ which were not probed by any ancestor of v . This will be the cell which is probed at node v .
- Finally, for each leaf node v , we choose uniformly random bit and this is the output at v .

This process outputs a complete decision-tree of depth t . The sampled table f is then given by $f(q, x) = f_1(q, x) \oplus f_2(q, x)$, where $f_2(q, x) = \bigoplus_{i \in P_q} E_2(x)_i$ as before, and $f_1(q, x) = T_q(E_1(x))$.

Sanity checks. For both constructions, one can readily verify that any single row $f(q, \cdot)$ of the table is uniformly random, since each entry x in that row chooses a random encoding $E_2(x)$, which gives us a completely random input to the parity $f_2(q, x) = \bigoplus_{i \in P_q} E_2(x)_i$. Since the parity is a balanced function, this is a fresh, independent random bit for each x . The same reasoning extends to any choice of up to k rows, because every k parities are independent.

Likewise, any single column $f(\cdot, x)$ is uniformly random: if we fix any encoding $E_1(x)$ for x , the different outputs $f_1(q, x)$ (over different q) are given by the different values $f_q(E(x)|_{S_q})$, or $T_q(E(x))$, and these values are themselves chosen independently at random. Less clear is when we look at different columns $x_1, \dots, x_\ell$. However, since the encodings $E(x)$ are random bits, one can see that the probability that $E(x_i)|_{S_q} = E(x_j)|_{S_q}$ is 2^{-t} . So provided that $\ell = o(|\mathcal{Q}|^2 2^t)$, the probability that any two such partial encodings are the same is $o(1)$, and conditioned on this event the submatrix $f(\cdot, x_1) \dots f(\cdot, x_\ell)$ is completely random. The same argument works for the second, non-adaptive candidate, noticing that the probability that $T_q(E(x_i))$ follows the same path as $T_q(E(x_j))$ is also 2^{-t} .

From the above it follows that: any test which looks at fewer than k rows, and any test which looks at fewer than $o(|\mathcal{Q}|^2 2^t)$ columns, will necessarily fail to distinguish the output of these generators from random, except with $o(1)$ probability.

Size of monochromatic rectangles. We just argued above that any k choice of rows is uniformly random, and we chose $k = \omega(n)$. It follows that the marginal distribution on any $2n \times 2n$ submatrix is uniformly random. The probability that such a submatrix is monochromatic is 2^{-4n^2} . The number of such submatrices is $\binom{|\mathcal{D}|}{n} \times \binom{|\mathcal{Q}|}{n} \leq 2^{2n^2}$. It follows from a union bound that with probability $1 - o(1)$ there will exist no monochromatic submatrix of size greater than $2n \times 2n$. This will be enough to imply several natural properties used in lower-bound proofs.

A similar argument using Hoeffding's inequality shows that, for any constant $\varepsilon > 0$ there exists a constant c such that, with probability $1 - o(1)$, f will have no rectangle of size $\geq cn \times cn$ and bias $\geq \frac{1}{2} + \Omega(1)$.

Other checks. In [Section 5](#) we will show that almost every proof technique used to prove static data-structure lower-bounds is natural. That means that almost every known technique produces an efficient distinguisher which separates easy functions (below the proved lower-bound) from random functions. We will show that our local PRF candidate fools every one of these distinguishers. We only found two exceptions. The first is lifting theorems — here it seems that the natural proofs barrier simply does not apply, as has been observed already in the work on the KRW conjecture [[KRW95](#)], which precisely seeks to circumvent this barrier. The second non-naturalizing proof is a proof in [[KPI25](#)] which uses a property of the table which they call being γ -biased. This property is *not* efficiently computable, nor does there seem to be any obvious way of making it efficiently computable. Nonetheless, we will show that our PRF also fools this hardness-implying property, which shows that the proof is essentially natural (as in [Definition 3.3](#)).

4.3 PRGs vs. PRFs, and why it matters

The definition of generator appearing in [[KPI25](#)] is slightly different, and we wish to explain the difference, because it is an important aspect of local PRFs. The definition in [[KPI25](#)] is the following:

Definition 4.5 (Definition 5 of [[KPI25](#)]). *A locally-computable generator with time complexity t and space complexity s is a map $G : (\{0,1\}^w)^s \rightarrow \{0,1\}^{\mathcal{Q}}$ such that, for every $q \in \mathcal{Q}$, there exists a depth- t decision-tree T_q over s -long words of w bits, so that $G(E)_q = T_q(E)$ for all $q \in \mathcal{Q}, E \in (\{0,1\}^w)^s$. $\lrcorner$*

The authors don't explicitly define what it means for such a generator to be secure, but the standard definition of (S, ε) -security would be that

$$|\Pr_E[C(G(E)) = 1] - \Pr_{y \in \{0,1\}^N}[C(y) = 1]| \leq \varepsilon$$

whenever C is computable by a circuit of size S . Let us call a *locally-computable PRG* (as opposed to our definition of *local PRF*) to such an object.

It turns out, if we have a locally-computable PRG, i.e. a locally-computable generator which is sufficiently secure, this implies the existence of a local PRF, as per our [Definition 4.1](#), but with worse parameters. Given a locally-computable PRG G , we sample a

table $f_G \in \text{Tables}(f)$ by sampling each of the $\mathcal{D}$ columns of f_G from independent runs of G . It follows from a standard hybrid argument that if G is a $(S, \frac{\varepsilon}{|\mathcal{D}|})$ -secure PRG, then f_G is an (S, ε) -secure PRF. Conversely, if f_G is (S, ε) -secure, then the PRG must have also been (S, ε) -secure, since f_G must have been indistinguishable from random, also by distinguishers which look at a particular, fixed column. But, as you can see, this is not an equivalence because of the loss in the distinguishing probability ε , and there remains the possibility of constructing f in a different way, i.e. which does not come from a PRG.

Our PRFs use this possibility, since our decision-trees T_q are themselves chosen at random, instead of being fixed choices. This would not make any sense if one is only choosing a single pseudorandom column — because the randomness needed to sample a decision tree is more than the randomness of any single column — but this is okay if one is choosing an entire table f , as a PRF does, because the entire table is much larger.

Furthermore, notice that even among PRFs f_G , i.e. whose columns are sampled from a single PRG G , it is *not necessarily required* that G is $(S, \frac{\varepsilon}{|\mathcal{D}|})$ -secure, so that the PRF f_G is (S, ε) -secure. In fact, the local PRFs which we will construct in the next section, are NOT (S, ε) -secure for some $\varepsilon \leq \frac{1}{|\mathcal{D}|}$, but only for $\varepsilon \leq \exp(-t)$. Since in our case we have $t \approx \log |\mathcal{Q}|$ and thus $\exp(-t) \approx |\mathcal{Q}|$, and in most data-structures of interest we have $|\mathcal{Q}| \ll |\mathcal{D}|$, this is a crucial difference, and is another reason why we need a more general, more permissive definition of local PRF, beyond those PRFs derived from locally-computable PRGs.

It is not at all clear how to create a PRF, or PRG, which is secure in the $\varepsilon \leq \frac{1}{|\mathcal{D}|}$ regime. This is a-priori possible since the length of the encoding E can be polynomially-larger in $\log |\mathcal{D}|$, but in most of our attempts some detectable $\omega(\frac{1}{|\mathcal{D}|})$ bias has crept in. And even when we could not see how to break a particular PRF with such small distinguishing probability ε , we were still not confident enough in the construction to conjecture that it was secure.

And for the purpose of establishing a natural-proofs barrier, it doesn't really matter, because all known natural proofs give us distinguishers with very large, $1 - o(1)$, distinguishing probability, so $\varepsilon = o(1)$ security is already enough to defeat all these proofs.

The final reason for considering PRFs instead of PRGs, is that it allows us to think also about *dynamic* data-structures. The notion of locally-computable PRG cannot be used in that context. We will of course make use of this in the forthcoming sections on dynamic data structures.

5 Thesis: All known static data structure lower bounds are natural

In this section, we cover the existing static data-structure lower-bound proofs and explicitly state natural properties which arise out of them. We will cover 5 techniques: the richness method (Section 5.1), round-elimination (Section 5.2), cell sampling lower-bounds (Section 5.3), and lower-bounds via CNF-refutation (Section 5.4). We will also briefly discuss (Section 5.5) the one lower-bound technique which doesn't seem to be natural, or even essentially natural: lifting theorems.

We will ignore several results in closely related lines of research, such as succinct data-structures [GM07, Pă09, CKL18], so that we can focus on the “vanilla” setting of data-structure lower-bounds in the cell-probe model.

5.1 The richness method

The richness method is a lower-bound technique in asymmetric communication complexity. It first appeared in a seminal paper of Miltersen, Nisan, Safra and Wigderson [MNSW98], and has been used and expanded upon many times, in order to prove static data-structure lower-bounds [BOR99, BR02, PT06a, ACP08, WY14, Yin16].

Definition 5.1. Let $d, p \in \mathbb{N}$. A table $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$ is called (d, p) -rich if there are at least d elements x of $\mathcal{D}$ for which we can find p elements q of $\mathcal{Q}$ with $f(q, x) = 1$. $\lrcorner$

Lemma 5.2 (Richness Lemma). If $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$ is (d, p) -rich and has an (s, w, t) solution, then there exists a 1-rectangle $A \times B \subseteq \mathcal{Q} \times \mathcal{D}$ of size $|A| \geq 2^{-t \log s} p$ and $|B| \geq 2^{-t(w + \log s)} d$, which is 1-monochromatic, meaning $f(A \times B) = \{1\}$.

It then follows that, if a function f is (d, p) -rich, and yet it is also such that no such large 1-monochromatic rectangles exist, then the corresponding data problem does not have an (s, w, t) -solution. For example, for $w = \Theta(\log n)$ and $s = n^{O(1)}$, the following results have been proven using this technique:⁸

- *Lopsided set disjointness.* Suppose $k \leq \ell \leq \frac{n}{2}$, and let $\mathcal{Q} = \binom{[n]}{k}$, $\mathcal{D} = \binom{[n]}{\ell}$, $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$, given by $f(S, T) = 1$ iff S and T are disjoint. Then f is $(\binom{n}{\ell}, \binom{n-\ell}{k})$ -rich, and yet if $A \times B \subseteq \mathcal{Q} \times \mathcal{D}$ is a 1-monochromatic rectangle, i.e. every (S, T) pair in $A \times B$ is disjoint, then $\cup A$ and $\cup B$ are also disjoint, which implies either $|A| \leq \binom{n/2}{k} = 2^{-\Omega(k)} \binom{n}{k}$ or $|B| \leq \binom{n/2}{\ell} = 2^{-\Omega(\ell)} \binom{n}{\ell}$. Hence, if $k \leq \frac{n}{4}$, it follows that $t \geq \Omega(k)$.

⁸There are also lower-bounds on the asymmetric communication complexity of the membership problem, but these are too small to be relevant for static data structures.

- *The span problem.* Let $f : \{0,1\}^n \times \{0,1\}^{\frac{n}{2} \times n} \rightarrow \{0,1\}$ so that $f(q, x) = 1$ iff $x \cdot q = 0$.⁹ Then (for any $\varepsilon > 0$) f is $(2^{(1-o(1))n^2}, 2^{(1-\varepsilon)n})$ -rich, but any 1-monochromatic rectangle $A \times B$ has $|A| \leq 2^{n/3}$ or $|B| \leq 2^{n^2/6}$. Hence $t \geq \Omega(\frac{n}{\log n})$.

It is not obvious how to naturalize these proofs. The issue is that, naively, the hardness-implying property $\Phi(f)$ would say that f is sufficiently rich, which is easy to check when given f as a table, and that f does not have large 1-monochromatic rectangles. This latter predicate, however, is equivalent to deciding whether a given bipartite graph has a large biclique, which is an NP-complete problem [Joh87]. After all, we are quantifying existentially over all rectangles, which are exponentially-many in $|f|$.

The key observation that helps us is that all of the functions above are not only free of 1-monochromatic rectangles, they also fail to have any nearly-1-monochromatic rectangles. In graph terminology, it's not just that they fail to have large bicliques, they even fail to have large near-bicliques. It is then possible to use a polynomial time algorithm that finds a near-biclique of a given size.

Definition 5.3. A δ -nearly-1-monochromatic rectangle is one such that $\frac{|f^{-1}(1) \cap A \times B|}{|A \times B|} \geq \delta$. $\square$

First, let us state the claims with respect to the above problems. Pay attention to the quadratic loss in the disjointness problem: avoiding it requires a long technical digression, which at this point would be a distraction.

Lemma 5.4. We can prove the following upper-bounds on the size of nearly-1-monochromatic rectangles:

- *Lopsided Set disjointness.* Let f be the lopsided set disjointness problem given above. Suppose k and ℓ are chosen with $k \cdot \ell = n$ (as in the birthday paradox, but here $\ell \gg k$ is allowed). Then there exists some constant $c > 0$ such that every $(1 - \varepsilon)$ -nearly-1-monochromatic rectangle $A \times B$ has $|A| \leq |\mathcal{Q}| \cdot 2^{-c \cdot k}$ or $|B| \leq |\mathcal{D}| \cdot 2^{-c \cdot \ell}$. This is proven by an argument of Babai, Frankl, and Simon [BFS86]. The quadratic loss is because we require $k \cdot \ell \leq n$, whereas in general we could have $k \cdot \ell = \Theta(n^2)$.
- *The Span problem.* Let f be the span problem given above. Then there exists some constant $c > 0$ such that any $(1 - \varepsilon)$ -nearly-1-monochromatic rectangle $A \times B$ has $|A| \leq |\mathcal{Q}| \cdot 2^{-cn}$ or $|B| \leq |\mathcal{D}| \cdot 2^{-cn^2}$.

Proof. For disjointness. Suppose $|A| \geq \binom{n}{k} 2^{-ck}$. Assume without loss of generality (by Markov's inequality) that every row in $A \times B$ is $1 - 2\varepsilon$ -nearly-1-monochromatic. We claim that we can find $m = \frac{n}{3k} = \Theta(\ell)$ sets $q_1, \dots, q_m \in A$ with $|q_i| \cup_{j < i} q_j| \geq \frac{k}{2}$. I.e. each q_i adds at least $\frac{k}{2}$ new elements. This is because, letting $z = \bigcup_{j < i} q_j$, we have $|z| \leq (i - 1)k < mk \leq \frac{n}{3}$. The number of $q \in \mathcal{D}$ such that $|q \cap z| > \frac{k}{2}$ is less than¹⁰ $n \binom{n/3}{k/2} \binom{2n/3}{k/2} =$

⁹This problem is sometimes defined so that V is an $\frac{n}{2}$ -dimensional subspace and $f(q, V) = 1$ iff q is in V , here we have simply encoded $V = \text{span}(x_1, \dots, x_{n/2})^\perp$.

¹⁰Using the estimate $\frac{1}{\sqrt{nk}} \left(\frac{2n}{k}\right)^k \leq \binom{n}{k} \leq \sqrt{n} \cdot \left(\frac{en}{k}\right)^k$, which follows from Stirling's approximation.

$n^{\Theta(1)} \left(\frac{2}{3}\right)^{\frac{k}{2}} \left(\frac{1}{3}\right)^{\frac{k}{2}} \left(\frac{e}{2}\right)^k \left(\frac{2n}{k}\right)^k \leq \binom{n}{k} 2^{-ck} = |A|$, therefore some $q_i \in A$ has $|q_i \cap z| \leq \frac{k}{2}$. Again by Markov, some subset $B' \subseteq B$ of $|B'| \geq \frac{|B|}{2}$ intersects at most $4\epsilon m$ of the queries $q_1, \dots, q_m$. So any $x \in B'$ must avoid at least $(1 - 4\epsilon)k$ of the q_i , and the union of any such set of q_i has size at least $(1 - 4\epsilon)\frac{mk}{2} \geq \frac{n}{7}$. Hence the number of such x is at most $\binom{k}{4\epsilon k} \binom{6n/7}{\ell} < \binom{n}{\ell} 2^{-c\ell} = |\mathcal{D}| 2^{-c\ell}$.

For the span problem. Suppose $|A| \geq 2^{\frac{1}{2}n}$, and assume again w.l.o.g. that every row in $A \times B$ is $(1 - 2\epsilon)$ -nearly-1-monochromatic. It follows that it spans a subspace of dimension at least $\frac{n}{2}$, so let $q_1, \dots, q_m$, $m = \frac{n}{2}$, be a basis of this space. Again by Markov, some subset $B' \subseteq B$ of $|B'| \geq \frac{|B|}{2}$ columns has, for every $x \in B$, $f(q_i, x) = 0$ for at most $4\epsilon m$ q_i . So the rows of every $x \in B'$ must be chosen so that their span avoids some space of dimension $(1 - 4\epsilon)\frac{n}{2}$. There are at most $\binom{m}{4\epsilon m} (2^{(1-4\epsilon)\frac{n}{2}})^{\frac{n}{2}} \leq |\mathcal{D}| \cdot 2^{-cn^2}$ such choices for x . $\square$

The following was proven by Siddharth Barman [Bar18]:¹¹

Theorem 5.5 (Theorem 5.5 in [Bar18]). *There exists an algorithm which, when given a table $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$ and a parameter k , such that f has a 1-monochromatic rectangle of size $k \times k$, finds a $(1 - \epsilon)$ -nearly-1-monochromatic rectangle of size $k \times k$. This algorithm runs in time $2^{\frac{(\log(|\mathcal{Q}| + |\mathcal{D}|))^2}{\epsilon^2}}$.*

Our natural property $\Phi(f)$ then states that f is sufficiently rich, and Barman's algorithm fails to output a large nearly-monochromatic rectangle. This property is:

- Constructive because the algorithm is efficient: it runs in time quasipolynomial in $|f|$, and if our conjecture holds then a polytime algorithm exists which is good enough for our purpose.
- Useful by the Richness Lemma, which implies that any sufficiently rich, sufficiently easy f has a large 1-monochromatic rectangle, the existence of which ensures that the algorithm finds a large nearly-monochromatic rectangle.
- Large, because w.h.p. every large rectangle is unbiased for a random function f . Indeed if we consider large enough rectangles ($\text{polylog}(|f|)$ will do), the exponential concentration around $1/2$ is so high, that this every-large-rectangle-is-unbiased property happens with probability $1 - |f|^{-\Omega(1)}$.

Hence, modulo the (important) quadratic loss, all the above applications of the richness method are strongly natural proofs. Let us now discuss how one might avoid this loss:

¹¹Although it is well beyond the scope of this paper, we speculate that the running time can be improved to a polynomial, if we allow for a gap between the size $k \times k$ of the 1-monochromatic rectangle (i.e. biclique) which is assumed to exist, and the size $k' \times k'$ of the $(1 - \epsilon)$ -nearly-1-monochromatic rectangle which is to be found.

- The proof of non-existence of a nearly-1-monochromatic rectangle for disjointness is the very same proof by Babai, Frankl, and Simon [BFS86], where they show the corruption bound of $\Omega(\sqrt{n})$ for disjointness under a certain product distribution. The result was later improved to $\Omega(n)$ by Razborov [Raz92], using a non-product hard distribution. Both proofs can be found in [CP10].
- [BFS86] had also shown that $\Omega(\sqrt{n})$ is the best-possible lower-bound under product distributions. This is the same reason why we have a quadratic gap.
- To overcome this gap, we need a non-product version of the richness lemma. Let $d, p \in [0, 1]$. Given some distribution μ over $\mathcal{Q} \times \mathcal{D}$, we say that $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$ is (μ, d, p) -rich if there exists a set $B \subseteq \mathcal{D}$ with marginal mass $\mu(B) \geq d$ such that, for every $x \in B$, the conditional mass is $\mu(f^{-1}(1) \cap \mathcal{Q} \times \{x\} \mid x) \geq p$. The same proof of the richness lemma now shows that, if f is (μ, d, p) -rich and has an (s, w, t) -solution, there exists a 1-monochromatic rectangle $A \times B$ with $\mu(B) \geq 2^{-t(w+\log s)}d$ and, for every $x \in B$, $\mu(A \mid x) \geq 2^{-t \log s}$.
- Using the same argument as in [Raz92], we can construct a distribution μ over the inputs to Lopsided Set Disjointness, and prove that no nearly-1-monochromatic rectangle can exist with such a large mass. Here “nearly” is with respect to μ -mass.
- We are now left to check that the proof has become natural. For this, we need to look at Barman’s algorithm, and realize that it will also work not just to find nearly-monochromatic cliques in unweighted bipartite graphs, but also in graphs where the edges have been given some weight μ .
- Incidentally, from the preceeding discussion one has also understood that Barman’s algorithm implies that lower-bounds based on the corruption method are natural proofs. This is not at all obvious for the same reason: one cannot naively test every single sub-rectangle.

As you can see, the work required to fully naturalize the set disjointness lower-bound is substantial and rather technical, it sidesteps the main message of we wish to convey, and there is a lot more to cover, so we will leave it for the journal version of the paper. We believe that there are no real obstacles to doing it. From the next paragraph, it already follows that the tight disjointness lower-bound is essentially natural (Definition 3.3), which further diminishes the importance of this technical addendum.

Our candidate PRF. We are only left to check, as promised, that our candidate passes the above natural property. As we have mentioned at the end of Section 4.2, if we choose some large enough constant c , then, for every choice of n^c columns, the marginal distribution of f is uniformly random on those columns, hence any subrectangle is unbiased, except with probability $2^{-\Omega(n^c)}$, which is enough to union bound over the choice of columns. It follows that every subrectangle of our pseudorandom function is unbiased: f will not just have large corruption, it will have large discrepancy.

5.2 Round Elimination

Now we turn to round elimination. Round elimination first appeared implicitly in a paper by Ajtai [Ajt88], and eventually was formalized using communication complexity in [MNSW98]. This style of argument has been used many times, in many different variations, to prove static data-structure lower bounds [BF02, CCGL03, PT06b, SV08, CR10, PT07, LPY16, GL16, SW19, AS25]. Here we will sketch the simplest proof available, for a simple, deterministic version of the round-elimination lemma. This proof consists of an information-theoretic argument by Sen and Venkatesh [SV08].

Round elimination is another good example of a proof technique which does not obviously naturalize. This time, the reason why the proof is not obviously natural is not because there is an obviously large property which is not obviously constructive. Instead, it is because the proof uses very specific properties of the functions involved in the lower-bound. More precisely it makes use of a certain kind of self-reducibility, where certain restrictions of the function are equal to the original function on smaller inputs, and this self-reducibility is simply not true of a random function. Nonetheless, the self-reducibility can be relaxed into a natural property, as we are about to see.

Again let's assume that $s = n^{O(1)}$ and $w = O(\log n)$. The existence of an (s, w, t) -solution for a static data-structure problem $f(q, x)$ naturally gives rise to a protocol where Alice gets q , Bob gets x , and then Alice plays the role of the decision-tree, and Bob gives the answers to the probes, so this protocol has $r = 2t$ rounds of communication, in odd rounds Alice communicates $a = \log s$ bits, and in even rounds Bob communicates $b = w$ bits. Let's call such a protocol a $[r, a, b]^A$ -protocol, since Alice starts. Likewise we define an $[r, a, b]^B$ protocol, if Bob were to start.

Suppose we have a family of functions $f_n : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \mathcal{A}$. The lemma states that given an $[r, a, b]^A$ -protocol for f_n , where a is very small, we can obtain an $[r - 1, a, b]^B$ -protocol that works for a sub-function of f . This is proven using the chain rule. We imagine that the input q, x are chosen uniformly at random. We then divide the input q into $\ell = \omega(a)$ blocks $q = q_1 \dots q_\ell$, and the chain rule implies that there exists one of the blocks $i \in [\ell]$ about which the first message π_1 of Alice reveals very little information. So, say, $I(\pi_1 : q_i \mid q_1, \dots, q_{i-1}) \leq \frac{a}{\ell} = o(1)$. It then follows that the first message is close to being independent of the i -th block. Hence there exists some fixing of $q_{<i} = q_1, \dots, q_{i-1}$ and π_1 , such that q_i conditioned on this fixing still has near maximal entropy $|q_i| - o(1) = \frac{n}{\ell} - o(1)$. So, by Pinsker's inequality, for a $1 - o(1)$ fraction $Q' \subseteq \{0, 1\}^{\frac{n}{\ell}}$ of all possible q_i , there exists some way of choosing $q_{>i} = q_{i+1}, \dots, q_\ell$ which is compatible with $q_{<i}$, q_i and π_1 . In this way we have defined a map $q_i \mapsto q_{<i} q_i q_{>i}$ from $Q' \subseteq \{0, 1\}^{\frac{n}{\ell}}$ to $\{0, 1\}^n$.

Alright, now suppose that the family of functions f_n has the following self-reducibility property. If we wish to compute $f_n^{\frac{n}{\ell}}(q_i, x_i)$, for some $(q_i, x_i) \in \{0, 1\}^{\frac{n}{\ell}} \times \{0, 1\}^{\frac{n}{\ell}}$, and we have a map $q_i \mapsto q = q_{<i} q_i q_{>i}$ where $q_{<i}$ is fixed (but $q_{>i}$ might not be), then we can construct a map $x_i \mapsto x = x_{<i} x_i x_{>i}$ such that $f_n^{\frac{n}{\ell}}(q_i, x_i) = f_n(x, q)$. Then, when Alice is given $q_i \in Q'$ and Bob is given $x_i \in \{0, 1\}^{\frac{n}{\ell}}$, they can run the above protocol for $f(q, x)$, by skipping the first message and pretending it was equal to π_1 . This is a $[r - 1, a, b]^B$

protocol for $f_{\frac{n}{\ell}}$ (restricted to $Q' \times \{0,1\}^{\frac{n}{\ell}}$). If the same trick can be applied in the other direction, this eventually gives us a zero-round protocol for a non-trivial function, which is a contradiction.

Here is an example of a function with this self-reducibility property, for which [MNSW98] have proven lower-bounds:

- The greater-than function $\text{GT}_n : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ such that $\text{GT}_n(x, y) = 1$ iff $x > y$ when interpreted as natural numbers written in binary. Indeed, if we split x into blocks $x_1 \dots x_\ell$, and fixed some prefix $x_1 \dots x_{i-1}$, then by setting $y = x_1 \dots x_{i-1} y_i 0 \dots 0$, we have $\text{GT}_{\frac{n}{\ell}}(x_i, y_i) = \text{GT}_n(x, y)$. It follows from the above reasoning that there does not exist a $[k, o(n^{1/k}), o(n^{1/k})]$ -protocol for GT_n , so in particular $t = \Omega(\log n)$ for any $(2^{n^{o(1)}}, n^{o(1)}, t)$ -solution.¹²

Clearly, a random function is not self-reducible in this way. However, by looking carefully at the above proof, what is needed is not, strictly speaking, that the function f is self-reducible. What is needed is that we are free to fix the inputs in certain ways so that, even after doing t rounds of such fixing, the resulting rectangle is not monochromatic. So one could generalize the useful (hardness-implying) property used in the round elimination lemma, to be something similar to the following.

The predicate $\Phi(f)$ is true if, for every choice i_1 of the block used to eliminate the first round, for every prefix $q_1, \dots, q_{i_1-1}$ that we might have fixed in the proof, to ensure that $I(x_{i_1} : \pi_1 \mid x_1 = q_1, \dots, x_{i_1-1} = q_{i_1-1})$ is small, for every setting of q_{i_1} , for every suffix $q_{i_1+1}, \dots, q_\ell$, and for every choice x_i of the i_1 -th block of x , there exists some choice $x_{<i_1}$ and $x_{>i_1}$ for the remaining blocks, such that, for every choice i_2 of the block used to eliminate the second round, for every prefix $\dots$ etc, $\dots$ the subfunction of f conditioned on all the fixing done so far, is not constant. This predicate is useful: any function f obeying this predicate is hard by the same argument used in the round-elimination lemma, and the functions f considered above all have $\Phi(f) = 1$. The property Φ obeys largeness, since random functions have no large monochromatic rectangles, in particular it doesn't matter how the bits of the input are partially fixed, that if at least, say, $10 \log n$ bits are left unfixed, the resulting subfunction of f is not constant. As for constructivity, this might at first appear to be a very complicated predicate, but notice that the number of bits being quantified is $O(n + t \log n)$, which is $O(n)$ for all the lower-bounds proven above. Hence, an algorithm can evaluate this predicate in time $2^{O(n)} = |f|^{O(1)}$. Hence Φ is a constructive property, and so we have just naturalized the round-elimination lemma.

Our candidate PRF. By the same observation as above, that our pseudorandom function f has no large monochromatic rectangles, we also have $\Phi(f) = 1$ with high probability.

¹²Their lower-bound is randomized. A fun exercise is to prove a matching randomized solution to the data-structure problem with $s = n^{O(1)}$, $w = \Theta(\log n)$, and $t = O(\log n)$. This is simple, but only given the known literature on GT_n .

5.3 Cell sampling arguments

The cell-sampling method first appeared in [PTW10], and was used in several static data-structure lower-bounds (e.g. [AV15, PY22]). More notably, it was used together with the chronogram method to prove super-logarithmic dynamic data-structure lower-bounds (we will see these in Sections 7.5 and 7.6).

Let us illustrate how the cell sampling method works, by proving a lower-bound against deterministic solutions. For simplicity suppose the data is $\mathcal{D} = [p]^n$, and the queries are $\mathcal{Q} = [m]$, with $n^{1+\Omega(1)} \leq m \leq \text{poly}(n)$, $2 \leq p \leq \text{poly}(n)$. Suppose that there exists a solution for $f : \mathcal{Q} \times \mathcal{D} \rightarrow [p]$ which makes $t = o(\frac{\log m}{\log \frac{sw}{n \log p}})$ queries. Now choose a random set $C \subseteq [s]$ of $|C| = \frac{n \log p}{2sw} + t$ cells¹³: there are $\binom{s}{|C|}$ many such choices. For a pair (q, x) , how many “good” choices for C will be such that the decision tree T_q , on input $E(x)$, only queries cells in C ? Exactly $\binom{s-t}{|C|-t}$. The probability that C is such a good choice is then at least

$$\frac{\binom{s-t}{|C|-t}}{\binom{s}{|C|}} = \frac{(s-t)!|C|!}{s!(|C|-t)!} \geq \left(\frac{|C|-t}{s}\right)^t = \left(\frac{n \log p}{2sw}\right)^t = m^{-o(1)}.$$

For each x , then, such a random C is good for an expected number of $\geq m^{1-o(1)} = n^{1+\Omega(1)}$ queries q . By an averaging argument, there is set $X \subseteq \{0, 1\}^n$ containing a large fraction $m^{-o(1)}$ of all possible databases, such that for every $x \in X$, there exist $m^{1-o(1)}$ queries $Q(x) \subseteq \mathcal{Q}$ such that, for every $q \in Q(x)$, the decision tree for computing $f(q, x)$ only probes the cells in C .

One then appeals to the redundancy of f . The prototypical example is when f is polynomial evaluation, so x is a polynomial, q is a field element, and $f(q, x)$ is the evaluation of the polynomial x at p . Data structure problems such as this have a redundancy property, so that x can always be determined when we know $f(q, x)$ for at least $m^{1-o(1)}$ different choices of q . It follows that any $x \in X$ can be encoded by knowing only the contents of the cells in C , since knowing these contents allows us to answer $m^{1-o(1)}$ different queries. This encoding has $|C(x)|w \leq \frac{n}{2}$ bits, which is less than $\log |X| \geq n - O(\log m)$, a contradiction.

This basic lower-bound strategy was used in many papers to prove static data structure lower-bounds. The proofs are essentially what we just explained, but with additional technical details that allow the lower-bound to work against randomized solutions.

The hardness-implying property would be then that f is redundant. However, in the case when $p \ll m$, such as in the binary case $p = 2$, a random f is not likely to be redundant, in the above strict sense: we must relax the notion of redundancy a little. The strict notion above states that, for any set $Q = \{q_1, \dots, q_k\}$ of size $k \geq m^{1-o(1)}$, for any $x \in [p]^n$, the string $f(Q, x) = f(q_1, x), \dots, f(q_k, x) \in [p]^k$ is unique, i.e. $f(Q, \cdot) : [p]^n \rightarrow [p]^k$ is injective. But when $p = 2$ this is not likely to be true for a random function: we will

¹³Which is reasonable to assume is less than s , since the s cells of w bits must encode x and t is small.

always be able to find two databases x_1, x_2 such that the map $f(\cdot, x_1)$ and $f(\cdot, x_2)$ agree on at least $\Omega(m) \gg k$ coordinates, so the redundancy property will fail for the set Q of such coordinates.

So, instead of requiring $f(Q, \cdot)$ to be one-to-one for every set Q of size $k \geq m^{1-o(1)}$, we require it to be few-to-one. Namely, our property $\Phi(f)$ is as follows (for some sufficiently small $\varepsilon > 0$): *for every set $Q \subseteq [m]$ of size $k = n^{1+\varepsilon}$, the map $f_Q : [p]^n \rightarrow [p]^k$ given by $f_Q(x) = f(Q, x)$ will have $|f_Q^{-1}(z)| \leq n$ for every $z \in [p]^k$.* The property is useful: the same argument as above works, we append to the $\frac{n}{2}$ bits containing the contents of the cells in C , which allows us to learn $z = f_{Q(x)}(x)$ for some set $Q(x)$ of size $\geq k$, a pointer to x in the set $f_{Q(x)}^{-1}(z)$. This pointer is only $\log n$ bits long. The property is also natural: the algorithm goes over all sets Q , and counts the size of all pre-images $f_Q^{-1}(z)$. This runs in time $2^{\text{poly}(n)}$ which is quasipolynomial in $|f|$.

Finally, the property is also large. Choose $f : [m] \times [p]^n \rightarrow [p]$ at random. Fix any set $Q \subseteq [m]$ of size $k = n^{1+\varepsilon}$, and any set $X \subseteq [p]^n$ of size $|X| \geq n+1$. What is the probability that $f(q, x)$ is constant on $Q \times X$? It is at most p^{-kn} , since $f_Q(x) \in [p]^k$ must be the same on all $n+1$ choices of $x \in X$. This is much less than the inverse of $\binom{m}{k} \binom{p^n}{n+1} \leq p^{n^{1+\varepsilon} \log m + n(n+1)}$, so by a union bound with high probability there will exist no such Q and X .

Our candidate PRF. Now, as we have seen, if f is chosen according to our local PRF candidates, any fixed choice X of $n+1$ columns will have uniformly random entries. Hence the probability that $f(q, x)$ is constant on $Q \times X$ can be upper-bounded in the same way, and the union bound still applies. Hence $\Phi(f) = 1$ with high probability.

5.4 Proofs via CNF refutation techniques

The paper by Korten, Pitasi and Impagliazzo proves lower-bounds using two different techniques. The first of these lower-bounds is a natural proof, as was already stated in that paper, here we will further check that our PRF satisfies the natural property. The second lower-bound does not appear to be natural, but here we will show that it is essentially natural (as per [Definition 3.3](#)).

Let us say that a distribution μ over $\{0, 1\}^{|Q|}$ is γ -almost k -wise independent if for every choice Q of $|Q| = k$ coordinates, and every possible setting $\alpha_1 \dots \alpha_k \in \{0, 1\}^k$, we have $\Pr_{a \sim \mu}[a_S = \alpha] \in [2^{-k} - \gamma, 2^{-k} + \gamma]$.

Theorem 5.6 (Theorem 1 of [\[KPI25\]](#)). *Let $t > 1, t = O(1)$ be a fixed even number, $s, t, w, k \in \mathbb{N}$ be such that $k \geq tw + 1$. Let $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$ be a table such that there exists a γ -almost k -wise-independent distribution supported on its columns $f(\cdot, x)$, with $\gamma \leq |\mathcal{Q}|^{-t} 2^{-O(ktw)}$. Then any (s, w, t) -solution for f must satisfy:*

$$1. s \geq \left(\frac{|\mathcal{Q}|}{\log |\mathcal{Q}|} \right)^{\frac{2}{t}} \left(\frac{k}{\log |\mathcal{Q}|} \right)^{1 - \frac{2}{t}} t^{-1} 2^{-O(w)} \text{ for all } k \in [t \log |\mathcal{Q}|, |\mathcal{Q}| 2^{-O(tw)} t^{-\frac{t}{2}}].$$

$$2. s \geq \left(\frac{|\mathcal{Q}|}{\log |\mathcal{Q}|} \right)^{\frac{1}{t(\frac{1}{2} + \frac{2}{k})}} t^{-1} 2^{-O(w)} \text{ for all } tw + 1 \leq k \leq t \log |\mathcal{Q}|.$$

Suppose that the table f is chosen uniformly at random and we are in the regime where $|\mathcal{D}| \gg |\mathcal{Q}|^{\omega(t^2 w)}$, for $w = \Theta(\log \log |\mathcal{D}|)$. This is a standard regime for $|\mathcal{D}|$, $|\mathcal{Q}|$ and w (usually the number of databases is much larger). Then let us show that the distribution which samples a random column from f is γ -almost k -wise independent, where $k = t \log |\mathcal{Q}|$ and $\gamma = |\mathcal{D}|^{-\frac{1}{4}} = |\mathcal{Q}|^{-\omega(t)} 2^{-\omega(ktw)}$. Indeed, choose any k rows of f , and fix any $\alpha \in \{0, 1\}^k$. When sampling the contents of f on those rows, each column will equal to α with probability exactly 2^{-k} . By a Chernoff bound, the probability that the number of seen α is not within $(1 \pm \gamma)2^{-k}|\mathcal{D}|$ is at most $2 \exp(-\frac{1}{3}\gamma^2 2^{-k}|\mathcal{D}|) = \exp(-\Omega(|\mathcal{D}|^{1/2})) \ll 2^{|\mathcal{Q}|}$, so we may union bound over the choice of k queries. To see that our PRG fools this property, recall that any $\omega(n) = \omega(\log |\mathcal{Q}|) \gg t \log |\mathcal{Q}|$ rows of our PRF have uniform marginal distribution, hence the same argument applies.

The second technique in [KPI25] uses a property of the table which they call being γ -biased. A table $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{+1, -1\}$ has this property if the following happens. A distribution over $\{+1, -1\}^{|\mathcal{Q}|}$ is called γ -biased if, for every non-empty $S \subseteq \mathcal{Q}$, the expected value of the XOR is $|\mathbb{E}[\prod_{i \in S} X_i]| \leq \gamma$. We say that f is γ -biased the distribution obtained by sampling a random column $x \in \mathcal{D}$ and outputting $X = (f(q, x))_{q \in \mathcal{Q}}$ is γ -biased. The theorem they prove is equivalent to the following:

Theorem 5.7 (Theorem 2 of [KPI25]). *Let $t > 1, t = O(1)$ be a fixed odd number. Let $f : \mathcal{Q} \times \mathcal{D} \rightarrow \{0, 1\}$ be a $|\mathcal{Q}|^{-c}$ -biased table, where $c = O_t(1)$ is a sufficiently large constant (depending on t). Then any $(s, 1, t)$ -solution for f must satisfy:*

$$s \geq \tilde{\Omega}(|\mathcal{Q}|^{\frac{2}{t} + \varepsilon_t})$$

where $\varepsilon_t > 0$ is a positive constant depending only on t .

The naive tester for the property of being γ -biased will take time $\Omega(2^{|\mathcal{Q}|})$, which is sometimes larger than quasipolynomial in the size of the table f . This is the case, e.g., for the VMV frame, and for the frame of the (artificial but constructive) problems for which they prove a lower-bound, but in the case of the ORC frame we actually have $2^{|\mathcal{Q}|} = |f|^{O(1)}$, so sometimes this naive algorithm does run efficiently in $|f|$.

A random table will be $2^{-\Omega(|\mathcal{D}|)} = |\mathcal{Q}|^{-\omega(1)}$ -biased, so the only question remains whether testing if a table is $|\mathcal{Q}|^{-c}$ -biased can be done efficiently. This seems unlikely, since the problem is equivalent to asking if the XOR of any non-zero number of rows of f has Hamming weight between $(\frac{1}{2} - |\mathcal{Q}|^{-c})|\mathcal{D}|$ and $(\frac{1}{2} + |\mathcal{Q}|^{-c})|\mathcal{D}|$, which is very similar to the problem of testing whether minimum distance of the $\mathbb{F}_2$ -linear code generated by the rows is at least $(\frac{1}{2} - |\mathcal{Q}|^{-c})|\mathcal{D}|$. This latter problem is hard to approximate [DMS03]. One might speculate as follows: for the proof to work, it is sufficient that there exists some set $\mathcal{D}' \subseteq \mathcal{D}$ of columns such that the corresponding subtable of f has the required two properties (1) and (2). It might be easier to decide whether this is the case (although we

do not immediately see how). But also, the proof might be naturalizable in some other way which we don't currently realize.

Regardless, we note that this property is fooled by our candidate PRF. Namely, let us show that a function f sampled by it will have a $|\mathcal{Q}|^{-c}$ -biased subtable with high probability. Specifically we will show that $f_1 = f_q(E_1(x)|_{S_q})$ will already have small bias (which is enough so that $f = f_1 \oplus f_2$ to also have small bias). This follows from the following sequence of observations:

- Take any arbitrary choice of sets S_q , and recall that $|S_q| = t$. Recall that the parameter t in the construction of our PRF is as large as: $t = \omega(\log n + \log |\mathcal{Q}|)$. Therefore, let $\ell = \omega(1)$ be super-constant, but still small enough so that the parameter t in the construction is $t = \omega(\ell(\log n + \log |\mathcal{Q}|))$. Then let $\mathcal{D}' \subseteq \mathcal{D}$ contain the first $|\mathcal{Q}|^\ell$ databases/columns of our table.
- Fix any query q . Each database $x \in \mathcal{D}'$ is given a random encoding $E_1(x) \in \{0,1\}^{n \log |\mathcal{Q}|}$. When we choose the S_q -bits of the encodings $e = E_1(x)|_{S_q}$ and $e' = E_1(x')|_{S_q}$ of two strings $x \neq x'$, these encodings will be different except with probability $2^{-\Omega(t)}$. The probability, then, that any two encodings in $\mathcal{D}'$ have the same S_q -bits is $\leq |\mathcal{D}'|2^{-t} \leq 2^{-\Omega(t)} \ll |\mathcal{Q}|^{-1}$. It follows that, with high probability over the choice of E_1 , for all queries q and all pairs $x \neq x'$ of databases in $\mathcal{D}'$, the encodings e and e' are different.
- Hence, the random choice of the functions f_q will give us a completely random subtable $f' : \mathcal{Q} \times \mathcal{D}' \rightarrow \{0,1\}$ of f . This random subtable is, indeed, $|\mathcal{Q}|^{-\omega(1)}$ -biased, and hence it will be hard according to [Theorem 5.7](#).

In summary, although being γ -biased is not a natural property — it is large but not constructive — this property is nonetheless fooled by our PRF, and thus cannot be used — either on its own or in conjunction with other natural properties — to prove lower-bounds beyond the state of the art.

5.5 The single exception we know: Lifting theorems

There is a single known application of lifting theorems in proving static data-structure lower-bounds, appearing in [\[CKLM18\]](#). Lifting theorems fail to be natural because they only address functions of the form $f = h \circ g$, and most functions are not of composed form, so the largeness property fails. However, in lifting theorems the gadget g is usually fixed, and it is true that lifting theorems will prove a lower-bound against $h \circ g$ for a random choice of h ; the lower-bound is then usually related with a simple complexity measure on h , such as decision-tree depth, and this complexity measure can be efficiently decided. So in this limited sense the used property is large (among composed functions) and constructive, and one might informally say that the proof has the flavor of a “natural” proof.

Nonetheless, lifting theorems are not natural proofs in the strict sense we have defined here (they are not even essentially natural), and one might hope that they could prove lower-bounds beyond what is currently known. This is also the main hope of the KRW conjecture, as an approach to proving lower-bounds against NC_1 [KRW95]. However, in this context, the paper [CKLM18] proves static data-structure lower-bounds via asymmetric communication complexity, and any such approach will fail to prove any lower-bound of the form $t_q = \omega(\log |Q|)$, simply because a communication protocol exists that uses $\log |Q|$ communication. There is a possibility, however, that lifting theorems could be designed specifically for data-structures, which might allow us to beat the natural proofs barrier. Such a proof would still have to contend with the remaining barriers [DGW19, RR20, Vio19], but lifting theorems are sophisticated methods of proving lower-bounds, so perhaps they can be used to prove such strong results. However, the authors of [CKLM18], two of whom also co-author this paper, have already tried to do precisely that: new ideas will be needed.

6 Proposal: A natural proofs barrier against dynamic data structure lower bounds

Recall that in the dynamic setting the frame includes a set of updates. In this section we restrict attention to Hamming Frames (see Example 2.7) which are the frames that arise in most dynamic problems of interest.

The problem which best illustrates where we are stuck in dynamic data-structure lower-bounds is the dynamic reachability problem. Our data set is $\mathcal{D} = \{0,1\}^{n^2}$, the set of all directed graphs on n vertices. Our query set is $\mathcal{Q} = [n] \times [n]$ with Boolean answers $\mathcal{A} = \{0,1\}$, so that $f((i,j),x) \in \{0,1\}$, for $x \in \mathcal{D}$, indicates whether there exists a (directed) path from i to j in the directed graph x . The update set consists of the Hamming updates $u_{(i,j),\tau}$ where $\tau : \{0,1\} \rightarrow \{0,1\}$ is $\tau(\alpha) = \alpha \oplus 1$. Thus $u_{(i,j),\tau}(x) = x \oplus e_{(i,j)}$ flips the presence of the edge (i,j) . (Strictly speaking, the (i,i) edges are not necessary, but we will ignore this detail to make notation less cumbersome.) This defines a dynamic data-structure frame $\mathcal{F} = (\{0,1\}^{n^2}, [n]^2, \{0,1\}, [n]^2)$ (where for simplicity we have identified the set of updates with the set $[n]^2$). We call this the *reachability frame*.

The best known upper-bounds for this problem allow for various tradeoffs between query and update time¹⁴, but in every case we have $\max(t_q, t_u) = n^{\Omega(1)}$. In contrast, the best known unconditional lower-bound is $\max(t_u, t_q) \geq \tilde{\Omega}((\log n)^{3/2})$ [LY23]. There is some hope, perhaps, to improve this lower-bound to $\tilde{\Omega}((\log n)^2)$ — such a lower-bound is known for some (non-Boolean) problems (see Section 7.5), even if $\tilde{\Omega}((\log n)^{3/2})$ is the best that is known for any problem with Boolean answers.

There is good reason to believe that these lower-bounds are far from optimal: we have strong *conditional* lower-bounds. Indeed, assuming either the Strong Exponential-Time Hypothesis (SETH), or the usual conjectured hardness of the 3SUM or triangle detection problems, or the OMv conjecture, it follows that dynamic reachability requires n^ε query or update time, for some $\varepsilon > 0$ depending on the specific conjecture used [Pät10, AW14, HKNS15, BNS19]. Nonetheless, the best *unconditional* lower-bound that has been proven for *any* dynamic data-structure problem with $n^{\Theta(1)}$ queries and updates is $\tilde{\Omega}((\log n)^2)$.

Here we propose that, also in this dynamic setting, there exists a natural-proofs barrier which explains precisely why we are stuck where we are. So, in Section 6.1, we will define a notion of a function generator, called a *local and locally updatable (LLU) function generator*, and propose the conjecture that such a function generator exists which is sufficiently secure to rule out natural lower-bound proofs. As far as we know, this is the first fully general barrier result which applies to *dynamic* data-structure lower-bounds.

It should be said, however, that our level of confidence on the existence of a LLU PRG is not as high as in the static case. In fact, the notion of *locally updatable* will have two versions, a natural notion of locally updatable derived directly from the existence of a dynamic data structure solution, and another, less stringent notion of locally updatable func-

¹⁴For example, $t_q = O(n)$, $t_u = O(m + n \log n)$ [RZ04] (m is the number of edges), $t_q = n^{0.575}$, $t_u = n^{1.575}$ [San04], and $t_q = t_u = n^{1.407}$ [BNS19].

tion generator (Relaxed Local-and-Locally-Updatable, RLLU) derived from the existence of a *relaxed* solution. The difference between solutions and relaxed solutions is as follows: in a solution to a dynamic data-structure problem, for each possible update $u \in \mathcal{U}$ there must exist a $\leq t_u$ -depth read-write decision tree T_u which transforms an encoding $E(x)$ to an updated encoding $E(u(x))$. The fact that T_u is low-depth implies a bound on the Hamming distance: $\Delta_H(E(x), E(u(x))) \leq t_u$. In a relaxed solution, there does not need to exist such a decision tree T_u ; instead, the *only requirement* is that $\Delta_H(E(x), E(u(x))) \leq t_u$ for all $x \in \mathcal{D}, u \in \mathcal{U}$.

In [Section 6.2](#), we construct a candidate RLLU function generator which we believe is secure. At first, it might appear that we have relaxed the notion of a function generator too much: indeed the requirement that the Hamming distance between $E(x)$ and $E(u(x))$ appears to be a weak requirement, it gives no algorithmic guarantees. Nonetheless, in [Section 7](#), we will show that *every single* dynamic lower-bound proof in the literature is a natural proof, which proves lower-bounds not just against solutions, but also against relaxed solutions! So whatever difference there might exist between solutions and relaxed solutions, the known lower-bound techniques fail to make use of it.

The different possibilities are then as follows (with parameters matching the state-of-the-art in dynamic data-structure lower-bounds):

- Secure LLU PRFs exist, in which case no natural proof of dynamic data-structure lower-bounds is possible beyond the current state of the art.
- Or perhaps only secure RLLU PRFs exist (for example, our candidate), in which case a natural proof of dynamic data-structure lower-bounds might still be possible, but such a proof would have to (somehow, unlike all proofs in the literature) use something about the update decision-trees T_u , other than the fact that $T_u(E(x))$ is close to $E(x)$ in Hamming distance. This is a real possibility, and deserves further thought.
- Or perhaps RLLU PRFs don't exist, and only in this case could we use the known lower bound techniques to go beyond the state of the art. In this case, lower-bound provers may as well begin by breaking our candidate RLLU function generator.

In any case, the natural proofs barrier, once again, brings clarity to the status quo in unconditional complexity lower-bounds.

6.1 LLU PRGs, and a natural proofs barrier against dynamic data-structure lower-bounds

Definition 6.1. Let $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U})$ be a dynamic data-structure frame. A generator over $\mathcal{F}$ is a distribution μ over $\text{Tables}(\mathcal{F})$. We say that μ is (S, ϵ) -secure, if for every Boolean circuit of size $\leq S$,

$$|\Pr_{f \sim \mu}[C(f) = 1] - \Pr_{f \sim \text{Tables}(\mathcal{F})}[C(f) = 1]| < \epsilon.$$

Let $\mathcal{B} = (w, s, t_u, t_q)$ be a dynamic data-structure bound. We say that μ is $\mathcal{B}$ -local-and-locally-updatable ($\mathcal{B}$ -LLU) if every f in the support of μ has a $\mathcal{B}$ -solution.

An $(\mathcal{F}, \mathcal{B}, S, \varepsilon)$ local-and-locally-updatable pseudorandom function (LLU PRF) is a $\mathcal{B}$ -LLU, (S, ε) -secure generator over $\mathcal{F}$. $\lrcorner$

We then conjecture that sufficiently secure LLU PRFs exist, right at the border where we know how to prove lower-bounds. This could be stated using different choices of frames and bounds. We exemplify the conjecture for the Reachability frame.

Conjecture 6.2. For any constants $a > 1, b \geq 1$, there exists a constant c for which the following holds for all large enough n . Let $\mathcal{F} = (\{0, 1\}^{n^2}, [n]^2, [n]^2, \{0, 1\}, \mathcal{U})$ be the reachability frame. Let $\mathcal{B} = (w, s, t_u, t_q)$ be a bound such that $w = O(\log n)$, $s \geq n^{2a}$, and $t_u, t_q \geq c(\log n)^2$, and let $S = 2^{n^b}, \varepsilon = \Omega(1)$. Then, there exists an $(\mathcal{F}, \mathcal{B}, S, \varepsilon)$ LLU PRF.

Although we have repeatedly tried to devise a reasonable LLU PRF, we failed to come up with a convincing candidate. After many attempts, we realized that by relaxing the requirements on solutions, and hence on the generator, we *could* provide a reasonable candidate PRF, while at the same time every known lower-bound proof also applies to such relaxed solutions. The definitions are as follows.

Definition 6.3 (analogous to [Definition 2.10](#)). An (s, w, t_q, t_u) -relaxed-solution to a dynamic data-structure problem $((\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U}), f)$, is a tuple $(E, (T_q)_{q \in \mathcal{Q}})$ where

- $E : \mathcal{D} \rightarrow (\{0, 1\}^w)^s$ is an encoding map.
- For each $u \in \mathcal{U}$ and every $x \in \mathcal{D}$, we have an upper-bound on the Hamming distance between adjacent encodings:

$$\Delta_H(E(x), E(u(x))) \leq t_u.$$

- For each $q \in \mathcal{Q}$, $T_q : (\{0, 1\}^w)^s \rightarrow \mathcal{A}$ is a decision tree over s -long $\{0, 1\}^w$ -words with output set $\mathcal{A}$, having depth at most t_q , and such that

$$\forall x \in \mathcal{D} \quad T_q(E(x)) = f(q, x).$$

$\lrcorner$

Please compare this with [Definition 2.10](#). As you can see, the two definitions are the same, except that we drop the requirement that there exist read-write decision trees T_u to transform $E(x)$ into $E(u, x)$. Instead, our only requirement is that these two strings are close in Hamming distance. This closeness is a consequence of the existence of the trees T_u in [Definition 2.10](#). Hence any solution is also a relaxed solution, but not necessarily the other way around. We then define the analogous notions:

Definition 6.4 (analogous to [Definition 6.1](#)). Let $\mathcal{F} = (\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U})$ be a dynamic data-structure frame, μ a generator over $\mathcal{F}$, and let $\mathcal{B} = (w, s, t_u, t_q)$ be a dynamic data-structure bound. We say that μ is $\mathcal{B}$ -relaxed-local-and-locally-updatable ($\mathcal{B}$ -RLLU) if every f in the support of μ has a $\mathcal{B}$ -relaxed-solution. An $(\mathcal{F}, \mathcal{B}, S, \varepsilon)$ relaxed-local-and-locally-updatable pseudorandom function (RLLU PRF) is a $\mathcal{B}$ -RLLU, (S, ε) -secure generator over $\mathcal{F}$. $\lrcorner$

We now make the following conjecture:

Conjecture 6.5 (analogous to [Conjecture 6.2](#)). *For any constants $a > 1$, $b \geq 1$, there exists a constant c for which the following holds for all large enough n . Let $\mathcal{F} = (\{0,1\}^{n^2}, [n]^2, [n]^2, \{0,1\}, U)$ be the reachability frame. Let $\mathcal{B} = (w, s, t_u, t_q)$ be a bound such that $w = O(\log n)$, $s \geq n^{2a}$, and $t_u, t_q \geq c(\log n)^2$, and let $S = 2^{n^b}$, $\varepsilon = \Omega(1)$. Then, there exists an $(\mathcal{F}, \mathcal{B}, S, \varepsilon)$ RLLU PRF.*

Unlike [Conjecture 6.2](#), we will actually provide a candidate RLLU generator which we believe is secure. The above two conjectures give rise to a natural proofs barrier in the context of dynamic data-structures:

Theorem 6.6. *If [Conjecture 6.2](#) is true, then there does not exist a natural proof showing that any solution to the dynamic reachability problem needs $\max(t_u, t_q) = \omega((\log n)^2)$. Likewise, if [Conjecture 6.5](#) is true, no natural proof can show such a lower-bound against relaxed solutions.*

6.2 A candidate RLLU generator for [Conjecture 6.5](#)

Here we propose a generator for Hamming frames. Hamming frames were defined in [Example 2.7](#). So, take a Hamming frame $\mathcal{H} = (\mathcal{D}, \mathcal{Q}, \mathcal{A}, \mathcal{U})$ with $\mathcal{D} = [p]^n$, with $|\mathcal{Q}|$ queries, $|\mathcal{A}|$ possible answers, and $|\mathcal{U}|$ updates $\mathcal{U} = \{u_{i_1, \phi_1}, u_{i_2, \phi_2}, \dots\}$. We will only consider the case where $|\mathcal{Q}|, |\mathcal{A}|, |\mathcal{U}| = n^{\Theta(1)}$, i.e., polynomial in n . (The reachability frame is an example of this.)

The parameter d . The construction of the PRF candidate depends on one fundamental parameter d . We will consider d within the range $[\omega(\log n), n^{o(1)}]$, i.e., d is some number that grows super-logarithmically in n , but less than n^γ for any constant $\gamma > 0$.

The choice of ℓ and w . There are two additional parameters: ℓ and w . The parameter ℓ is the “number of blocks”. The parameter w is the bitlength of a cell (as before). We let $\ell = w = c \log n$ for $c \in \mathbb{N}$ as large a constant as we need.

Then, except perhaps with probability $o(1)$, the tables sampled by our generator will have a (w, s, t_u, t_q) -solution where $s = O(nd)$, and $t_u, t_q = O(\ell d) \in [\omega((\log n)^2), n^{o(1)}]$. We will conjecture that setting the parameter d to any value $d = \omega(\log n)$, so we have $t_q, t_u = \omega((\log n)^2)$, already makes our PRF secure against quasipolynomial-size circuits (this is [Conjecture 6.7](#) below). However, the generator is still interesting for larger d , such as $d = (\log n)^2$ or $d = n^\gamma$ for an arbitrarily small constant $\gamma > 0$. We will eventually prove that our PRF fools the natural proofs in the next section, except for one (we conjecture it also fools this one, but we could not prove it, yet).

The memory layout. Let us begin by explaining how the s cells of memory are organized. We encode each $x \in \{0,1\}^n$ with an encoding $E = E(x) \in [2^w]^s$, namely, using s cells of $w = \Theta(\log n)$ bits each.

These cells are divided into $1 + \ell$ blocks, where $\ell = \Theta(\log n)$, plus one special block. Let $\kappa > 0$ be a small constant, so that $(1 + \kappa)^\ell = 100n$. The j -th block $E^{(j)} = E^{(j)}(x)$, for

$j = 0, \dots, \ell$ contains $\lceil 2(1 + \kappa)^j d \rceil$ cells, so the size of the blocks grows geometrically with basis $1 + \kappa$, and the largest block has $200nd$ cells. The special block $E' = E'(x)$ contains exactly d cells.

$$\begin{aligned}
E^{(0)}(x) & \quad \boxed{} \boxed{} \boxed{} \cdots \boxed{} \boxed{} \quad \approx 2d \text{ cells} \\
E^{(1)}(x) & \quad \boxed{} \boxed{} \boxed{} \cdots \boxed{} \boxed{} \boxed{} \quad \approx 2(1 + \kappa) d \text{ cells} \\
& \quad \vdots \\
E^{(\ell)}(x) & \quad \boxed{} \boxed{} \boxed{} \cdots \boxed{} \boxed{} \boxed{} \quad \approx 2(1 + \kappa)^\ell d \text{ cells} \\
E'(x) & \quad \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \quad = d \text{ cells}
\end{aligned}$$

How to sample the encodings $E(x)$. The encodings $E(x)$ are chosen as follows. For each block $j \in \{0, \dots, \ell\}$, each $i \in [n]$ and each $z \in [p]$, we uniformly sample a string $D_{i,z}^{(j)} \in (\{0, 1\}^w)^{|E^{(j)}|}$, where $|E^{(j)}| = \lceil 2(1 + \kappa)^j d \rceil$ is the size of the j -th block, as follows:

- each cell of $E^{(j)}$ will be non-zero in $D_{i,z}^{(j)}$ with probability exactly $\frac{2d}{|E^{(j)}|}$, so that, on expectation, we will have $2d$ non-zero w -bit words in $D_{i,z}^{(j)}$.
- Each of the non-zero cells are chosen uniformly at random from $\{0, 1\}^w$.

The encoding $E(x)$ of each non-special block is then $E^{(j)}(x) = \bigoplus_{i=1}^n D_{i,x_i}^{(j)}$. I.e. we bitwise XOR all the strings $D_{i,x_i}^{(j)}$.¹⁵

The special block is chosen as follows. Let $c \geq 1$ be the largest power such that $r^c \leq 2^w$. We then choose the encodings $E'(x) \in (\{0, 1\}^w)^d$ of the special block so that $E'(x)$ is sampled uniformly from $[r^c]^d$ (encoded in binary). It then follows that the number of possibilities for the contents of the special block is r^{cd} , a power of r .

It is easy to check that this choice of encoding obeys the requirements of a relaxed solution, with probability $1 - o(1)$. Indeed, it follows from the Chernoff bound that the probability of seeing less than d , or more than $3d$, non-zero cells in any given $D_{i,z}^{(j)}$ string is at most $\exp(-\Omega(d)) \leq \exp(-\omega(\log n))$, which is much less than the number of j, i, z , so the probability that there exists *any* D_i with more than $3d$ non-zero cells is $o(1)$.

Thus, in case there are no such D_i , the distance between $E(x)$ and $E(u_{i,\phi}(x))$ is $t_u \leq 6d\ell + d = O(\ell d)$, since to update the encoding $E(x)$ to $E(u_{i,\phi}(x))$, we need only to flip

¹⁵More generally, here we could allow ourselves any commutative associative operation on $\{0, 1\}^w$, such as product over $\mathbb{F}_{2^w}$, it does not have to be bitwise-XOR.

the bits in $D_{i,x_i} \oplus D_{i,\phi(x_i)}$ at every block, and change the contents of the special block from $E'(x)$ to $E'(u_{i,\phi}(x))$.

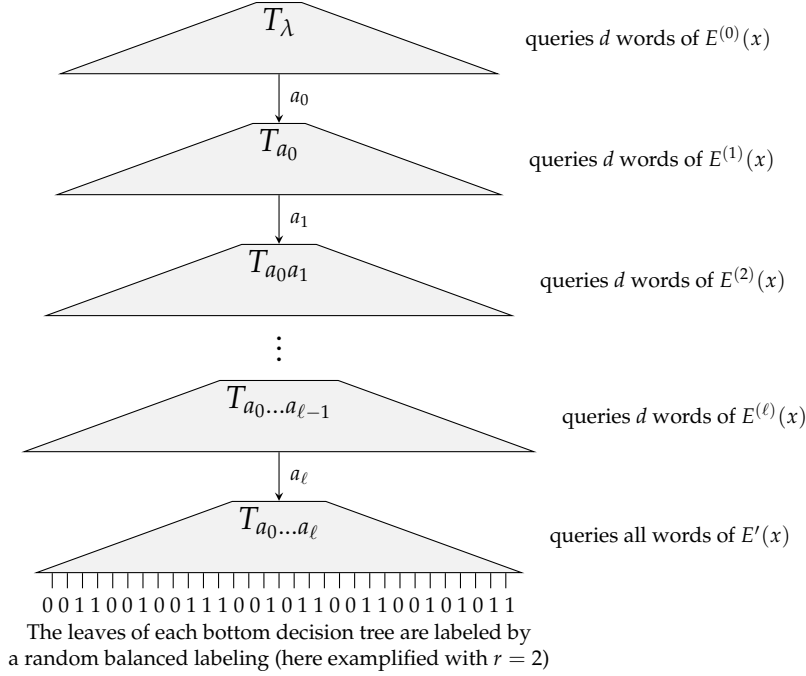
The queries. We now sample decision-trees $T_1, \dots, T_m$ of depth $(\ell + 2)d$, for computing the queries. Each such tree is sampled independently from the same distribution, which we now define.

Below, when we say “sample a random decision-tree of depth d over x ”, we mean that we construct a decision tree of depth d , starting from the root down to the leaves, by choosing, at each node v , a variable which was not queried on the path from the root to v . The leaves of this tree will be unlabelled, unless stated otherwise.

We construct a decision-tree T of depth $(\ell + 2)d$ by first sampling at random several decision-trees of depth d :

- A random decision tree T_λ of depth d over the first block $E^{(0)}$;
- For each $j = 1, \dots, \ell$, and each $a_0 \dots a_{j-1} \in (\{0, 1\}^w)^{jd}$, a random decision tree $T_{a_0 \dots a_{j-1}}$ of depth d over the j -th block $E^{(j)}$;
- Finally, for each $a_0 \dots a_\ell \in (\{0, 1\}^w)^{(1+\ell)d}$, the decision tree $T_{a_0 \dots a_\ell}$ queries every word in the special block E' . Recall that $E' \in [r^c]^d$, so that the total number of possibilities for E' is a power of r . We may then label the leaves of this tree with numbers in $[r]$, choosing uniformly among all possible balanced labelings, i.e., such that each value $a \in [r]$ appears exactly $\frac{r^{cd}}{r}$ times.

Then, the decision tree T is obtained by concatenating all of the above decision trees in the natural way: T_λ appears at the root; on the a_1 leaf of T_λ we append T_{a_1} ; then on the path a_2 of T_{a_1} we append $T_{a_1 a_2}$, etc, as in the following figure:



The table. Now that we have sampled $T_1, \dots, T_m$ ($m = |\mathcal{Q}|$) and $(E(x))_{x \in \{0,1\}^n}$, the table f is defined: $f(q, x) = T_q(E(x))$. By construction, this distribution over tables has a relaxed solution with bound $\mathcal{B} = (w, s, t_u, t_q) = (\Theta(\log n), O(dn), O(\ell d), O(\ell d))$, so we have constructed a $\mathcal{B}$ -RLLU generator. We conjecture that it is indistinguishable from random by small circuits:

Conjecture 6.7. Choose d so that $d = \omega(\log n)$. Then, for any $b \geq 1$ and all large enough n , the RLLU given above is $(2^{n^b}, o(1))$ -secure.

A weaker conjecture would suffice to rule out natural proofs of polynomial lower-bounds (i.e., of the form $\max(t_q, t_u) = n^{\Omega(1)}$):

Conjecture 6.8. For any $\gamma > 0$, choose $d = n^\gamma$. Then, for any $b \geq 1$ and all large enough n , the RLLU given above is $(2^{n^b}, o(1))$ -secure.

6.3 Pseudorandom properties of our PRF

The first observation we can make is that any fixed row $f(q, \cdot)$ is completely uniform, because we choose $E'(x)$ uniformly at random; this choice will result in us picking a random leaf within the last level sub-tree ($T_{a_0 \dots a_k}$), which will thus give us a uniformly-random bit.

But we will make the following additional observation, which will be used below, when we show that our PRF fools every natural property extracted from the lower-bounds which are based on the chronogram method.

Suppose that $d = \omega(\log n)$, and that we have $|\mathcal{Q}| = n^{\Theta(1)}$ queries so that $|\mathcal{Q}| \ll 2^d$. Take any Hamming subcube $D \subseteq [p]^n$ of dimension n' , so there exists some set $F \subseteq [n]$ with $|F| = n'$ “free” coordinates, and some string $y \in [p]^n$ so that $D = \{x \in [p]^n \mid x_i = y_i \forall i \in [n] \setminus F\}$. (The parameter $n' \ll n$ will refer to the number of coordinates updated by the i -th epoch in chronogram-based lower-bounds.)

Take also any set $Q \subseteq \mathcal{Q}$ containing $|Q| \geq n'$ queries. Any such triple (y, F, Q) defines a map $\tau = \tau_{y,F,Q} : D \rightarrow (\{0,1\}^{w(1+\ell)d})^{|Q|}$, so that $\tau(x)_q = a_0 \dots a_\ell \in \{0,1\}^{w(1+\ell)d}$, for $q \in Q$, is the sequence of answers to the probes made by the query decision tree T_q on input $E(x)$, i.e. the answers to all the probes made on the non-special blocks. Let us think of $[p]$ as the “data alphabet”, so we may talk of “data-Hamming-distance” to mean the Hamming distance between two databases $x, x' \in D$ as p -ary strings, hence a number between 0 and n' . In contrast, we think of $\{0,1\}^{w(1+\ell)d}$ as the “query alphabet”, so here we are thinking of the output of τ as a word of length $|Q|$, and we may talk of “query-Hamming-distance” to mean the number of q such that $\tau(x)_q \neq \tau(x')_q$, hence a number between 0 and $|Q|$. (We will later also talk of “encoding-Hamming-distance”, which will be again over a different alphabet $(\{0,1\}^w)$.) Then:

Definition 6.9. Let v be a parameter in the range $[10, n^\gamma]$ (you may take it as $v = 10$, this is the only case we will use) We say that $\tau = \tau_{y,F,Q}$ is “ v -well-separated” if for every $x, x' \in D$ with data-Hamming-distance $|x \oplus x'| \geq \frac{n'}{100}$, the query-Hamming-distance is $|\{q \in Q \mid \tau(x)_q = \tau(x')_q\}| \geq \frac{|Q|}{v}$. $\dashv$

I.e. pairs x and x' which are far in data-Hamming-distance are mapped to pairs $\tau(x)$ and $\tau(x')$ which are far in query-Hamming-distance. In succinct notation, we could write that τ is well separated if, for every $x \in D$, $|\tau^{-1}(\mathcal{N}(\tau(x), \frac{|Q|}{v}))| \leq 2^{\frac{1}{100}n'}$, where $\mathcal{N}(z, k)$ denotes words at query-Hamming-distance $\leq k$ from z .

We then claim:

Theorem 6.10. Let D be a subcube and Q a set of queries, as above. Suppose that $d = \omega(v \frac{\log n}{\log \log n})$. Then, with high probability over the choice of the trees T_q , for every single (y, F, D) , $\tau = \tau_{y,F,D}$ is a “ v -well-separated” map:

$$\Pr[\exists y \exists F \exists D \exists x \in D \mid \tau^{-1}(\mathcal{N}(\tau(x), \frac{|Q|}{v}))| \geq 2^{\frac{1}{100}n'}] \leq 2^{-\Omega(d \frac{|Q|}{v} \log \log n)}.$$

Proof. Let us assume for now that $y \in [p]^n$ is some fixed string, and think of D as equal to $[p]^F = [p]^{n'}$. Fix any $x \in D$, and consider only those $x' \in D$ such that $|x \oplus x'| \geq \frac{n'}{100}$, i.e., their data-Hamming-distance to x is not too small, and let us prove that there exists some block $j \in 0, \dots, \ell$ such that, with high probability, every $x' \notin \mathcal{N}(x, \frac{n'}{100})$ has “encoding-Hamming-distance” $\Delta_H(E^{(j)}(x), E^{(j)}(x')) \geq (1 - \frac{1}{\log n})|E^{(j)}|$, where $\Delta_H(E_1, E_2)$ is the number of cells where E_1 and E_2 differ (this is over a different alphabet than query-Hamming-distance).

The intuition is as follows: for every block $E^{(j)}$, every coordinate i where $x_i \neq x'_i$ will cause $\approx d$ positions in that block to be randomly flipped, between $E^{(j)}(x)$ and $E^{(j)}(x')$,

when we choose the strings $D_{i,z}^{(j)}$ that determine the encoding $E^{(j)}$. If the block length $E^{(j)}$ is *not much larger* than n' , then because x and x' differ in $\geq \frac{n'}{100}$ coordinates, the D_i flips between x and x' will transform every cell in $E^{(j)}$ with reasonable probability. And if the block length $E^{(j)}$ is *sufficiently large* when compared with n' , then this process will never cause $E^{(j)}(x')$ to become equal to $E^{(j)}(x)$. So we can expect that, for the right block length, neither too small nor too large, $E^{(j)}(x)$ and $E^{(j)}(x')$ behave roughly like independently-sampled random strings (in $(\{0,1\}^w)^{|E^{(j)}|}$), and because the alphabet size $2^w \gg |E^{(j)}|$, we can expect $\Delta_H(E^{(j)}(x), E^{(j)}(x'))$ to be close to the maximum $|E^{(j)}|$ for all x' .

Let us prove this formally. Let j be such that $|E^{(j)}| \in [3n', (1+\kappa)3n']$. Let x' be any string with Hamming distance $|x \oplus x'| \geq \frac{n'}{100}$. Then $E^{(j)}(x) \oplus E^{(j)}(x') = \bigoplus_{i: x_i \neq x'_i} D_{i,0}^{(j)} \oplus D_{i,1}^{(j)}$, where, as per our construction the $D_{i,z}^{(j)}$ have each block non-zero with probability $\frac{d}{|E^{(j)}|}$ and each non-zero block is uniformly chosen. A particular cell of $E^{(j)}(x) \oplus E^{(j)}(x')$ is zero iff (1) that cell is never chosen, or (2) it is chosen, but happens to be the same for both encodings. Event (1) happens with probability $(1 - \frac{d}{|E^{(j)}|})^{n'/10} \leq \exp(-\frac{d}{30}) \ll \frac{1}{|E^{(j)}|}$, and event (2) happens with probability $2^{-w} \ll \frac{1}{|E^{(j)}|}$, so the expected number of zero cells in the difference is $\mu \leq 1$. These events happen independently over different cells of $E^{(j)}$, so by a Chernoff bound, the probability that $\Delta_H(E^{(j)}(x), E^{(j)}(x')) \leq (1 - \frac{1}{\ln n'})|E^{(j)}|$ is at most¹⁶ $\exp(-(1 - o(1))|E^{(j)}|) \ll 2^{-2n'}$. We may then union bound over all $\geq \frac{n'}{100}$ -distant pairs (x, x') , to find that every such pair will have $\Delta_H(E^{(j)}(x), E^{(j)}(x')) \geq (1 - \frac{1}{\ln n'})|E^{(j)}|$.

A crucial point to understand about the above argument, is that it does not depend on the coordinates outside of F , where the x 's are equal to y , because $x \oplus x'$ equals zero on those coordinates, no matter what value y takes in them. So the positions $E^{(j)}(x) \oplus E^{(j)}(x')$ where the encodings of x and x' differ (on the j -th block) are the same for every choice of y , and hence we will prove well-separatedness simultaneously for all y , and do not need to union bound over that choice.

Now consider what happens on a particular query $q \in Q$ when we choose the trees $T_{a_1 \dots a_{j-1}}$, i.e. the trees which query the j -th block. These trees will select $\Theta(d)$ cells within $E^{(j)}$, and output the w -bit words in those cells. Since the encodings are $(1 - \frac{1}{\ln n})$ -far (in fractional encoding-Hamming-distance), the probability that $E^{(j)}(x) = E^{(j)}(x')$ within these bits, for a pair x, x' with $|x \oplus x'| \geq \frac{n}{100}$, is at most $(\frac{1}{\ln n})^{\Omega(d)}$. Hence, the probability

¹⁶Let $N = |E^{(j)}| = \Theta(n')$ and let $X = \sum X_i$ denote the number of zero cells, and suppose $\Pr[X_i] = \frac{1}{N}$ (in our case the probability of being zero is even smaller). We then have:

$$\begin{aligned} \text{(Chernoff)} \quad \Pr[X \geq (1+\delta)\mu] &\leq \left(\frac{e^\delta}{(1+\delta)^{1+\delta}} \right)^\mu \\ \xrightarrow{\mu=1, \delta=\frac{N}{\ln n'}-1} \quad \Pr \left[X \geq \frac{N}{\ln n'} \right] &\leq \exp \left(-\frac{N}{\ln n'} \ln \frac{N}{\ln n'} + \frac{N}{\ln n'} - 1 \right) \leq \exp(-N(1 - o(1))). \end{aligned}$$

that this happens for some $\frac{|Q|}{v}$ queries $q \in Q$ is at most

$$\binom{|Q|}{\frac{|Q|}{v}} \left(\frac{1}{\ln n} \right)^{\Omega(d \frac{|Q|}{v})} \leq 2^{|Q|} \cdot 2^{-d \frac{\log \log n}{v} |Q|} = 2^{-\Omega(d \frac{\log \log n}{v} |Q|)}.$$

We may now union bound over the choice of Q , since there are at most $\binom{|Q|}{|Q|} = n^{O(|Q|)}$ such choices, which is $\ll 2^{-\Omega(d \frac{\log \log n}{v} |Q|)}$ if $d = \omega(v \frac{\log n}{\log \log n})$. Outside of this small probability, every single $\tau_{y,F,Q}$ is well-separated. $\square$

7 Thesis: All known dynamic data structure lower bounds are natural

In this section we cover all known dynamic data-structure lower-bound proof techniques and verify, one by one, that they are all natural, and hold against relaxed solutions. We will cover the chronogram method (Section 7.1), the information transfer technique (Section 7.2), lower-bounds against non-adaptive (Section 7.3) and semi-adaptive (Section 7.4) solutions, lower-bounds that mix the chronogram method and cell-sampling techniques (Section 7.5), and finally lower-bounds that mix the chronogram method with weak one-way simulation (Section 7.6).

We will show that our PRF fools the natural property in all these proofs techniques, except for the last. We believe that this is also the case for the remaining technique, but have not been able to show it, yet.

7.1 The chronogram method

The first method to prove lower-bounds against dynamic data structures was the chronogram method of Fredman and Saks [FS89]. It was originally used to prove a lower-bound for prefix-sum, and since then it has been applied to many others (e.g. [AHR98, BAG01, PT11, RR18, Nav25], see also Sections 7.5 and 7.6). For concreteness, let us illustrate the method for prefix-sum-mod-2. In this problem, our goal is to store a sequence $x = x_1 \dots x_n \in \{0, 1\}^n$ under bit-flip updates (so the frame is the Boolean cube), and we have n queries so that $f(i, x) = \sum_{j \leq i} x_j \pmod 2$.

Suppose we start with $E(x)$ for some arbitrary $x \in \{0, 1\}^n$ and perform a sequence of random updates, organized into k epochs $U_k \dots U_1$ (which we number so that U_k happens before U_{k-1} , etc), as follows. Set $\beta = 10wt_u$ and $k = \frac{1}{10} \log_\beta n$. In the first (oldest) epoch U_k we will do $\beta^k = n^{1/10}$ random updates, in evenly-spaced positions $j_1, \dots, j_{\beta^k}$ (so $j_z = \lfloor z \frac{n}{\beta^k} \rfloor$); by “random” updates we mean that, for each of these positions $j_1, \dots, j_{\beta^k}$, we toss a balanced coin, which determines whether to flip the bit of x at this position. Similarly, in

the i -th epoch U_i , for $i = k - 1, \dots, 1$ we do $\beta^i = \frac{n^{1/10}}{\beta^{k-i}}$ updates in approximately-evenly-spaced positions, so that the updated positions are actually disjoint between epochs.¹⁷ Let $U_i \in \{0, 1\}^{\beta^i}$ be the sequence of coin tosses of the i -th epoch, and $U = U_k \dots U_1$.

Let also $x^{(i)}$ be the random variable obtained from x after performing the updates $U_k \dots U_i$. After all the updates have been performed, we choose a random position $q \in [n]$ and perform the q -th query. Let $S_{k+1}, S_k, \dots, S_1 \subseteq [s]$ be random variables, so that $\ell \in S_i$ if and only if the ℓ -th cell in the encoding was last written by some update in epoch i (S_{k+1} contains those positions which were never written during any of the updates). Let $P(q) \subseteq [s]$ be the random variable containing those positions in the encoding which were probed during the query q . We now claim that, for every $i = k, \dots, 1$, $\mathbb{E}_{U,q}[|S_i \cap P(q)|] = \Omega(1)$. It follows by linearity of expectation that $\mathbb{E}_{U,q}[\text{\# queries}] = \Omega(k)$, which is the desired lower bound.

To prove the claim, suppose by contradiction that, for some i , $\mathbb{E}_{U,q}[|S_i \cap P(q)|] = o(1)$. Fix any choice $U^{-i} = U_k \dots U_{i+1}, U_{i-1}, \dots, U_1$, conditioned on which $\mathbb{E}_{U_i,q}[|S_i \cap P(q)|] = o(1)$. Since $|S_i \cap P(q)|$ is a natural number, it follows that, for a $1 - o(1)$ fraction of all possible i -th epoch updates U_i , for a $1 - o(1)$ fraction of all queries $q \in [n]$, $|S_i \cap P(q)| = 0$, i.e. $f(q, x^{(1)})$ can be answered without probing any cells in S_i . We now claim that every such U_i (among the $1 - o(1)$ fraction) can be encoded with $o(\beta^i)$ bits, which is a contradiction. Indeed, since we have fixed $x, U_k, \dots, U_{i+1}$, we know the state $E(x^{(i+1)})$ of the data structure before the start of the i -th epoch. We can also encode the position and contents of every cell in $S_{i-1}, \dots, S_1$, since this is $\sum_{j=1}^{i-1} \beta^j t_u w \leq \frac{1}{5} \beta^i$ bits (and the same will hold for **relaxed solutions**, since we do not really use the fact that the updated positions come from a decision tree, only that there are few such positions). So, if we take the string $E(x^{(i+1)})$ and update the cells in $S_{i-1}, \dots, S_1$ with the given contents, we obtain some string E' which equals $E(x^{(1)})$ on every cell except the cells in S_i . If we then run every query q on E' , we will obtain query answers $a_1 \dots a_n$, a $1 - o(1)$ fraction of which are correct, since the corresponding decision trees don't query any cell in S_i .

Thus far, the entire argument works for any problem using the same frame. Now comes the part of the argument which is specific to the table of the prefix-sum problem. It can be shown that knowing a string $a_1 \dots a_n$ which is $o(1)$ -close (in fractional Hamming distance) to the string $f(1, x^{(1)}), \dots, f(n, x^{(1)})$ of true answers, will allow us to learn of a small set A of size, say, $2^{\frac{1}{10} \beta^i}$, which contains U_i . Hence we append to our encoding the position of U_i in the lexicographic ordering of this small set.

We now claim that this final requirement, which is the only place in the proof that depends on the specific choice of table, directly gives us a natural property. First let us define the following family of maps. The maps in the family are indexed by $z = (i, U^{-i}) = (i, U_k, \dots, U_{i+1}, U_{i-1}, \dots, U_1)$, where $i \in \{k, \dots, 1\}$ is an epoch, and U^{-i} is a sequence of updates for all the remaining epochs. For each such z , the map ψ_z maps $U_i \in \{0, 1\}^{\beta^i}$ to

¹⁷This is for technical reasons; n positions is much more than $k\beta^k$, so there are more than enough positions to make the updates positions disjoint between epochs.

the sequence of query answers $\psi_z(U_i) = f(1, x^{(1)}) \dots f(n, x^{(1)})$, where $x^{(1)}$ is the database obtained from $x = 0^n$ by applying the updates $U_k \dots U_{i+1} U_i U_{i-1} \dots U_1$. Let $\mathcal{N}(x, \delta)$ be the set of binary strings of length $|x|$ at Hamming distance $\leq \delta|x|$ from x . We say ψ_z is *good* if for every U_i , $|\psi_z^{-1}(\mathcal{N}(\psi_z(U_i), \frac{1}{100}))| \leq 2^{\frac{1}{10}\beta^i}$. I.e., the number of U'_i such that $\psi_z(U'_i)$ is $\frac{1}{100}$ -close to $\psi_z(U_i)$, is reasonably small. The natural property of f , then, is that ψ_z is good for every z . It is clear that in time $2^{O(n)}$ we can go over every single z and every single U_i , and do the necessary accounting, so the property is constructive. This property will allow us to carry out the proof (it is useful), and it holds for prefix-sum-mod-2. We now prove that it holds of a random function.

The proof is by a union bound. Take any two distinct $U_i, U'_i \in \{0, 1\}^{\beta^i}$. Because the updates $U_k \dots U_1$ touch different bits of x , it follows that the string $x^{(1)}$ obtained from 0^n by doing the updates $U_k \dots U_1$ is different for U_i and U'_i . And hence, each image $\psi_z(U_i)$ is a different column in the table f , and so all the query answers $\psi_z(U_i)$ are chosen independently at random for different choices of U_i . Hence, the likelihood (over the choice of f) that $\psi_z(U_i)$ and $\psi_z(U'_i)$ are $\frac{1}{100}$ -close is at most $\frac{\binom{n}{n/100}}{2^n} \leq 2^{-\frac{3}{4}n}$, a really small probability. Then, by a Chernoff bound, for any U_i , the likelihood that there exist many U'_i , say $2^{\frac{1}{10}\beta^i}$ many, having $\psi_z(U'_i)$ close to $\psi_z(U_i)$, this is now a negligible probability ($2^{-n^{\omega(1)}}$). Hence by a union bound, with high probability, this will never happen for any z, U_i , i.e., every ψ_z is good.

Our candidate PRF. To finish, let us prove that our RLLU PRF satisfies the above natural property, already when $d = \omega(\log n)$, by applying [Theorem 6.10](#) with $v = 10$. Begin by noticing that each choice $z = (i, U^{-i})$ defines a subcube $D \subseteq \mathcal{D}$ of the data set: the coordinates F of U_i are free, and the remaining coordinates are fixed to some value y according to U^{-i} . We let $Q = [n]$ contain every query. Since each update U_i has size $n_i \ll n$, this subcube has dimension $\ll n$. As in [Section 6.3](#), let $\tau = \tau_{y,F,Q} : D \rightarrow (\{0, 1\}^{(1+\ell)d})^{|Q|}$, so that $\tau(x)_q = a_0 \dots a_\ell \in \{0, 1\}^{w(1+\ell)d}$, for $q \in Q$, is the sequence answers to the probes made by T_q on input $E(x)$, i.e. the answers to all the probes made on the non-special blocks. [Theorem 6.10](#), applied with $v = 10$, states that, with high probability over the choice of the encodings E , and the query decision-trees T_q , every $x, x' \in D$ with data-Hamming-distance $\geq \frac{n_i}{100}$ between them, will have at most $\frac{|Q|}{10} = \frac{n}{10}$ queries $q \in Q$ such that $\tau(x)_q = \tau(x')_q$. So, for the remaining $\frac{9}{10}n$ queries in Q , the two inputs x and x' will choose different subtrees in the decision-tree T_q . Notice that this property only depends on the non-special blocks, and the choice of the trees T_q before the last level (which probes the special block), so let us consider what happens now, as we choose the encodings E' of the special block, and the last level decision-trees which probe it. For each the $\frac{9}{10}n$ queries q such that $\tau(x)_q \neq \tau(x')_q$, the outputs $f(q, x)$ and $f(q, x')$ are two independent random bits. Hence the probability that $\psi_z(x)$ and $\psi_z(x')$ have Hamming distance $\leq \frac{n}{100}$ (which is much smaller than $\frac{1}{2} \frac{9n}{10}$), is $\exp(-\Omega(n))$, which is much less than the inverse of: the number of pairs x, x' in D times the number of choices for z , because $|x| + |x'| + |z| = n_i + \sum_j n_j \ll n$. It then follows $\psi_z(x)$ and $\psi_z(x')$

are $\frac{n}{100}$ -far whenever x and x' are at data-Hamming-distance greater than $\frac{n}{100}$, and hence $|\psi_z^{-1}(\mathcal{N}(\psi_z(x), \frac{n}{100}))| \leq \binom{n_i}{\frac{n_i}{10}} \leq 2^{H_2(\frac{1}{100}n_i)} \leq 2^{\frac{n_i}{10}}$, as intended.

7.2 Patrascu and Demaine's information transfer technique

Patrascu and Demaine's information transfer technique [PD06] (see also [CJS13, CJS16, LN18, LLYZ23]) was first used to prove the first $\Omega(\log n)$ lower-bound against dynamic data structures. More specifically, they proved a lower-bound against the *prefix-group-product problem*, which is the dynamic problem of storing a sequence $x_1, \dots, x_n$ of elements from a finite group G , under updates $u_{i,y}$ that change x_i to y (so the frame is $[|G|]^n$ with coordinate-wise updates), and queries $q_{i,z}(x_1, \dots, x_n) = 1$ if the product $x_1 \cdots x_i = z$, and 0 otherwise¹⁸. They show that $\max(t_u, t_q) = \Omega(\log n \cdot \frac{\log |G|}{w})$. Applied when $G = \mathbb{Z}_n$, $w = \Theta(\log n)$, this gives us a lower-bound of $\Omega(\log n)$ for the prefix-sum problem. When $G = \mathbb{S}_n$, $w = \Theta(\log n)$, this gives us a lower-bound of $\Omega(\log n)$ for the dynamic connectivity problem¹⁹.

In order to make sense of the natural-proofs barrier in this case, we need an *ad-hoc* change to the notion of natural property. Notice that the table of the function above has the following non-random-looking feature: for every $x = x_1 \dots x_n \in G^n$, and for every $i \in [n]$, there exists exactly one $z \in G$ such that $q_{i,z}(x) = 1$, namely $z = x_1 \cdots x_i$. Then, rather than sampling a random table by choosing each query answer $q_{i,z}(x)$ is a random bit, we will instead choose a random table with this “unique z ” property, i.e. for each x, i we choose a single z and set $q_{i,z}(x) = 1$ and $q_{i,z'}(x) = 0$ for every $z' \neq z$. We will later define a natural property where the *largeness* is with respect to a table f uniformly sampled among all tables with the “unique z ” property. We will argue later how to change our PRF to ensure this property holds.

The lower-bound proceeds as follows. Suppose n is a power of 2. Consider a sequence $u_{i_1, y_1} q_{i_1, z_1} \dots u_{i_n, y_n} q_{i_n, z_n}$ of update-query pairs, where y_j are chosen at random and each z_j is the product of $x_1 \dots x_{i_j}$ at the time when query q_{i_j, z_j} is made (i.e. z_j is the only value for which q_{i_j, z_j} will be answered 1). At time-step j we update position i_j , so that $i_1 \dots i_n$ appears in reverse-lexicographic order of the binary representation of the positions. So for example for $n = 8$ the updated positions would be $(i_1, \dots, i_8) = (000_2, 100_2, 010_2, 110_2, 001_2, 101_2, 011_2, 111_2) = (0, 4, 2, 6, 1, 5, 3, 7)$. Now think of these sequence of positions as the leaves of a full binary tree, which we call the *time tree*. The crucial property to be used is that, for any node v in this binary tree, the sequence of positions appearing in the right sub-tree of v interlaces the sequence of positions appearing in the left sub-tree of v . E.g. for $n = 8$, and v the root, $(1, 5, 3, 7)$ interlaces $(0, 4, 2, 6)$, or if v is the left child of the root, then $(2, 6)$ interlaces $(0, 4)$.

¹⁸Such queries are called VERIFY-SUM queries in [PD06], as a reference to the prefix-sum problem, but the lower-bound also holds for non-commutative G , where VERIFY-PRODUCT would be more adequate.

¹⁹Think of the sequence $x_1 \dots x_n$ as a graph with n layers, with the edges in each layer given by the permutation; each coordinate-wise update is implemented with n edge-updates and each verify-sum query is implemented with n connectivity queries, respectively.

The contradiction with the existence of an efficient solution is via an encoding argument. Let v be any node in the time tree and let $2L$ be the number of children of v . Suppose, for contradiction, that the updates and queries made on the right-subtree of v make a $o(\frac{L \log |G|}{w})$ average number of probes to positions which were last written by the left sub-tree of v . Demaine and Patrascu show that in this case there exists an encoding, of average length $o(L \log |G|)$, of the *right-side query answers*, meaning: from this encoding we can extract, for each position j in the right-subtree of v , the value of $z_j = x_1 \dots x_{i_j}$ at the time when the query q_{i_j, z_j} is made. This encoding argument uses only the specific choice of frame, and the “unique z ” property of f , it does not use any other property of the table f . The encoding will also work for **relaxed solutions**, since all its bounds depend only on the fact that an update changes few cells, and not on the fact that these cells were updated by a decision tree²⁰.

Now it turns out that, when the table f is the prefix-group-product problem, because of the interlacing property, we can recover all the updates y_j for all positions j in the left sub-tree of v if we know (1) the right-side query answers, and (2) the *initial* x , meaning, the values of $x_1 \dots x_n$ before the updates in the left sub-tree of v , and (3) *the right-side y_j 's*, meaning, the sequence of updates y_j for all j in the right sub-tree of v . So we have encoded the left-side y_j 's from $o(L \log |G|)$ bits (this gives us 1) plus some information which is independent of the left-side y_j 's (namely 2 and 3), which is a contradiction. So for every node v , the right-side queries must make $\Omega(\frac{L \log |G|}{w})$ probes which were last written by the left-side updates. For any query (which is a leaf in the time tree), the probes accounted for in this way, in the nodes v between the root and the query, are distinct, since, when read, they were last written at different times. Hence there are $\Omega(\log n \cdot \frac{n \log |G|}{w})$ probes in total, and we have done n updates and n queries, so we must have $\max(t_u, t_q) = \Omega(\log n \cdot \frac{\log |G|}{w})$. This is the promised lower-bound.

This suggests the following candidate for a natural property. For the above lower-bound to work, we need only consider “top” nodes v in the time-tree at the top $\frac{1}{2} \log n$ levels, so that $L \geq \sqrt{n}$. The property will state, then, that each top node v in the time tree, each choice $x = x_1, \dots, x_n$ of the initial x , and each choice $y^{(r)}$ of the right-side y_j 's, define the function $\phi_{v,x,y^{(r)}} : G^L \rightarrow G^L$ so that $\phi_{v,x,y^{(r)}}(y^{(\ell)}) = z^{(r)}$ equals the the sequence of z_j , for each position j updated and queried in the right sub-tree of v , such that $q_{i_j, z_j}(x') = 1$, where x' equal to x after doing the updates $y^{(\ell)}$ and the updates $y^{(r)}$ up to position j . I.e., $\phi_{v,x,y^{(r)}}(y^{(\ell)})$ contains the answers to the queries made by the right-subtree, in the above update/query sequence (this is the information which I can recover from the small encoding in the argument). As we have mentioned, the prefix-group-product function has the property that every single $\phi_{v,x,y^{(r)}}$ is a 1-1 function — indeed, it is a permutation. We could take this as our hardness-implying property, and the lower-bound will follow, but this 1-1 property will not hold for a randomly chosen f . So, instead, our natural

²⁰Cf. the proof of Lemma 6.1.; the argument *does use* the fact that the *queries* are obtained via decision trees, it does not obviously apply, e.g., to certificates.

property is that, for every v of depth $\leq \frac{1}{2} \log n$, every x and every $y^{(r)}$, $\phi_{v,x,y^{(r)}}$ is a few-to-1 function, namely, each pre-image $\phi_{v,x,y^{(r)}}^{-1}(z^{(r)})$ has size $\leq |G|^{\frac{99}{100}L}$. Since each z in the image is chosen uniformly from G , the probability that any given pre-image has more than k elements is $\leq |G|^{-k}$, which in our case is $|G|^{-|G|^{\frac{99}{100}L}}$, much less than the inverse of the number of $v, x, y^{(r)}, y^{(\ell)}$. Hence by a union bound, a random f will have the natural property. The property is also easily seen to be constructive.

Our candidate PRF. We first explain how to adapt our RLLU PRF to output tables with the “unique z ” property. We consider our PRF over the frame $\mathcal{H}_{n,n,|G|,|G|}$, so the data is G^n , we have n query trees $T_1, \dots, T_n$, and the output is an element of G . From each of our chosen T_i , we then derive the different T_{i,z_i} to be exactly the same decision tree, but have a binary output which indicates whether the value which T_i would have output is z_i .

We now wish to show that this PRF has the hardness implying property above, i.e., each function $\phi_{v,x,y^{(r)}}$ should have $|\phi_{v,x,y^{(r)}}^{-1}(z^{(r)})| \leq |G|^{\frac{L}{10}}$. For this, let us assume that $|G| = \omega(L^3)$, and recall that we only consider nodes v with $L \geq \sqrt{n}$.

Again, each choice of $v, x, y^{(r)}$ defines a subcube (the free variables are the $y^{(\ell)}$, let us denote them by w), and a set Q of $|Q| = L$ queries (namely, we have one query for each leaf in the left subtree of v). Consider the event, for some fixed w , that $|\phi^{-1}(\phi(w))| > |G|^{\frac{99}{100}L}$. Then, by the sphere-packing bound (since each $\frac{L}{100}$ -data-Hamming-ball has size at most $G^{\frac{2}{100}L}$ points), there are at least $G^{\frac{97}{100}L}$ w' in $\phi^{-1}(\phi(w))$ which are $\geq \frac{L}{100}$ -far from each other (in data-Hamming-distance). Let W be such a large set containing those many points distant from each other. From [Theorem 6.10](#), applied with $v = 10$, we conclude that every $w', w'' \in W$ has $\tau(w) \neq \tau(w')$ (in fact they are $\frac{L}{10}$ -far-apart in query-Hamming-distance, but we won't use that here). Let V be the number of distinct leaves of the decision-trees $(T_q)_{q \in Q}$, which are touched by the queries in W . Then $G^{\frac{97}{100}L} \leq |W| \leq V^L$, and so $V \geq G^{\frac{97}{100}} = \omega(L^2) = \omega(n)$, so in particular we have $\omega(n)$ leaves which are not touched by the query trees on input w .

For each such distinct leaf, touched by a decision-tree T_q on some input $w' \in W$, the output $f(q, w')$ is an independently-chosen elements of G . The condition that $f(q, w') = f(q, w)$ forces these outputs to be some specified value, and the probability of this event is hence $|G|^{-\omega(n)}$. This probability is low enough for us to union bound over the choice of $v, x, y^{(r)}$ and w . And so the bad event never happens.

7.3 Non-adaptive data-structures: Brody and Larsen's Lower-bounds

We cannot prove static data structure lower-bounds beyond the $\log |Q|$ barrier, not even against non-adaptive data structures. The case is different in the dynamic setting. Brody and Larsen [\[BL15\]](#) have shown, circa 2015, polynomial lower-bounds against dynamic data structures for reachability, against solutions where either (1) the queries are non-adaptive, or (2) the updates are non-adaptive and memoryless. This result was

later expanded upon in a couple of papers, also dealing with the non-adaptive case [BBK18, LPP⁺24].

The lower-bound for non-adaptive queries. Brody and Larsen first prove a lower-bound on the dynamic indexing problem, which they then reduce to dynamic reachability. In the dynamic indexing problem, the goal is to store a set of n binary strings $\bar{S} = (S_1, \dots, S_n) \in (\{0, 1\}^n)^n$, together with an index $j \in [n]$. The sets cannot be changed but the index j can be, so we have n updates possible, for changing j into each of the possible n values. There are also n queries, and the table is defined by $f(i, (\bar{S}, j)) = S_{i,j}$, i.e. we output the j -th bit of S_i .

The lower-bound is as follows: we will encode $\bar{S}$ in its entirety by the contents of some of the cells of $E = E(\bar{S}, 1)$, namely: (a) the cells probed by every possible query, when run on E , and (b) the cells probed during every possible update of j , when run on E (all updates are run on the same E , i.e. not in sequence). This totals $n(t_q + t_u)w$ bits. Notice that if we know the values of these cells of E , the addresses of the probed cells can be deduced by running the query and update decision trees, answering probes with the known contents. Furthermore, we can uniquely decode $\bar{S}$: to obtain $S_{i,j}$, we first run update j to the desired value (we know all the cells involved), thus obtaining $E(\bar{S}, j)$, and then run the i -th query (we also know all the cells involved because the query is non-adaptive), to obtain $S_{i,j}$. Hence $n(t_q + t_u)w \geq n^2$, which proves the lower-bound $t_q + t_u \geq \frac{n}{w}$.

The claim that the addresses of the probed cells can be derived from the contents is *not* true of **relaxed solutions**, however. The lower-bound argument in this case must append also the locations (and not just the contents) of the updated cells; the length of the encoding is then $nt_qw + nt_u(w + \log s)$, which is in the same order of magnitude under the usual assumption that $w = \Theta(\log s)$ (i.e. words are big enough to address the memory). So a small change in the argument *will* make it valid against relaxed solutions.

As usual, the proof appears to use specific properties of the indexing gadget, hence on a first glance does not appear to be natural. What have we used about indexing, however? We used that $\bar{S}$ can be uniquely determined by knowing $B(\bar{S}) = (f(i, (\bar{S}, j)))_{i,j \in [n]} \in \{0, 1\}^{n^2}$. If f is a randomly chosen table, then the n^2 bits of $B(\bar{S})$ are chosen uniformly and independently at random for each $\bar{S}$. This random process is the same as tossing 2^{n^2} balls into 2^{n^2} bins ($B(\bar{S})$ is the bin of $\bar{S}$), and hence with probability $1 - o(1)$ the maximum load of any bin is $\ll n^2$. Hence we can append to the values of the cells $2 \log n$ bits to indicate which $\bar{S}$ we are encoding, among the fewer than n^2 choice of $\bar{S}$ having the same $B(\bar{S})$. We get a marginally weaker lower-bound of $n(t_q + t_u)w + 2 \log n \geq n^2$, but now we have a natural proof: the natural property is that for every choice of $\bar{S}$, there are fewer than n^2 values of $\bar{S}'$ with $B(\bar{S}) = B(\bar{S}')$. This is easily checkable by inspecting f , in time polynomial in the size of f .

The lower-bound for non-adaptive, memoryless updates. Again the lower-bound for reachability is by reduction, this time from the dynamic set disjointness problem. In this problem, the task is to encode a pair of sets $(S, T) \in (\{0, 1\}^n)^2$, under bit-flip updates, and

there is a single query q asking whether $S \cap T = \emptyset$. A non-adaptive read-write decision-tree is called *memoryless* if, whenever it changes the contents of a cell, the new value is a function of the current value, i.e. it is independent of the contents of any other probed cell.

Again we have an encoding argument. We will encode any set $S \in \{0,1\}^n$ as follows. Recall that $E(S, S') \in (\{0,1\}^w)^s$ denotes the encoding of the pair (S, S') . Let $I(S) \subseteq [s]$ be the addresses of the cells which were probed during by the query decision tree running on $E(S, [n] \setminus S)$ (this can be represented by the path traversed by the query tree, hence $\leq t_q$ bits). Let $C(S) = E(S, \emptyset)|_{I(S)} E_1(S, [n] \setminus S)|_{I(S)}$ be the value of those cells in $E(S, \emptyset)$ and $E(S, [n] \setminus S)$. We now claim that S can be decoded by knowing $I(S)$ and $C(S)$, from which the lower-bound follows: $|I(S)| + |C(S)| = t_q(1 + 2w) \geq n$. The reason is because, for any set S'' , knowing $E(S, \emptyset)|_{I(S)}$, we can compute $E(S, S'')|_{I(S)}$, by running the updates required to transform the second set from $\emptyset$ to S'' : since the updates are memoryless, we do not need to know any cell outside of $I(S)$ in order to update the cells in $I(S)$. We then look at $E(S, S'')|_{I(S)}$ for all S'' , and discard all S'' for which this is different than $E(S, [n] \setminus S)$. The remaining S'' must all be disjoint from S , since the output of the query is the same for all such S'' and $[n] \setminus S$. It follows that $[n] \setminus S$ is the largest remaining S'' , so we have learned S .

Again it is not obvious how to naturalize this argument, as it seems to depend on properties very specific to disjointness. However, consider the map $(S, S') \mapsto \psi(S, S') = I(S)E(S, \emptyset)|_{I(S)}E(S, S')|_{I(S)}$ we just looked at. Suppose that a certain function $f(S, S')$ has a solution with non-adaptive memoryless updates. The memoryless property actually implies that the inverse images $\psi^{-1}(z)$ of this map form a rectangle partition of the communication matrix of f ! Meaning, if $\psi(S_1, S'_1) = \psi(S_2, S'_2) = z$, then actually $\psi(S_1, S'_2) = \psi(S_2, S'_1) = z$ also. Indeed, since $I = I(S_1) = I(S_2)$, $E(S_1, \emptyset)|_I = E(S_2, \emptyset)|_I$, and $E(S_1, S'_1)|_I = E(S_2, S'_2)|_I$, the updates from $E(S_1, \emptyset)$ to $E(S_1, S'_1)$ and from $E(S_2, \emptyset)$ to $E(S_2, S'_2)$ will see identical cell contents in the I -cells, and change those contents in the same way. Thus, since the updates are non-adaptive they will probe the same cells, which have the same initial content, and since they are memoryless, they will make the same changes (within I) when updating $E(S_1, \emptyset)$ to $E(S_1, S'_2)$ and when updating $E(S_2, \emptyset)$ into $E(S_2, S'_1)$.

It follows that f has a large 1-monochromatic rectangle, i.e. containing a fraction of $2^{-t_q(1+2w)}$ of all the 1s of f . And, as we have seen in [Section 5.1](#), the property of not having a nearly-1-monochromatic rectangle, true of both disjointness and a random function, can be tested efficiently by Barman's algorithm ([Theorem 5.5](#)). And we have hence naturalized Brody and Larsen's proof.

Our PRF satisfies these properties because for any frame with a single query, the table will be completely random. (In truth it is not fair to call it a PRF in this case, since there is more randomness being used to generate the table, than the size of the table itself. The principle remains that any distinguisher which looks only at a single row will necessarily fail against our PRF.)

7.4 Semi-adaptive data-structures: Ko and Weinstein’s Lower-bounds

Given Brody and Larsen’s result, one way of looking at the open problem of proving n^ε lower-bounds for dynamic data-structures is to see it as if the only issue missing is for us to understand adaptivity. In this way, Ko and Weinstein [KW20] considered the case of *semi-adaptive* data structures. Some technical considerations are necessary to properly define what that means: the crucial observation is that we may assume, without loss of generality, that the data structure is organized in two parts $(\{0,1\}^w)^{s_1} \times (\{0,1\}^w)^{s_2}$ as follows: some part of the data structure contains the encoding $E(x) \in (\{0,1\}^w)^{s_1}$ of the initial data, which is *read-only*, so the updates are not allowed to write over this part, but can only write in a disjoint set of specially-designated cells $U(E(x)) \in (\{0,1\}^w)^{s_2}$. For this to be at all possible we must drop the history-independence requirement. Then, it is not hard to see that this disjoint-data/updates assumption can be made without loss of generality, up to a logarithmic-factor loss in t_q and t_u , which we don’t care about if we are aiming for polynomial lower-bounds. Under this assumption, we may now impose the following *semi-adaptive* constraint: every decision tree T_q associated with any query q must first read the $E(x)$ cells, and only after read the $U(E(x))$ cells. Meaning, on any root-to-leaf path of T_q , there are some probes to the first s_1 cells, followed by some probes to the last s_2 cells. So it is as if we had two adaptive stages, but no cross-adaptivity, meaning once we learn some of the updates we cannot go back and read more cells in the $E(x)$ part.

Ko and Weinstein [KW20] prove a lower-bound of $\max(t_q, t_u) = \Omega(\sqrt{n})$ for the dynamic set disjointness problem mentioned in the previous sub-section. In a follow-up paper, Dvořák and Loff [DL20] have shown that the same technique actually yields a lower-bound on any table (such as disjointness, or a random table) which has a large corruption bound²¹. As mentioned at the end of Section 5.1, Barman’s algorithm can be used to efficiently estimate this bound, and hence Ko and Weinstein’s technique also naturalizes.

The technique makes no assumption that the cells being updated are obtained by a decision tree, and is applicable whenever updates change few cells (cf. Section 4.5 of [KW20], on the arXiv version 1910.13543v1). Hence it applies to relaxed solutions.

Again, **our PRF satisfies the property** because the lower-bound considers a frame with a single query.

7.5 Chronogram + Cell Sampling

Building on the *Chronogram method* for proving lower-bounds against dynamic data structures, Larsen proved an $\tilde{\Omega}(\log^2 n)$ lower-bound²² for 2D Range Counting [Lar12a] and Polynomial evaluation [Lar12b]. This is the highest known lower bound for Dynamic Data Structures. However, there’s one caveat, the set of possible answers $\mathcal{A}$ is not Boolean,

²¹Under a product distribution, such as the uniform distribution.

²² n is the dimension of the problem

but rather has a size of $|\mathcal{A}| = n^{\Omega(1)}$. In an earlier work, Patrascu [Pät07] had shown an $\tilde{\Omega}(\log^2 n)$ lower bound, but for a problem where the set of possible answers $\mathcal{A}$ is super-polynomial in n .

Let us give the example of polynomial evaluation. Polynomial evaluation is the problem of storing a polynomial $p_x(y) = \prod_{i=1}^n (y - x_i)$ over a finite field $\mathbb{F}_p$, so here p denotes a prime power. The data $x \in \mathbb{F}_p^n$ is the roots of the polynomial p_x , and the updates are allowed to change the roots. So, in the corresponding frame, we have p^n databases, and the updates allow us to change any single coordinate to any value. There are $|\mathcal{Q}| = p$ queries and $|\mathcal{A}| = p$ possible answers. For concreteness set $p = n^2$, but the proof works for any $p = n^{\Omega(1)}$. Let $\mathcal{P}$ denote an arbitrary dynamic data-structure problem over this frame, with table f .

Suppose there exists a solution for $\mathcal{P}$ with storage size s , word size w , query time t_q , and update time t_u . We assume $s = n^{O(1)}$ and $w = \log s = O(\log n)$. Larsen's proof in [Lar12a] shows a lower-bound of $t_q = \Omega\left(\frac{\log^2 n}{(\log t_u w)(\log(t_u w / \log p))}\right)$.

Recall that in the *Chronogram method*, we begin by considering the encoding $E(x)$ of an arbitrary element $x \in \mathbb{F}_p^n$ ²³. We then perform a sequence of updates, which are divided into ℓ distinct epochs ($U = U_\ell, \dots, U_1$). These epochs are defined such that the number of updates in each epoch decreases geometrically. Specifically, let $\beta = (t_u w)^2 = (\log n)^{\Theta(1)}$ ²⁴ and for the i -th epoch, we make $n_i = \beta^i$ updates. The total number of epochs is $\ell = \frac{1}{10} \log_\beta n = \Theta\left(\frac{\log n}{\log \log n}\right)$.

As in the chronogram method, the partition $S_{\ell+1}, S_\ell, \dots, S_1$ of the memory cells $[s]$ (a random variable), is defined so that $\ell \in S_i$ if cell ℓ was last written by an update performed during epoch i . Locations that were not updated in any of the epochs are assigned to $S_{\ell+1}$.

Following all these updates, we choose a random query $q \in [p]$. Let $P(q)$ be the random variable representing the set of memory locations probed by the solution when answering query q . The original Chronogram technique argued that for a $1 - o(1)$ fraction of the updates, a random query needs to make at least $\Omega(1)$ reads from each color class. That is, for all $i \in [\ell]$, $\mathbb{E}_{U,q}[|P(q) \cap S_i|] = \Omega(1)$.

Larsen strengthened this claim, showing that the expected number of reads from each S_i must be significantly higher, specifically $\tilde{\Omega}(\log n)$. Formally, for all epochs $i \in [\ell]$,

$$\mathbb{E}_{U,q}[|P(q) \cap S_i|] = \Omega\left(\frac{\log p}{\log \frac{sw}{n_i \log p}}\right)$$

Since the total number of epochs is $\ell = \log_\beta n$, this strengthened claim directly leads to an overall $\tilde{\Omega}(\log^2 n)$ lower bound.

The claim is proved by contradiction. Specifically, the existence of an epoch i with small expected $|P(q) \cap S_i|$ will imply an impossible compression of $\mathbb{F}_p^{n_i}$ to fewer than

²³In the chronogram write -up p was chosen to be 2

²⁴as opposed to $\beta = t_u w$ from [FS89].

$n_i \log p$ bits. This part of the proof skillfully leverages ideas from Cell Sampling.

Encoding (1) Suppose there is an epoch $i \in [\ell]$ such that $\mathbb{E}_{U,q}[|P(q) \cap S_i|] = o(\frac{\log p}{\log \frac{sw}{n_i \log p}})$. Let us abbreviate $z = U_i \in \mathbb{F}_p^{n_i}$. Fix a choice of $U^{-i} = U_\ell, \dots, U_{i+1}, U_{i-1}, \dots, U_1$ such that, conditioned on U^{-i} , $\mathbb{E}_{U_i,q}[|P(q) \cap S_i|] \leq o(\frac{\log p}{\log \frac{sw}{n_i \log p}})$. By Markov's inequality (applied twice), some set $\hat{\mathcal{U}}_i \subseteq \mathbb{F}_p^{n_i}$, containing at least a half fraction of all updates, and some set $\mathcal{Q}_1$ containing at least $\frac{1}{2}$ of all the queries, will have $|P(q) \cap S_i| \leq o(\frac{\log p}{\log \frac{sw}{n_i \log p}})$ for every $z \in \hat{\mathcal{U}}_i, q \in \mathcal{Q}_1$.

Now cell sampling comes into play. For each $z \in \hat{\mathcal{U}}_i$, by a probabilistic argument (see [Section 5.3](#)), there exists a set of cells $C_z \subseteq [s]$ of size $|C_z| \leq \frac{n_i \log p}{4w} + o(\frac{\log p}{\log \frac{sw}{n_i \log p}})$, and a set of queries $\mathcal{Q}_2(z) \subseteq [p]$ of size at least $p^{1-o(1)}$, such that for every query $q \in \mathcal{Q}_2(z)$, the decision tree for computing $f(q, x^{(1)})$ (where $x^{(1)}$ is the database after all updates, including $U_i = z$) probes only cells in C_z .

The natural property. Up to this point, the entire argument depends only on the frame. Now we argue in a way which depends on the table. The lower-bound-implying property is as follows. For every epoch i , every choice U^{-i} of updates in other epochs, for every $z = U_i$, and every subset $\mathcal{Q}'$ with n_i queries, knowing the answers $A = (f(q, z))_{q \in \mathcal{Q}'}$ allows us to produce a ("small") set $Z = Z(i, U^{-i}, A)$ of size $|Z| \leq p^{o(n_i)}$ such that $z \in Z$ (i.e., it gives an encoding of z up to $o(n_i \log p)$ bits). We will argue later that this is a natural property. In the case of polynomial evaluation, the property holds with $|Z| = 1$, since, having fixed all of the roots except for the n_i -many roots updated in the i -th epoch, evaluating $p_x(y)$ on any n_i points will further reveal the n_i missing roots (i.e., z). For a random function, it will still be true with slightly larger Z .

Encoding (continued) So assume the table f has this property. We cannot simply choose n_i queries among $\mathcal{Q}_2(z)$ and list them as part of z 's encoding, because this would already cost us $n_i \log p$ bits. So we do a trick. We break the query set $\mathcal{Q} = [p]$ into $\ell = \sqrt{p}$ contiguous blocks of (roughly) equal size $\approx \sqrt{p}$. Since $|\mathcal{Q}_2(z)| \geq p^{1-o(1)}$, there exists a "good" block such that $\mathcal{Q}_2(z)$ still has $p^{\frac{1}{2}-o(1)} \geq n_i$ queries within it. Let $\mathcal{Q}_3(z)$ be n_i queries within this good block. Because all the queries are in the same block, the encoding of $\mathcal{Q}_3(z)$ is now cheaper: we saved $\frac{1}{2} \log p$ bits per query.

So, the encoding of $z \in \mathbb{F}_p^{n_i}$ proceeds as follows:

- If $z \notin \hat{\mathcal{U}}_i$ (it is "expensive"), we encode a bit '0' followed by z itself, requiring $1 + n_i \log p$ bits.
- If $z \in \hat{\mathcal{U}}_i$, its encoding has three parts:

1. $E_1(z)$: Encodes the locations and contents of the cells in C_z . The size is $|E_1(z)| \leq |C_z|(w + \log s) \leq \left(\frac{n_i \log p}{4w} + o(\frac{\log p}{\log \frac{sw}{n_i \log p}}) \right) (w + \log s) = \frac{1+o(1)}{4} n_i \log p$.

2. $E_2(z)$: Encodes the locations and contents of the cells in $S_{<i} = \bigcup_{j=1}^{i-1} S_j$. The size is $|E_2(z)| \leq \sum_{j=1}^{i-1} n_j t_u (w + \log s) < 2\beta^{i-1} t_u (w + \log s) = o(n_i)$.
3. $E_3(z)$: Encodes the set of queries $\mathcal{Q}_3(z)$. The size is $|E_3(z)| \leq \frac{1}{2} \log p + n_i \log \sqrt{p} = \frac{1+o(1)}{2} n_i \log n$.
4. The index of z in the set Z . The size here is $o(n_i \log p)$.

The expected length of the encoding for $z \notin \hat{\mathcal{U}}_i$ is $(\frac{7}{8} + o(1))n_i \log p$, which will contradict the information-theoretic lower bound for encoding $z \in \mathbb{F}_p^{n_i}$.

In the above encoding argument, we have never used that the cells which are modified in each update are determined by way of a read-write decision tree. One again, the *only* property which we use of the updates is that the *number* of modified cells is at most t_u . Hence the entire argument also applies to relaxed solutions.

Decoding The decoding algorithm proceeds as follows: If the first bit of the encoding is '0', it directly outputs z . Otherwise, it reconstructs a partial memory state M' from the encoded C_z and $S_{>i}$ information, assuming the memory state before epoch i (M_{i-1}) is known (this is independent of z , hence allowed). By simulating the queries in $\mathcal{Q}_3(z)$ on M' , the decoder recovers the answers to these queries. Since $\mathcal{Q}_3(z)$ is chosen to uniquely determine z , the original z can then be recovered. Now all that is left to verify is the largeness and constructivity of the property.

Recall that the property corresponding to this proof technique is as follows, For each $i \in [\log_\beta n]$ and for each $\mathcal{U}^{-i}$ and each $\mathcal{Q}' \subseteq \mathcal{Q}$ of size n_i , we consider the map $\phi_{i, \mathcal{U}^{-i}, \mathcal{Q}'} : z \mapsto (f(q, z))_{q \in \mathcal{Q}'}$. We want that $|\phi^{-1}(z)| = p^{o(n_i)}$.

For every epoch i , every choice $\mathcal{U}^{-i}$ of updates in other epochs, for every $z = U_i$, and every subset $\mathcal{Q}'$ with n_i queries, knowing the answers $A = (f(q, z))_{q \in \mathcal{Q}'}$ allows us to produce a ("small") set $Z = Z(i, \mathcal{U}^{-i}, A)$ of size $|Z| \leq p^{o(n_i)}$ such that $z \in Z$ (i.e., it gives an encoding of z up to $o(n_i \log p)$ bits). We will argue later that this is a natural property.

Constructivity: We can compute check if a table f has this property by going over all $(i, \mathcal{U}^{-i}, \mathcal{Q}')$ and doing the following, compute the $\phi_{i, \mathcal{U}^{-i}, \mathcal{Q}'}(z)$, bucket them based on the output and check if the size each of bucket is at most $p^{o(n_i)}$. So the running time of this algorithms is at most $\sum_{i=1}^{\log_\beta n} p^{n-n_i} \binom{|\mathcal{Q}|}{n_i} p^{n_i} = O((p|\mathcal{Q}|)^n) = p^{O(n)}$ when $|\mathcal{Q}| = p$. Hence it is polynomial in the size of the table $p^n |\mathcal{Q}|$.

Largeness: Given a random table f , for a fixed choice of $(i, \mathcal{U}^{-i}, \mathcal{Q}')$ and the map $\phi_{i, \mathcal{U}^{-i}, \mathcal{Q}'}$ is a random map from $\mathbb{F}_p^{n_i} \rightarrow \mathbb{F}_p^{n_i}$. Bounding the maximum size of the pre-image reduces to the follows balls and bins problem where have r balls and r bin and each ball is thrown into a random bin independently and quantity that we are interested in is the *maximum load* which is size of the bin with most number of balls.

The probability that a particular bin $\geq k$ balls is $\binom{r}{k} \frac{1}{r^k} \leq \frac{1}{k!}$. The probability that there exists a bin with at least k balls $\leq \frac{r}{k!} = 2^{\log r - \omega(k \log k)}$. Therefore for $k = \Omega(\log r / \log \log r)$, this quantity becomes $r^{-\Omega(1)}$.

This tells us that whp the map $\phi_{i, \mathcal{M}^{-i}, \mathcal{Q}'}$ is k to 1 for $k = O(n_i \log p / \log n_i)$.

Our candidate PRF. The proof is analogous to the proof that our PRF fools the natural property of [Section 7.2](#). In this setting, p plays the role of $|G|$ and n_i plays the role of L . Otherwise the argument is identical.

7.6 Chronogram + Weak One-way Simulation

Even after the breakthrough results of Larsen, for several years no technique was known for proving super-logarithmic lower-bounds against dynamic data-structure problems with Boolean answers. Eventually Larsen, Weinstein and Yu [[LWY18](#)] developed a technique which we are calling the *Chronogram + weak One-way simulation* technique. It combines the *Chronogram* and *Cell Sampling* technique, as in [Section 7.5](#), with an additional ingredient called the *Peak-to-Average lemma*. This technique is used to prove a lower bound of $\tilde{\Omega}(\log^{3/2} n)$ on the update and query time for the following dynamic data structure problems (all having boolean answers):

- (1) Boolean Polynomial Evaluation [[LWY18](#)]
- (2) Parity searching in Butterfly graphs [[LWY18](#)]
- (3) 0-XOR in multiple butterflies [[LY23](#)]

By reduction from the somewhat artificial problems (2) and (3), lower-bounds were proven for the more natural problems of 2D rectangle stabbing, Range selection and Range median [[LWY18](#)], and Dynamic Reachability [[LY23](#)].

The lower-bounds for problems (1-3), themselves proceed by a series of reductions. From each of the above dynamic data structure problems $\mathcal{P}$ we can define, via a chronogram-type argument, a certain one-way communication problem, denoted $G_{\mathcal{P}}^i$ (there will be one such problem for each epoch i). One then shows that an efficient data structure for $\mathcal{P}$ implies the existence of a one-way protocol which solves $G_{\mathcal{P}}^i$ with a large advantage over random guessing. This part of the proof is technically difficult, but it ultimately depends only on the frame, and applies to relaxed solutions, and so we will not describe it in detail. The second part of the lower-bound proof proceeds by showing that, if the table for the problem satisfies a certain property, the game $G_{\mathcal{P}}^i$ cannot be solved with large advantage, and this establishes the lower bound. Our task here will be to show how this second part of the proof naturalizes, i.e., the property is constructive and holds for a random table over the same frames.

The lower-bound for Boolean polynomial evaluation

We will overview the lower-bound for Boolean polynomial evaluation [LWY18], and after we will say a few words on how it may be generalized to problems (2) and (3).

Frame and table. Boolean polynomial evaluation only differs from polynomial evaluation (which we saw in Section 7.5) in that the field size p is assumed to be a power of 2, and each query $q \in \mathbb{F}_p$ should output the last bit of the polynomial evaluation $p_x(q)$, instead of the entire field element. Otherwise the frame is the same.

Chronogram Setup. We begin by assuming the existence of a solution that solves the problem using worst-case update time t_u and expected query time t_q . The technique initially follows the Chronogram method: we consider what happens to the data structure as we make a sequence of random updates $U = U_\ell \dots U_1$ in epochs of geometrically decreasing sizes, followed by a random query q . To define the epochs, we split the n roots of p into sets $U_\ell, \dots, U_1$, for some $\ell = \Theta(\frac{\log n}{\log \log n})$, so that U_i contains β^i -many roots, with $\beta = (\log n)^{\Theta(1)}$. The bitlength of U_i is thus $n_i = \beta^i \log p$.

The game $G_{\mathcal{P}}^i$. For each epoch $i \in [\ell]$, a one-round communication game $G_{\mathcal{P}}^i$ is defined as follows. Alice gets all random updates $U = U_\ell, \dots, U_1$, while Bob gets the updates in all but the i -th epoch (U^{-i}) and a random query $q \in \mathcal{Q}$. Only a one-round message from Alice to Bob is allowed, and, after receiving it, Bob is expected to output the correct answer $f(q, U)$.

The advantage $\text{adv}(G_{\mathcal{P}}^i, C)$. So far the setup is similar to Section 7.5. We can think of that earlier result as a one-way protocol where the goal is for Bob to recover U_i entirely, given Alice's message. But now, instead, Bob wishes to output $f(q, U)$ for a randomly chosen q . Already, in principle, this could be a lot easier. But furthermore, we do not require that Bob is always correct. Instead, we want to study the "advantage", namely with how much probability over $1/2$ can Bob can output the correct answer? Formally, given a bound C on the number of bits sent from Alice to Bob and a one-way protocol π which communicates at most C bits, if the output of π is correct with probability $1/2 + \epsilon$ (where the probability is over the choice of U and q), then the advantage of the protocol is $\text{adv}_\pi(G_{\mathcal{P}}^i, C) = \epsilon$. The advantage of the game, $\text{adv}(G_{\mathcal{P}}^i, C)$, is defined as the largest advantage possible for any C -bit protocol π .

Weak One-way Simulation. A dynamic data structure lower bound is proven by obtaining a one-way protocol with bounded communication for the epoch game $G_{\mathcal{P}}^i$ by using the dynamic data structure that solves problem $\mathcal{P}$, and showing that this protocol has high advantage. This part of the proof depends only on the frame of the problem $\mathcal{P}$, and is independent of the specific table f . Formally stated:

Theorem 7.1 (Weak One-way Simulation, see Theorem 1 in Section 4 of [LWY18]). Suppose $\mathcal{P}$ admits a dynamic solution with worst-case update time t_u and query time t_q , satisfying $t_q(w \cdot t_u)^{O(1)} \leq n^{1/2}$. Suppose we do a sequence $U_\ell \dots U_1$ of n updates, where each epoch U_i does $n_i = |U_i| = \beta^i$ updates, with $\beta = (w \cdot t_u)^a$ for some constant $a \geq 2$. And suppose that these updates are followed by a single random query $q \in \mathcal{Q}$.

Then there exists some epoch $i \in [\ell/2, \ell]$ for which

$$\text{adv} \left(G_{\mathcal{P}}^i, \frac{n_i}{(w \cdot t_u)^{a-1}} \right) \geq \exp \left(-\tilde{O} \left(\frac{t_q}{\sqrt{\log n}} \right) \right),$$

where the $\tilde{O}$ in the exponent hides $\text{polylog}(w \cdot t_u)$ factor.

I.e., existence of a solution to $\mathcal{P}$ implies existence of a solution to $G_{\mathcal{P}}^i$ with communication $o(|U_i|)$, and advantage roughly $\exp(-t_q / \sqrt{\log n})$. (By choosing a appropriately we can make the communication $\frac{|U_i|}{\text{polylog}(n)}$ for any polylog factor we would like.)

This theorem is **also true of relaxed solutions**: the only assumption about the updates which is used in the proof is that they only change t_u cells, there is no requirement that these cells are actually changed using a read-write decision tree.

Low Advantage via Quasi-independence. The lower bound is then completed by showing that no one-way protocol for $G_{\mathcal{P}}^i$, which communicates $o(|U_i|)$ bits, can have an advantage greater than $n^{-\varepsilon}$, for some $\varepsilon > 0$. Together with Theorem 7.1, this implies that $t_q \geq \tilde{\Omega}((\log n)^{3/2})$.

This is the only part of the proof that depends on the problem's table, and it can be naturalized. By analyzing the proof in [LWY18], we will define a certain property of the table, which we will call Γ -closeness to k -wise quasi-independence, which is enough to carry out the lower bounds in that paper, and which can be shown to hold for random tables. To cover the lower-bounds for problems (2) and (3), we will later extract a slightly different property to the same effect.

Moment Norms. Given a random variable $X = (X_q)_{q \in \mathcal{Q}} \in [-1, 1]^{\mathcal{Q}}$ and a family $\mathcal{F} \subseteq \mathcal{Q}^k$ of multi-sets of size at most k , we define the $\mathcal{F}$ -moment norm denoted by $\|X\|_1^{\mathcal{F}}$ as follows

$$\|X\|_1^{\mathcal{F}} = \frac{1}{|\mathcal{F}|} \cdot \sum_{(q_1, \dots, q_k) \in \mathcal{F}} \left| \mathbb{E} \left[\prod_{j \in [k]} X_{q_j} \right] \right| \in [0, 1]$$

Γ -closeness to k -wise quasi-independence. Given a real valued random variable $X = (X_q)_{q \in \mathcal{Q}}$, X and $\Gamma \in [0, 1]$, X is said to be Γ -close to k -wise quasi-independence if

$$\|X\|_1^{\mathcal{Q}^k} = \frac{1}{|\mathcal{Q}|^k} \cdot \sum_{(q_1, \dots, q_k) \in \mathcal{Q}^k} \left| \mathbb{E} \left[\prod_{j \in [k]} X_{q_j} \right] \right| \leq \Gamma$$

The Property. Let $z = (i, U^{-i})$ denote the choice of an epoch i and a setting of all the updates U^{-i} outside of epoch i . Define the $\{-1, 1\}^{|\mathcal{Q}|}$ -valued random variable $X^z = (\mathcal{P}(q, U))_{q \in \mathcal{Q}} \mid U^{-i} = z$ as the sequence of query answers obtained by uniformly picking updates for the i -th epoch, U_i .²⁵ The property we need for the proof is that, for every i , for every z , X^z is Γ -close to k -wise quasi-independence, where $k = \Theta(n_i)$ and $\Gamma \leq n^{-\Omega(k)}$.

Upper bound on advantage. Let π be a protocol that sends a message m , and let $\chi_m \in \{\pm 1\}^{\mathcal{Q}}$ be the set of Bob's answers for different q if he receives message m . Let us suppose that π achieves an advantage of ϵ . Then we have $\mathbb{E}_{U_i}[|\langle \psi_z(U_i), \chi_{\pi(U_i)} \rangle|] = 2\epsilon|\mathcal{Q}|$, where $\psi_z(U_i) = (\mathcal{P}(q, U))_{q \in \mathcal{Q}}$ is the vector of query answers for a given U_i and fixed $U^{-i} = z$.

Claim 7.2. *If the protocol π has advantage ϵ , i.e. $\mathbb{E}_{U_i}[|\langle \psi_z(U_i), \chi_{\pi(U_i)} \rangle|] = 2\epsilon|\mathcal{Q}|$, then there exists a message $\tilde{\pi}$ such that*

1. $\mathbb{E}_{U_i}[\langle \psi_z(U_i), \chi_{\pi(U_i)} \rangle \mid \pi(U_i) = \tilde{\pi}] \geq \epsilon|\mathcal{Q}|$
2. $\Pr_{U_i}[\pi(U_i) = \tilde{\pi}] \geq |\text{supp}(\pi)|^{-1}\epsilon$

(This follows via an averaging argument: we first restrict to messages with mass at least $|\text{supp}(\pi(U_i))|^{-1}\epsilon$. In doing so the expected value of $|\langle \psi_z(U_i), \chi_{\pi(U_i)} \rangle|$ restricted such heavy messages still remains above $\epsilon|\mathcal{Q}|$. Therefore by averaging, there must be a message which achieves this advantage.)

Using this claim, we can upper-bound the advantage by considering the k -th moment of the dot product $\langle \psi_z(U_i), \chi_{\tilde{\pi}} \rangle$ for $\tilde{\pi}$ given by Claim 7.2.

$$\begin{aligned} \mathbb{E}_{U_i}[\langle \psi_z(U_i), \chi_{\tilde{\pi}} \rangle^k] &= \mathbb{E}_{U_i} \left[\left(\sum_{q \in \mathcal{Q}} \mathcal{P}(q, U_i) \chi_{\tilde{\pi}}(q) \right)^k \right] \\ &= \sum_{q_1, \dots, q_k \in \mathcal{Q}} \left(\prod_{j=1}^k \chi_{\tilde{\pi}}(q_j) \right) \mathbb{E}_{U_i} \left[\prod_{j=1}^k \mathcal{P}(q_j, U_i) \right] \\ &\leq \sum_{q_1, \dots, q_k \in \mathcal{Q}} \left| \mathbb{E}_{X^z} \left[\prod_{j=1}^k X^z(q_j) \right] \right| \leq \Gamma |\mathcal{Q}|^k \end{aligned}$$

The last inequality follows from the property of the table that X^z is Γ -close to k -wise

²⁵Note that we are viewing the entries of the table as coming from $-1, 1$ by the standard transformation of $0 \mapsto 1, 1 \mapsto -1$.

quasi independence. By combining this with the claim, and assuming k is even, we get:

$$\begin{aligned}
\Gamma |\mathcal{Q}|^k &\geq \mathbb{E}_{U_i}[\langle \psi_z(U_i), \chi_{\tilde{\pi}} \rangle^k] \\
&\geq \mathbb{E}[\langle \psi_z(U_i), \chi_{\tilde{\pi}} \rangle^k | \pi(U_i) = \tilde{\pi}] \Pr[\pi(U_i) = \tilde{\pi}] \\
&\geq \mathbb{E}[\langle \psi_z(U_i), \chi_{\tilde{\pi}} \rangle^k | \pi(U_i) = \tilde{\pi}]^k \Pr[\pi(U_i) = \tilde{\pi}] \\
&\geq (\epsilon |\mathcal{Q}|)^k \cdot \frac{\epsilon}{|\text{supp}(\pi)|}
\end{aligned}$$

Rearranging, we get an upper bound on the advantage:

$$\epsilon \leq (\Gamma \cdot |\text{supp}(\pi)|)^{1/(k+1)}$$

Since the protocol is short, $|\text{supp}(\pi)| \leq 2^{o(|U_i|)} = 2^{o(k)}$, and since $\Gamma \leq n^{-\Omega(k)}$ is small, the advantage ϵ is ultimately $\leq n^{-\Omega(1)}$, as intended.

Boolean Polynomial evaluation. In the case of polynomial evaluation, $|\mathcal{Q}| = p$ and $n = \sqrt{p}$. The Boolean polynomial evaluation satisfies the property that the random variable X^z is k -wise independent, with $k = \beta^i = \frac{n_i}{\log p}$, for every choice $z = (i, U^{-i})$.

Independence vs Quasi-independence. Let us show that, if a random variable $X \in \{-1, +1\}^{\mathcal{Q}}$ is k -wise independent, then it is Γ -close to k -wise independent for $\Gamma = \left(\frac{k}{|\mathcal{Q}|}\right)^{k/2}$. For our choice of parameters we have $\Gamma = n^{-\Omega(k)}$, as desired.

Observe that if X is indeed k -wise independent, we know that $\mathbb{E}[\prod_{q \in S} X_q] = 0$ for all subsets $S \subseteq \mathcal{Q}$ of size at most k . Therefore, if the X_q are $\{-1, 1\}$ -valued random variables, the term corresponding to the tuple $(q_1, \dots, q_k)$ survives and evaluates to 1 in absolute value only if all indices in the tuple appear an even number of times.

Therefore, the sum $\sum_{(q_1, \dots, q_k) \in \mathcal{Q}^k} |\mathbb{E}[\prod_{j \in [k]} X_{q_j}]|$ is upper bounded by the size of the set $\{(q_1, \dots, q_k) | \text{each } q_j \text{ appears even number of times}\}$. The size of this set is $\leq |\mathcal{Q}|^{k/2} (k/2)! \leq (|\mathcal{Q}|k)^{k/2}$. Therefore Γ in this case is at most $\left(\frac{k}{|\mathcal{Q}|}\right)^{k/2}$.

Constructivity. For each $z = (i, U^{-i})$, $\|X^z\|_1^{\mathcal{Q}^k}$ can be computed in time $\tilde{O}(|\mathcal{Q}|^k \cdot 2^{|U_i|})$. Since $|\mathcal{Q}| = n^{O(1)}$ and $k = n_i \leq n / \log n$, this is $2^{O(|U|)}$. The number of possibilities for z is $O(2^{|U|})$. Therefore, the property can be decided in time $2^{O(U)}$, i.e., polynomial in the size of the table.

Largeness. If the table is random, since each row is chosen independently, we have independence across the coordinates of X^z . These are not exactly unbiased. Let us show, however, that with high probability over the choice of f , every $z = (i, U^{-i})$ and every

query q will result in the random variable X_q^z (whose randomness is given by the random choice of U_i) having small bias.

So fix some choice of z, q . Let $k = n_i$ be the degree of quasi-independence. The values $(X_q^z(U_i))_{U_i} \in \{-1, +1\}^{2^{n_i}}$ correspond to different entries in row q of the table f , and these are all independent random bits. By the Chernoff bound, the probability that $|\frac{1}{2^{n_i}} \sum_{U_i} x_q^z(U_i)|$ is $> 2^{-\frac{1}{4}n_i}$ is $\exp(-2^{n_i/2})$. This is much less than the number of z and q , so by a union bound this bad event never happens, and all tuples $(X_q^z(U_i))_{U_i}$ have bias $\leq 2^{-\frac{1}{4}n_i}$.

With this, we can analyze the total moment sum $S = \sum_{q_1, \dots, q_k \in \mathcal{Q}} |\mathbb{E}_{U_i} [\prod_{j=1}^k X^{z_j}(q_j)]|$ by partitioning the tuples:

1. **Even Counts:** Tuples where every query appears an even number of times. As we have already seen, the number of such tuples is $\leq (|\mathcal{Q}|k)^{k/2}$, each contributing at most 1 to the sum.
2. **Odd/Mixed Counts:** Tuples where at least one query q_{j^*} appears an odd number of times. Since each row of the table is picked independently in a random table, we have independence across the queries in the random variable X^z . Therefore, $\mathbb{E}_{U_i} \left[\prod_{j \in [k]} X^z(q_j) \right] = \mathbb{E}_{U_i} [X^z(q_{j^*})] \mathbb{E}_{U_i} \left[\prod_{j \in [k] \setminus \{j^*\}} X^z(q_j) \right] \leq 2^{-\frac{1}{4}n_i}$

Combining these, we conclude that $\Gamma = 2^{-\frac{1}{4}n_i} + (\frac{k}{|\mathcal{Q}|})^{k/2} = p^{-k/4} + (\frac{k}{|\mathcal{Q}|})^{k/2} = n^{-\Omega(k)}$, as desired.

Our PRF. As we have just seen, the property used in the proof is natural. We believe that our PRF will fool it, and that this should follow from [Theorem 6.10](#) as before, but we have not been able to show this, yet.

The lower-bound against problems (2) and (3)

We do not wish to go into the technical details of problems (2) and (3). These problems have the additional difficulty that it is not true, for just any choice of k queries, that the outputs $f(q_1, x), \dots, f(q_k, x)$ are quasi-independent when x is chosen at random. However, it can be shown that there is a *large family* $\mathcal{F} \subseteq \mathcal{Q}^k$ of k -tuples of queries, such that any tuple $(q_1, \dots, q_k) \in \mathcal{F}$ will give quasi-independent outputs $f(q_1, x), \dots, f(q_k, x)$ for random x . So the hardness-implying property for problem (2) will be that the random variable $X = f(q_1, x), \dots, f(q_{|\mathcal{Q}|}, x) \in \{+1, -1\}^{|\mathcal{Q}|}$, which outputs all the queries for a random x , has small $\|X\|_1^{\mathcal{F}}$ for this particular family $\mathcal{F}$. Problem (3) adds the additional technical difficulty that $f(q, x)$ is no longer an unbiased bit (for every q and a random x). To deal with this, the definition of X is changed so that $X_q \in \{-\varepsilon_q, 1 - \varepsilon_q\}$, where $\varepsilon_q = \Pr_x[f(q, x) = 1]$ is the bias of query q . Under this transformation, we have $\mathbb{E}_x[X_q] = 0$ once again, and we can work with X_q as if it were unbiased. The proof then proceeds similarly as for problem (2).

A k -wise independent LDPC code

Theorem A.1. $N(s, k, t) \geq \exp((1 - o(1))t)$ whenever $s = \omega(kt)$.

Proof. Suppose $s \geq c \cdot kt$ for some $c = \omega(1)$. We choose a binary $N \times s$ matrix M by letting $p = \frac{t}{s} = \frac{1}{ck}$, and setting each entry of M to 1 with probability p .

For a fixed $j \in \{1, \dots, k\}$, and for a fixed set J of j rows, the probability that a particular column has Hamming weight 1 in the rows J is exactly

$$jp(1-p)^{j-1} > jp(1-jp) > jp \left(1 - \frac{1}{c}\right).$$

(On the last inequality we used $j \leq k$.) Thus, the probability that no column has Hamming weight 1 in the rows J is at most

$$\left(1 - jp \left(1 - \frac{1}{c}\right)\right)^s \leq \exp\left(-jps \left(1 - \frac{1}{c}\right)\right) = \exp\left(-jt \left(1 - \frac{1}{c}\right)\right).$$

Taking a union bound over all sets of j rows, the probability that there is a set J of j rows that has no column with exactly one 1 is at most

$$N^j e^{-jt(1-1/c)} = (Ne^{-t(1-1/c)})^j.$$

Summing over all $j \geq 1$ this is at most $\frac{Ne^{-t(1-1/c)}}{1 - Ne^{-t(1-1/c)}}$ which will be small provided that $N \ll e^{t(1-1/c)}$.

The expected number of rows with exactly t ones is at least N/t so by Chernoff the chance of seeing fewer than $N/2t$ such rows is small. The output of our procedure, then, is precisely the submatrix containing all rows of Hamming weight t . With high probability this matrix will have at least $N/2t \geq \exp((1 - o(1))t)$ rows.

□

Bibliography

- [ACP08] Alexandr Andoni, Dorian Croitoru, and Mihai Pătraşcu. Hardness of Nearest Neighbor under L-infinity. In *Proceedings of the 49th FOCS*, pages 424–433. IEEE, 2008.
- [AHR98] Stephen Alstrup, Thore Husfeldt, and Theis Rauhe. Marked ancestor problems. In *Proceedings of the 39th FOCS*, pages 534–543. IEEE, 1998.
- [Ajt88] Miklós Ajtai. A lower bound for finding predecessors in Yao’s cell probe model. *Combinatorica*, 8(3):235–247, 1988.
- [AS25] Peyman Afshani and Nodari Sitchinava. A Cell Probe Lower Bound for the Predecessor Search Problem in PRAM. In *Proceedings of the 36th SODA*, pages 3998–4008. SIAM, 2025.
- [AV15] Amirali Abdullah and Suresh Venkatasubramanian. A Directed Isoperimetric Inequality with application to Bregman Near Neighbor Lower Bounds. In *Proceedings of the 47th STOC*, pages 509–518. ACM, 2015.
- [AW14] Amir Abboud and Virginia Vassilevska Williams. Popular conjectures imply strong lower bounds for dynamic problems. In *Proceedings of the 55th FOCS*, pages 434–443. IEEE, 2014.
- [BAG01] Amir M. Ben-Amram and Zvi Galil. A generalization of a lower bound technique due to Fredman and Saks. *Algorithmica*, 30(1):34–66, 2001.
- [Bar18] Siddharth Barman. Approximating nash equilibria and dense subgraphs via an approximate version of carathéodory’s theorem. *SIAM Journal on Computing*, 47(3):960–981, 2018.
- [BBK18] Joseph Boninger, Joshua Brody, and Owen Kephart. Non-Adaptive Data Structure Bounds for Dynamic Predecessor. In *Proceedings of the 37th FSTTCS*, pages 20:1–20:12. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.
- [BCF⁺22] Michael A. Bender, Alex Conway, Martín Farach-Colton, Hanna Komlós, William Kuszmaul, and Nicole Wein. Online List Labeling: Breaking the $\log^2 n$ Barrier. In *Proceedings of the 63rd FOCS*, pages 980–990. IEEE, 2022.
- [BF02] Paul Beame and Faith E. Fich. Optimal bounds for the predecessor problem and related problems. *Journal of Computer and System Sciences*, 65(1):38–72, 2002.
- [BFGK24] Michael A. Bender, Martín Farach-Colton, Michael T. Goodrich, and Hanna Komlós. History-independent dynamic partitioning: Operation-order privacy in ordered data structures. *Proc. ACM Manag. Data*, 2(2):108, 2024.

- [BFS86] László Babai, Peter Frankl, and János Simon. Complexity classes in communication complexity theory (preliminary version). In *Proceedings of the 27th FOCS*, pages 337–347. IEEE, 1986.
- [BGS75] Theodore Baker, John Gill, and Robert Solovay. Relativizations of the $p = ?np$ question. *SIAM Journal on Computing*, 4(4):431–442, 1975.
- [BL15] Joshua Brody and Kasper Green Larsen. Adapt or Die: Polynomial Lower Bounds for Non-Adaptive Dynamic Data Structures. *Theory of Computing*, 11(19):471–489, 2015.
- [BNS19] Jan Van Den Brand, Danupon Nanongkai, and Thatchaphol Saranurak. Dynamic matrix inverse: Improved algorithms and matching conditional lower bounds. In *Proceedings of the 60th FOCS*, pages 456–480. IEEE, 2019.
- [BOR99] Allan Borodin, Rafail Ostrovsky, and Yuval Rabani. Lower bounds for high dimensional nearest neighbor search and related problems. In *Proceedings of the 31st STOC*, pages 312–321. ACM, 1999.
- [BR02] Omer Barkol and Yuval Rabani. Tighter Lower Bounds for Nearest Neighbor Search and Related Problems in the Cell Probe Model. *Journal of Computer and System Sciences*, 64(4):873–896, 2002.
- [BR17] Andrej Bogdanov and Alon Rosen. Pseudorandom functions: Three decades later. In Yehuda Lindell, editor, *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich*, pages 79–158. Springer, 2017.
- [CCGL03] Amit Chakrabarti, Bernard Chazelle, Benjamin Gum, and Alexey Lvov. A Lower Bound on the Complexity of Approximate Nearest-Neighbor Searching on the Hamming Cube. In *Discrete and Computational Geometry: The Goodman-Pollack Festschrift*, pages 313–328. Springer, 2003.
- [CJS13] Raphaël Clifford, Markus Jalsenius, and Benjamin Sach. Tight Cell-Probe Bounds for Online Hamming Distance Computation. In *Proceedings of the 24th SODA*, pages 664–674. SIAM, 2013.
- [CJS16] Raphaël Clifford, Markus Jalsenius, and Benjamin Sach. Cell-Probe Lower Bounds for Bit Stream Computation. In *Proceedings of the 24th ESA*, pages 31:1–31:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016.
- [CKL18] Diptarka Chakraborty, Lior Kamma, and Kasper Green Larsen. Tight cell probe bounds for succinct Boolean matrix-vector multiplication. In *Proceedings of the 50th STOC*, pages 1297–1306. ACM, 2018.
- [CKLM18] Arkadev Chattopadhyay, Michal Koucký, Bruno Loff, and Sagnik Mukhopadhyay. Simulation beats richness: new data-structure lower bounds. In *Proceedings of the 50th STOC*, pages 1013–1020. ACM, 2018.

- [CP10] Arkadev Chattopadhyay and Toniann Pitassi. The story of set disjointness. *ACM SIGACT News*, 41(3):59–85, 2010.
- [CR10] Amit Chakrabarti and Oded Regev. An Optimal Randomized Cell Probe Lower Bound for Approximate Nearest Neighbor Searching. *SIAM Journal on Computing*, 39(5):1919–1940, 2010.
- [CW13] Timothy M. Chan and Bryan T. Wilkinson. Adaptive and approximate orthogonal range counting. In *Proceedings of the 24th SODA*, pages 241–251. SIAM, 2013.
- [DGW19] Zeev Dvir, Alexander Golovnev, and Omri Weinstein. Static data structure lower bounds imply rigidity. In *Proceedings of the 51st STOC*, pages 967–978. ACM, 2019.
- [DL20] Pavel Dvořák and Bruno Loff. Lower bounds for semi-adaptive data structures via corruption. In *Proceedings of the 40th FSTTCS*, pages 20:1–20:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020.
- [DMS03] Ilya Dumer, Daniele Micciancio, and Madhu Sudan. Hardness of approximating the minimum distance of a linear code. *IEEE Transactions on Information Theory*, 49(1):22–37, 2003.
- [FS89] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In *Proceedings of the 21st STOC*, pages 345–354. ACM, 1989.
- [GL16] Allan Grønlund and Kasper Green Larsen. Towards Tight Lower Bounds for Range Reporting on the RAM. In *Proceedings of the 43rd ICALP*, pages 92:1–92:12. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016.
- [GM07] Anna Gál and Peter Bro Miltersen. The cell probe complexity of succinct data structures. *Theoretical Computer Science*, 379(3):405–417, 2007.
- [HKNS15] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. Unifying and Strengthening Hardness for Dynamic Problems via the Online Matrix-Vector Multiplication Conjecture. In *Proceedings of the 47th STOC*, pages 21–30. ACM, 2015.
- [JMS04] Joseph F. JáJá, Christian Worm Mortensen, and Qingmin Shi. Space-efficient and fast algorithms for multidimensional dominance reporting and counting. In *Proceedings of the 15th ISAAC*, pages 558–568. Springer, 2004.
- [Joh87] David S. Johnson. The np-completeness column: An ongoing guide. *Journal of Algorithms*, 8(2):285–303, 1987.

- [KN97] Eyal Kushilevitz and Noam Nisan. *Communication complexity*. Cambridge University Press, 1997.
- [KPI25] Oliver Korten, Toniann Pitassi, and Russell Impagliazzo. Stronger cell probe lower bounds via local prgs. In *Proceedings of the 66th FOCS*. IEEE, 2025. To appear.
- [KRW95] Mauricio Karchmer, Ran Raz, and Avi Wigderson. Super-logarithmic depth lower bounds via the direct sum in communication complexity. *Computational Complexity*, 5(3):191–204, 1995.
- [KW20] Young Kun Ko and Omri Weinstein. An adaptive step toward the multiphase conjecture. In *Proceedings of the 61st FOCS*, pages 752–761. IEEE, 2020.
- [Lar12a] Kasper Green Larsen. The cell probe complexity of dynamic range counting. In *Proceedings of the 44th STOC*, pages 85–94. ACM, 2012.
- [Lar12b] Kasper Green Larsen. Higher Cell Probe Lower Bounds for Evaluating Polynomials. In *Proceedings of the 53rd FOCS*, pages 293–301. IEEE, 2012.
- [LLYZ23] Tianxiao Li, Jingxun Liang, Huacheng Yu, and Renfei Zhou. Tight Cell-Probe Lower Bounds for Dynamic Succinct Dictionaries. In *Proceedings of the 64th FOCS*, pages 1842–1862. IEEE, 2023.
- [LN18] Kasper Green Larsen and Jesper Buus Nielsen. Yes, There is an Oblivious RAM Lower Bound! In *Proceedings of the 38th CRYPTO*, pages 523–542. Springer, 2018.
- [LPP⁺24] Kasper Green Larsen, Rasmus Pagh, Giuseppe Persiano, Toniann Pitassi, Kevin Yeo, and Or Zamir. Optimal Non-Adaptive Cell Probe Dictionaries and Hashing. In *Proceedings of the 51st ICALP*, pages 104:1–104:12. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024.
- [LPPS96] Hanno Lefmann, Pavel Pudlák, and Petr Petr Savický. On sparse parity check matrices. In *Proceedings of the 2nd COCOON*, pages 41–49. Springer, 1996.
- [LPY16] Mingmou Liu, Xiaoyin Pan, and Yitong Yin. Randomized Approximate Nearest Neighbor Search with Limited Adaptivity. In *Proceedings of the 28th SPAA*, pages 23–33. ACM, 2016.
- [LWY18] Kasper Green Larsen, Omri Weinstein, and Huacheng Yu. Crossing the logarithmic barrier for dynamic Boolean data structure lower bounds. In *Proceedings of the 50th STOC*, pages 978–989. ACM, 2018.
- [LY23] Kasper Green Larsen and Huacheng Yu. Super-Logarithmic Lower Bounds for Dynamic Graph Problems. In *Proceedings of the 64th FOCS*, pages 1589–1604. IEEE, 2023.

- [Mil99] Peter Bro Miltersen. Cell probe complexity - a survey. *Advances in Data Structures Workshop, 19th FSTTCS*, 1999.
- [MNSW98] Peter Bro Miltersen, Noam Nisan, Shmuel Safra, and Avi Wigderson. On Data Structures and Asymmetric Communication Complexity. *Journal of Computer and System Sciences*, 57(1):37–49, 1998.
- [Nav25] Gonzalo Navarro. (Worst-Case) Optimal Adaptive Dynamic Bitvectors, 2025. arXiv:2405.15088.
- [NT01] Moni Naor and Vanessa Teague. Anti-persistence: history independent data structures. In *Proceedings of the 33rd STOC*, pages 492–501. ACM, 2001.
- [Păt07] Mihai Pătraşcu. Lower bounds for 2-dimensional range counting. In *Proceedings of the 39th STOC*, pages 40–46. ACM, 2007.
- [Păt09] Mihai Pătraşcu. A Lower Bound for Succinct Rank Queries, 2009. arXiv:0907.1103.
- [Păt10] Mihai Pătraşcu. Towards polynomial lower bounds for dynamic problems. In *Proceedings of the 42nd STOC*, pages 603–610. ACM, 2010.
- [PD06] Mihai Pătraşcu and Erik D. Demaine. Logarithmic Lower Bounds in the Cell-Probe Model. *SIAM Journal on Computing*, 35(4):932–963, 2006.
- [PT06a] Mihai Pătraşcu and Mikkel Thorup. Higher Lower Bounds for Near-Neighbor and Further Rich Problems. In *Proceedings of the 47th FOCS*, pages 646–654. IEEE, 2006.
- [PT06b] Mihai Pătraşcu and Mikkel Thorup. Time-space trade-offs for predecessor search. In *Proceedings of the 38th STOC*, pages 232–240. ACM, 2006.
- [PT07] Mihai Pătraşcu and Mikkel Thorup. Randomization does not help searching predecessors. In *Proceedings of the 18th SODA*, pages 555–564. SIAM, 2007.
- [PT11] Mihai Pătraşcu and Mikkel Thorup. Don’t rush into a union: take time to find your roots. In *Proceedings of the 43rd STOC*, pages 559–568. ACM, 2011.
- [PTW10] Rina Panigrahy, Kunal Talwar, and Udi Wieder. Lower Bounds on Near Neighbor Search via Metric Expansion. In *Proceedings of the 51st FOCS*, pages 805–814. IEEE, 2010.
- [PY22] Giuseppe Persiano and Kevin Yeo. Limits of Preprocessing for Single-Server PIR. In *Proceedings of the 33rd SODA*, pages 2522–2548. SIAM, 2022.
- [Raz92] Alexander A. Razborov. On the distributional complexity of disjointness. *Theoretical Computer Science*, 106(2):385–390, 1992.

- [RR94] Alexander A. Razborov and Steven Rudich. Natural proofs. In *Proceedings of the 26th STOC*, pages 204–213. ACM, 1994.
- [RR18] Sivaramakrishnan Natarajan Ramamoorthy and Anup Rao. Lower Bounds on Non-Adaptive Data Structures Maintaining Sets of Numbers, from Sunflowers. In *Proceedings of the 33rd CCC*, pages 27:1–27:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.
- [RR20] Sivaramakrishnan Natarajan Ramamoorthy and Cyrus Rashtchian. Equivalence of Systematic Linear Data Structures and Matrix Rigidity. In *Proceedings of the 11th ITCS*, pages 35:1–35:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020.
- [RY20] Anup Rao and Amir Yehudayoff. *Communication complexity: and applications*. Cambridge University Press, 2020.
- [RZ04] Liam Roditty and Uri Zwick. A fully dynamic reachability algorithm for directed graphs with an almost linear update time. In *Proceedings of the 36th STOC*, pages 184–191. ACM, 2004.
- [San04] Piotr Sankowski. Dynamic transitive closure via dynamic matrix inverse. In *Proceedings of the 45th FOCS*, pages 509–517. IEEE, 2004.
- [SV08] Pranab Sen and Srinivasan Venkatesh. Lower bounds for predecessor searching in the cell probe model. *Journal of Computer and System Sciences*, 74(3):364–385, 2008.
- [SW19] Sandip Sinha and Omri Weinstein. Local decodability of the Burrows-Wheeler transform. In *Proceedings of the 51st STOC*, pages 744–755. ACM, 2019.
- [Vio19] Emanuele Viola. Lower bounds for data structures with space close to maximum imply circuit lower bounds. *Theory of Computing*, 15(1):1–9, 2019.
- [Wil85] Dan E. Willard. New data structures for orthogonal range queries. *SIAM Journal on Computing*, 14(1):232–253, 1985.
- [WY14] Yaoyu Wang and Yitong Yin. Certificates in Data Structures. In *Proceedings of the 41st ICALP*, pages 1039–1050. Springer, 2014.
- [Yao78] Andrew Chi-Chih Yao. Should tables be sorted? In *Proceedings of the 19th FOCS*, pages 22–27. IEEE, 1978.
- [Yin16] Yitong Yin. Simple Average-Case Lower Bounds for Approximate Near-Neighbor from Isoperimetric Inequalities. In *Proceedings of the 43rd ICALP*, pages 84:1–84:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016.